Introduction

Welcome to the **godot-rust book**! This is a work-in-progress user guide for **gdext**, the Rust binding for Godot 4.

If you're new to Rust, before getting started, it is highly recommended that you familiarize yourself with concepts outlined in the officially maintained Rust Book.

Additional resources that may be interesting for you:

- 📘 Latest API docs
- 🛸 Demo projects
- 🀄 This book in Simplified Chinese
- 📓 Book on gdnative (Godot 3 binding)

The purpose of godot-rust

Godot is a batteries-included game engine that fosters a productive and fun gamedev workflow. It ships GDScript as a built-in scripting language and also provides official support for C++ and C# bindings. Its GDExtension mechanism allows more languages to be integrated, in our case Rust.

Rust brings a modern, robust and performant experience to game development. If you are interested in scalability, strong type systems or just enjoy Rust as a language, you may consider using it with Godot, to combine the best of both worlds.

See also Philosophy to learn more about the core ideas behind godot-rust.

About this project

godot-rust is a community-developed open source project. It is maintained independently of Godot itself, but we are in close contact with engine developers, to foster a steady exchange of ideas. This has allowed us to address a lot of Rust's needs upstream, but also led to improvements of the engine itself in several cases.

Currently supported features

For an up-to-date overview of implementation status, consult issue #24.

Terminology

To avoid confusion, here is an explanation of names and technologies you may encounter over the course of this book:

- **godot-rust**: The Godot 4 Rust bindings, and sometimes also the entire project (book, community, website etc.).
- **gdext** (lowercase): explicitly just the Rust bindings for Godot 4. We prefer the term "godot-rust" which is more recognizable outside this specific ecosystem.
- **gdnative** (lowercase): the Rust binding for GDNative (Godot 3), no longer actively maintained.
- **GDExtension**: C API provided by Godot 4.
- **GDNative**: C API provided by Godot 3.
- **Extension**: An extension is a dynamic C library, developed by any language binding (Rust, C++, Swift, ...). It uses the GDExtension API and can be loaded by Godot 4.

These are *WRONG* terms: `GDRust`, `gdrust`, `godot-rs`.

Getting Started

This chapter guides you through the process of setting up **godot-rust** and developing your first application with it.

> **Note**
>
> To read this book, we assume intermediate Rust knowledge. If you are new to Rust, reading the Rust Book first is highly encouraged. You won't need to know 100% of the language, but you should know basic concepts (type system, generics, traits, borrow checking, safety).
>
> Some familiarity with Godot is also necessary, although it is possible to learn godot-rust together with Godot. However, we won't reiterate basic Godot concepts -- so if you choose that approach, we recommend to read the official Godot tutorial in parallel.

In addition to this book, you can use the following resources to learn more about the project:

- The official API documentation.
- A small example game Dodge the Creeps.

Setup

Before we can start writing Rust code, we need to install a few tools.

Godot Engine

While you can write Rust code without the Godot engine, we highly recommend to install Godot for quick feedback loops. For the rest of the tutorial, we assume that you have Godot 4 installed and available either:

- in your `PATH` as `godot4`,
- or an environment variable called `GODOT4_BIN`, containing the path to the Godot executable.

Godot from pre-built binaries

Binaries of Godot 4 can be downloaded from the official website.
For beta and older versions, you can also check the download archive.

Installing Godot via command-line

```
# --- Linux ---
# Fedora/RHEL.
dnf install godot

# Arch Linux.
pacman -Syu godot
paru -Syu godot

# Flatpak (e.g. Ubuntu, Debian, or distro-independent).
flatpak install flathub org.godotengine.Godot


# --- Windows ---
winget install -e --id GodotEngine.GodotEngine
choco install godot
scoop bucket add extras && scoop install godot


# --- macOS ---
brew install godot
```

> **Other Godot versions**
>
> If you plan to target Godot versions different from the latest stable release, please read Selecting a Godot version.

Rust

rustup is the preferred way to install the Rust toolchain. It includes the compiler, standard library, Cargo (the package manager) as well as tools like rustfmt or clippy. Visit the website to download binaries or installers for your platform. Alternatively, you can install it via command-line.

Installing rustup via command-line

```
# Linux (distro-independent)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# Windows
winget install -e --id Rustlang.Rustup

# macOS
brew install rustup
```

After installation of rustup and the `stable` toolchain, you can verify that they are working:

```
$ rustc --version
rustc 1.88.0 (6b00bc388 2025-06-23)
```

LLVM

> **Tip**
>
> In general, you do **NOT** need to install LLVM.

This was necessary in the past due to `bindgen`, which depends on LLVM. However, we now provide pre-built artifacts, so that most users can simply add the Cargo dependency and start immediately. This also significantly reduces initial compile times, as `bindgen` was quite heavyweight with its many transitive dependencies.

You will still need LLVM if you plan to use the `api-custom` feature, for example if you have a forked version of Godot or custom modules. To just use a different API version of Godot, you do *not* need LLVM though; see Selecting a Godot version.

LLVM binaries can be downloaded from llvm.org. Once installed, you can check whether LLVM's clang compiler is available:

```
clang -v
```

Hello World

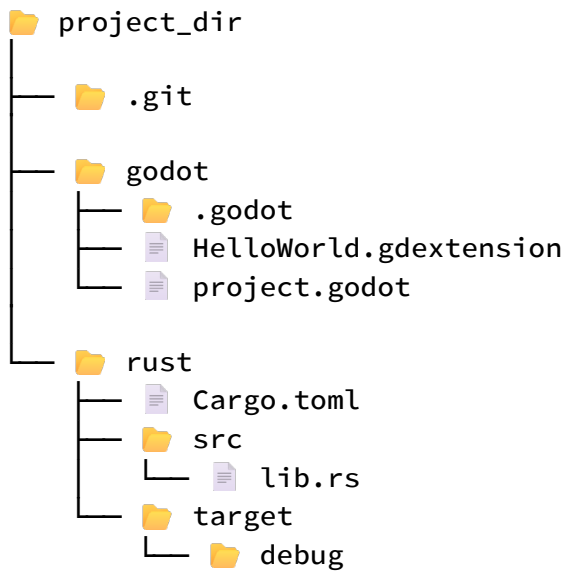
This page shows you how to develop your own small extension library and load it from Godot. The tutorial is heavily inspired by Creating your first script from the official Godot documentation. We recommended to follow that alongside this tutorial, in case you're interested how certain GDScript concepts map to Rust.

Table of contents

- Directory setup
- Create a Godot project
- Create a Rust crate
- Wire up Godot with Rust
  - The `.gdextension` file
  - `extension_list.cfg`
- Your first Rust extension
  - Rust entry point
  - Troubleshooting
- Creating a Rust class
  - Class declaration
  - Method declaration
  - Custom Rust APIs

Directory setup

We assume the following file structure, with separate directories for the Godot and Rust parts:



Create a Godot project

To use godot-rust, you need Godot version of 4.1 or later. Feel free to download the latest stable one. You can download in-development versions, but we do not provide official support for those, so we recommend stable ones.

Open the Godot project manager and create a new Godot 4 project in the `godot/` subfolder. Add a `Sprite2D` to the center of a new scene. We recommend that you follow the Official tutorial and stop at the point where it asks you to create a script.

Run your scene to make sure everything is working. Save the changes and consider versioning each step of the tutorial in Git.

Create a Rust crate

To make a new crate with cargo, open your terminal, navigate to your desired folder and then type:

```
cargo new "{YourCrate}" --lib
```

where `{YourCrate}` will be used as a placeholder for a crate name of your choice. To fit with the file structure, we choose `rust` as the crate name. `--lib` is used to create a library (not an executable), but there is some extra configuration that the crate requires.

Open `Cargo.toml` and modify it as follows:

```
[package]
name = "rust_project" # Part of dynamic library name; we use {YourCrate}
placeholder.
version = "0.1.0"     # You can leave version and edition as-is for now.
edition = "2021"

[lib]
crate-type = ["cdylib"]  # Compile this crate to a dynamic C library.
```

The `cdylib` crate type is not very common in Rust. Instead of building an application (`bin`) or a library to be utilized by other Rust code (`lib`), we create a *dynamic* library, exposing an interface in the C programming language. This dynamic library is loaded by Godot at runtime, through the GDExtension interface.

Now add godot-rust to your project with:

```
cargo add godot
```

To compile each iteration of the extension as you write code, you can use `cargo` as you normally do with any other Rust project:

```
cargo build
```

This should output to `{YourCrate}/target/debug/` at least one variation of a compiled library depending on your setup.

> **Tip**
>
> If you want to follow bleeding-edge development (with the associated risks), you can directly link to the GitHub repo in the `[dependencies]` section of your Cargo.toml. For this, replace:
>
> ```
> godot = "0.x.y"
> ```
>
> with:
>
> ```
> godot = { git = "https://github.com/godot-rust/gdext", branch = "master" }
> ```

Wire up Godot with Rust

The `.gdextension` file

This file tells Godot how to load your compiled Rust extension. It contains the path to the dynamic library, as well as the entry point (function) to initialize it with.

First, add an empty `.gdextension` file anywhere in your `godot` subfolder. In case you're familiar with Godot 3, this is the equivalent of `.gdnlib`. In this case, we create `res://HelloWorld.gdextension` inside the `godot` subfolder and fill it as follows:

```
[configuration]
entry_symbol = "gdext_rust_init"
compatibility_minimum = 4.1
reloadable = true

[libraries]
linux.debug.x86_64 =      "res://../rust/target/debug/lib{YourCrate}.so"
linux.release.x86_64 =    "res://../rust/target/release/lib{YourCrate}.so"
windows.debug.x86_64 =    "res://../rust/target/debug/{YourCrate}.dll"
windows.release.x86_64 =  "res://../rust/target/release/{YourCrate}.dll"
macos.debug =             "res://../rust/target/debug/lib{YourCrate}.dylib"
macos.release =           "res://../rust/target/release/lib{YourCrate}.dylib"
macos.debug.arm64 =       "res://../rust/target/debug/lib{YourCrate}.dylib"
macos.release.arm64 =     "res://../rust/target/release/lib{YourCrate}.dylib"
```

The `[configuration]` section should be copied as-is.

- Key `entry_symbol` refers to the entry point function that **godot-rust** exposes. We choose `"gdext_rust_init"`, which is the library's default (but can be configured if needed).
- Key `compatibility_minimum` specifies the minimum version of **Godot** required by your extension to work. Opening the project with a version of Godot lower than this will prevent your extension from running.
    - If you build a plugin to be used by others, set this as low as possible for maximum ecosystem compatibility. This might however limit the features you can use.
- Key `reloadable` specifies that the editor should reload the extension when the editor window loses and regains focus. See Godot issue #80284 for more details.
    - If Godot is crashing, you may want to try turning off or removing this setting.

The `[libraries]` section should be updated to match the paths of your dynamic Rust libraries.

- The keys on the left are the build targets of the **Godot** project.
  - Consult GDExtension docs for more possible values.
- The values on the right are the file paths to your dynamic library.
  - The `res://` prefix represents the path to files **relative to your Godot directory**, regardless of where your `HelloWorld.gdextension` file is. You can learn more about Godot's resource paths here.
  - If you remember the file structure, the `godot` and `rust` directories are siblings, so we need to go up one level to reach `rust`.
- You can add configurations for as many platforms as you like, if you plan to export your project to those later. At the very least, you need to have your current OS in `debug` mode.

> **Tip**
>
> You can also employ the use of symbolic links and git submodules and then treat those as regular folders and files. Godot reads those just fine too!

> **Export paths**
>
> When exporting your project, you need to use paths *inside* `res://`.
> Outside paths like `..` are not supported.

> **Custom Rust targets**
>
> If you specify your cargo compilation target via the `--target` flag or a `.cargo/config.toml` file, the rust library will be placed in a path name that includes target architecture, and the `.gdextension` library paths will need to match. For example, for M1 Macs (`macos.debug.arm64` and `macos.release.arm64`), the path would be `"res://../rust/target/aarch64-apple-darwin/debug/lib{YourCrate}.dylib"`.

`extension_list.cfg`

A second file `res://.godot/extension_list.cfg` should be generated once you open the Godot editor for the first time. This file lists all extension registered within your project. If the file does not exist, you can also manually create it, simply containing the Godot path to your `.gdextension` file:

```
res://HelloWorld.gdextension
```

Your first Rust extension

> **.gdignore**
>
> If you do not follow the recommended gdext project directory setup of having separate `rust/` and `godot/` directories and instead place your rust source directly within your Godot project, then please consider adding a .gdignore file at the root folder of your Rust code. This avoids cases where the Rust Compiler may produce a file in your rust folder with an ambiguous extension such as `.obj`, which the Godot Editor may inappropriately attempt to import, resulting in an error and preventing you from building your project.

Rust entry point

As mentioned earlier, our compiled C library needs to expose an *entry point* to Godot: a C function that can be called through the GDExtension. Setting this up requires quite some low-level FFI code, which godot-rust abstracts for you.

In your `lib.rs`, replace the template with the following:

```
use godot::prelude::*;

struct MyExtension;

#[gdextension]
unsafe impl ExtensionLibrary for MyExtension {}
```

There are multiple things going on here:

1. Place the `prelude` module from the `godot` crate into scope. This module contains the most common symbols in the godot-rust API.
2. Define a struct called `MyExtension`. This is just a type tag without data or methods, you can name it however you like.
3. Implement the `ExtensionLibrary` trait for our type, and mark it with the `#[gdextension]` attribute.

The last point declares the actual GDExtension entry point, and the proc-macro attribute

takes care of the low-level details.

Troubleshooting

It's common that there are some issues with first-time setup. Particularly, errors related to the library not being found or the `gdext_rust_init` entry point symbol being missing or impossible to resolve come up, usually due to an incorrect initial setup. Here are a few troubleshooting steps that should solve the most common problems.

- Have you run `cargo build`?
- In `Cargo.toml`, have you set `crate-type = ["cdylib"]`?
- Does Godot crash immediately on opening your project? Run `godot4 path/to/project.godot` from the terminal to see the error output.
- In `my-extension.gdextension`, have you set `entry_symbol = "gdext_rust_init"`? No other symbol can work.
- Are the paths set in `my-extension.gdextension` correct?
  - Are you sure? Double check `/rust/target/debug/` to see if the name of the `.so` / `.dll` / `.dylib` is spelled the way you expect.
  - The paths must also be relative to the directory that `project.godot` is in. Typically it'll be `res://../rust/...`.
- Have you written the Rust code necessary to generate the entry point symbol?
  - See above for how.
- Are your godot-rust and Godot versions compatible? See this page for how to select the correct versions.
- Did you try clearing the Godot cache, i.e. deleting all files in the `.godot` folder except for `extension_list.cfg`?
- In case you use `api-custom`, do you have
  - Godot in your `PATH` as `godot4`,
  - or an environment variable called `GODOT4_BIN`, containing the path to the Godot executable?
- Is your directory structure like this below? It's much easier when you ask for help if it is.

```
my-cool-project
├── godot
│   ├── project.godot
│   └── my-extension.gdextension
└── rust
    ├── Cargo.toml
    ├── src
    └── target
        └── debug
            └── (lib)?my_extension.(so|dll|dylib)
```

Creating a Rust class

Now, let's write Rust code to define a *class* that can be used in Godot.

Every class inherits an existing Godot-provided class (its *base class* or just *base*). Rust does not natively support inheritance, but the godot-rust API emulates it to a certain extent.

Class declaration

In this example, we declare a class called `Player`, which inherits `Sprite2D` (a node type). This can be either defined in `lib.rs` or in a separate file `player.rs`. In case you go for the latter, don't forget to declare `mod player;` in your `lib.rs` file.

```
use godot::prelude::*;
use godot::classes::Sprite2D;

#[derive(GodotClass)]
#[class(base=Sprite2D)]
struct Player {
    speed: f64,
    angular_speed: f64,

    base: Base<Sprite2D>
}
```

Let's break this down.

1. The `godot` prelude contains the most common symbols. Less frequent classes are located in the `engine` module.

2. The `#[derive]` attribute registers `Player` as a class in the Godot engine. See API docs

for details about `#[derive(GodotClass)]`.

3. The optional `#[class]` attribute configures how the class is registered. In this case, we specify that `Player` inherits Godot's `Sprite2D` class. If you don't specify the `base` key, the base class will implicitly be `RefCounted`, just as if you omitted the `extends` keyword in GDScript.

4. We define two fields `speed` and `angular_speed` for the logic. These are regular Rust fields, no magic involved. More about their use later.

5. The `Base<T>` type is used for the `base` field, which allows `self` to access the base instance (via composition, as Rust does not have native inheritance). This enables two methods that can be accessed as `self.base()` and `self.base_mut()` on your type (through an extension trait).

    - `T` must match the declared base class. For example, `#[class(base=Sprite2D)]` implies `Base<Sprite2D>`.
    - The name can be freely chosen, but `base` is a common convention.
    - You do not *have to* declare this field. If it is absent, you cannot access the base object from within `self`. This is often not a problem, e.g. in data bundles inheriting `RefCounted`.

> ⚠ **Correct node type**
>
> When adding an instance of your `Player` class to the scene, make sure to select node type `Player` **and not its base** `Sprite2D`. Otherwise, your Rust logic will not run. We will guide you to make that change to your scene later, when you're ready to test it.
>
> If Godot fails to load a Rust class (e.g. due to an error in your extension), it may silently replace it with its base class. Use version control (git) to check for unwanted changes in `.tscn` files.

Method declaration

Now let's add some logic. We start with overriding the `init` method, also known as the constructor. This corresponds to GDScript's `_init()` function.

```
use godot::classes::ISprite2D;

#[godot_api]
impl ISprite2D for Player {
    fn init(base: Base<Sprite2D>) -> Self {
        godot_print!("Hello, world!"); // Prints to the Godot console

        Self {
            speed: 400.0,
            angular_speed: std::f64::consts::PI,
            base,
        }
    }
}
```

Again, those are multiple pieces working together, let's go through them one by one.

1. `#[godot_api]` - this lets godot-rust know that the following `impl` block is part of the Rust API to expose to Godot. This attribute is required here; accidentally forgetting it will cause a compile error.

2. `impl ISprite2D` - each of the engine classes has a `I{ClassName}` trait, which comes with virtual functions for that specific class, as well as general-purpose functionality such as `init` (the constructor) or `to_string` (String conversion). The trait has no required methods.

3. The `init` constructor is an associated function ("static method" in other languages) that takes the base instance as argument and returns a constructed instance of `Self`. While the base is usually just forwarded, the constructor is the place to initialize all your other fields. In this example, we assign initial values `400.0` and `PI`.

Now that initialization is sorted out, we can move on to actual logic. We would like to continuously rotate the sprite, and thus override the `process()` method. This corresponds to GDScript's `_process()`. If you need a fixed framerate, use `physics_process()` instead.

```
use godot::classes::ISprite2D;

#[godot_api]
impl ISprite2D for Player {
    fn init(base: Base<Sprite2D>) -> Self { /* as before */ }

    fn physics_process(&mut self, delta: f64) {
        // In GDScript, this would be:
        // rotation += angular_speed * delta

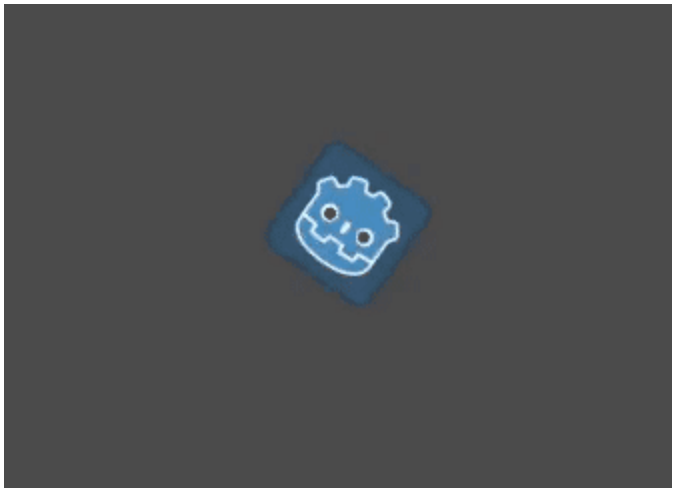
        let radians = (self.angular_speed * delta) as f32;
        self.base_mut().rotate(radians);
        // The 'rotate' method requires a f32,
        // therefore we convert 'self.angular_speed * delta' which is a f64 to
a f32
    }
}
```

GDScript uses property syntax here; Rust requires explicit method calls instead. Also, access to base class methods -- such as `rotate()` in this example -- is done via `base()` and `base_mut()` methods.

> **⚠ Direct field access**
>
> Do not use the `self.base` field directly. Use `self.base()` or `self.base_mut()` instead, otherwise you won't be able to access and call the base class methods.

This is a point where you can see the result. Compile your code and launch the Godot editor. Right-click on your `Sprite2D` in the scene tree, and choose "Change Type..." Find and choose the `Player` node type, which will be a child of `Sprite2D` in the Change Type dialog that appears.

Now, save your changes, and run the scene. The sprite should rotate at a constant speed.



> **Tip**
>
> **Launching the Godot application**
>
> Unfortunately there is a GDExtension limitation that prevents recompilation while the editor is open before Godot 4.2. Since Godot 4.2, it is possible to hot-reload extensions. This means you can recompile your Rust code and Godot will pick up changes, without needing to restart the editor.
>
> However, if you don't need to modify anything in the editor itself, you can launch Godot from the command-line or even your IDE. Check out the command-line tutorial for more information.

We now add a translation component to the sprite, following the upstream tutorial.

```
use godot::classes::ISprite2D;

#[godot_api]
impl ISprite2D for Player {
    fn init(base: Base<Sprite2D>) -> Self { /* as before */ }

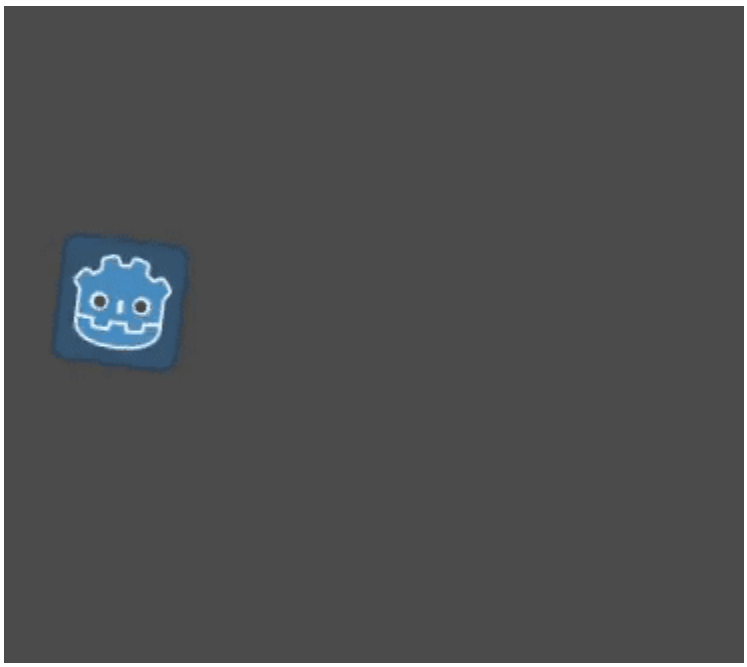
    fn physics_process(&mut self, delta: f64) {
        // GDScript code:
        //
        // rotation += angular_speed * delta
        // var velocity = Vector2.UP.rotated(rotation) * speed
        // position += velocity * delta

        let radians = (self.angular_speed * delta) as f32;
        self.base_mut().rotate(radians);

        let rotation = self.base().get_rotation();
        let velocity = Vector2::UP.rotated(rotation) * self.speed as f32;
        self.base_mut().translate(velocity * delta as f32);

        // or verbose:
        // let this = self.base_mut();
        // this.set_position(
        //     this.position() + velocity * delta as f32
        // );
    }
}
```

The result should be a sprite that rotates with an offset.



Custom Rust APIs

Say you want to add some functionality to your `Player` class, which can be called from GDScript. For this, you have a separate `impl` block, again annotated with `#[godot_api]`. However, this time we are using an *inherent* `impl` (i.e. without a trait name).

Concretely, we add a function to increase the speed, and a signal to notify other objects of the speed change.

```
#[godot_api]
impl Player {
    #[func]
    fn increase_speed(&mut self, amount: f64) {
        self.speed += amount;
        self.signals().speed_increased().emit();
    }

    #[signal]
    fn speed_increased();
}
```

`#[godot_api]` takes again the role of exposing the API to the Godot engine. But there are also two new attributes:

- `#[func]` exposes a function to Godot. The parameters and return types are mapped to their corresponding GDScript types. Functions are elaborated in the Functions chapter.
- `#[signal]` declares a signal. These are later explained in the Signals chapter.

API attributes typically follow the GDScript keyword names: `class`, `func`, `signal`, `export`, `var`, ...

That's it for the *Hello World* tutorial! The following chapters will go into more detail about the various features that godot-rust provides. We recommend following in chronological order, as important concepts are introduced along the way.

Using the Godot API

In this chapter, you will learn how to interact with the Godot engine from Rust code. After introducing you to builtins and objects, we will delve into engine API calls and discuss godot-rust specific idioms surrounding them.

If you are interested in exposing your own Rust symbols to the engine and to GDScript code, check out the chapter Registering Rust symbols. It is however strongly recommended to read this chapter first, as it introduces vital concepts.

Built-in types

The so-called "built-in types" or just "builtins" are the basic types that Godot provides. Notably, these are not *classes*. See also basic built-in types in Godot.

Table of contents

- List of types
  - Rust mapping
- String types
- Arrays and dictionaries
- Packed arrays

List of types

Here is an exhaustive list of all built-in types, by category. We use the GDScript names; below, we explain how they map to Rust.

Simple types

- Boolean: `bool`
- Numeric: `int`, `float`

Composite types

- Variant (able to hold anything): `Variant`
- String types: `String`, `StringName`, `NodePath`
- Ref-counted containers: `Array` (`Array[T]`), `Dictionary`
- Packed arrays: `Packed*Array` for following element types:
  `Byte`, `Int32`, `Int64`, `Float32`, `Float64`, `Vector2`, `Vector3`, `Vector4` [1], `Color`, `String`
- Functional: `Callable`, `Signal`

Geometric types

- Vectors: `Vector2`, `Vector2i`, `Vector3`, `Vector3i`, `Vector4`, `Vector4i`
- Bounding boxes: `Rect2`, `Rect2i`, `AABB`

- Matrices: `Transform2D`, `Transform3D`, `Basis`, `Projection`
- Rotation: `Quaternion`
- Geometric objects: `Plane`

Miscellaneous

- Color: `Color`
- Resource ID: `RID`

Rust mapping

Rust types in the library's API represent the corresponding Godot types in the closest way possible. They are used in parameter and return type position of API functions, for example. They are accessible through `godot::builtin`, and most symbols are also part of the prelude.

Most builtins have a 1:1 equivalent (e.g. `Vector2f`, `Color` etc.). The following list highlights some noteworthy mappings:

GDScript type	Rust type	Rust example expression
`int`	`i64` [2]	`-12345`
`float`	`f64` [2]	`3.14159`
`real`	`real` (either `f32` or `f64`)	`real!(3.14159)`
`String`	`GString`	`"Some string"` [3]
`StringName`	`StringName`	`"MyClass"` [3]
`NodePath`	`NodePath`	`"Nodes/MyNode"` [3]
`Array[T]`	`Array<T>`	`array![1, 2, 3]`
`Array`	`VariantArray` or `Array<Variant>`	`varray![1, "two", true]`
`Dictionary`	`Dictionary`	`vdict!{"key": "value"}`
`AABB`	`Aabb`	`Aabb::new(pos, size)`
`Object`	`Gd<Object>`	`Object::new_alloc()`
`SomeClass`	`Gd<SomeClass>`	`Resource::new_gd()`
`SomeClass` (nullable)	`Option<Gd<SomeClass>>`	`None`
`Variant` (also implicit)	`Variant`	`Variant::nil()`

Note that Godot does not have nullability information in its class API yet. This means that we have to conservatively assume that objects can be null, and thus use `Option<Gd<T>>` instead

of `Gd<T>` for object return types. This often needs unnecessary unwrapping.

Nullable types are being looked into on Godot side. If there is no upstream solution for a while, we may consider our own workarounds, but it may come with manual annotation of many APIs.

String types

Godot provides three string types: `String` (`GString` in Rust), `StringName`, and `NodePath`. `GString` is used as a general-purpose string, while `StringName` is often used for identifiers like class or action names. The idea is that `StringName` is cheap to construct and compare.[4]

When working with Godot APIs, you can pass references to the parameter type (e.g. `&GString`), as well as Rust strings `&str`, and `&String`. To convert different string types in argument contexts (e.g. `StringName -> GString`), you can call `arg()`.

```
// Label::set_text() takes impl AsArg<GString>.
label.set_text("my text");
label.set_text(&string);           // Rust String
label.set_text(&gstring);          // GString
label.set_text(string_name.arg()); // StringName
```

Outside argument contexts, the `From` trait is implemented for string conversions: `GString::From("my string")`, or `"my_string".into()`.

`StringName` in particular provides a direct conversion from C-string literals such as `c"string"`, introduced in Rust 1.77. This can be used for *static* C-strings, i.e. ones that remain allocated for the entire program lifetime. Don't use them for short-lived ones.

Arrays and dictionaries

Godot's linear collection type is `Array<T>`. It is generic over its element type `T`, which can be one of the supported Godot types (generally anything that can be represented by `Variant`). A special type `VariantArray` is provided as an alias for `Array<Variant>`, which is used when the element type is dynamically typed.

`Dictionary` is a key-value store, where both keys and values are `Variant`. Generic dictionaries are not currently supported in godot-rust.

Arrays and dictionaries can be constructed using three macros:

```
let a = array![1, 2, 3];          // Array<i64>
let b = varray![1, "two", true];  // Array<Variant>
let c = vdict!{"key": "value"};   // Dictionary
```

Their API is similar, but not identical to Rust's standard types `Vec` and `HashMap`. An important difference is that `Array` and `Dictionary` are reference-counted, which means that `clone()` will not create an independent copy, but another reference to the same instance. Furthermore, since internal elements are stored as variants, they are not accessible by reference. This is why the `[]` operator (`Index/IndexMut` traits) is absent, and `at()` is provided instead, returning by value.

```
let a = array![0, 11, 22];

assert_eq!(a.len(), 3);
assert_eq!(a.at(1), 11);         // Panics on out-of-bounds.
assert_eq!(a.get(1), Some(11));  // Also by value, not Some(&11).

let mut b = a.clone();   // Increment reference-count.
b.set(2, 33);            // Modify new ref.
assert_eq!(a.at(2), 33); // Original array has changed.

b.clear();
assert!(b.is_empty());
assert_eq!(b, Array::new()); // new() creates an empty array.


let c = vdict! {
    "str": "hello",
    "int": 42,
    "bool": true,
};

assert_eq!(c.len(), 3);
assert_eq!(c.at("str"), "hello".to_variant());    // Panics on missing key.
assert_eq!(c.get("int"), Some(42.to_variant()));  // Option<Variant>, again by
value.

let mut d = c.clone();            // Increment reference-count.
d.insert("float", 3.14);          // Modify new ref.
assert!(c.contains_key("float")); // Original dict has changed.
```

To iterate, you can use `iter_shared()`. This method works almost like `iter()` on Rust collections, but the name highlights that you do not have unique access to the collection during iteration, since there might exist another reference to the collection. This also means it's your responsibility to ensure that the array/dictionary is not modified in unintended

ways during iteration (which should be safe, but may lead to data inconsistencies).

```
let a = array!["one", "two", "three"];
let d = vdict!{"one": 1, "two": 2.0, "three": Vector3::ZERO};

for elem in a.iter_shared() {
    // elem has type GString.
    println!("Element: {elem}");
}

for (key, value) in d.iter_shared() {
    // key and value both have type Variant.
    println!("Key: {key}, value: {value}");
}
```

Packed arrays

`Packed*Array` types are used for storing elements space-efficiently ("packed") in contiguous memory. The `*` stands for the element type, e.g. `PackedByteArray` or `PackedVector3Array`.

```
// Create from slices.
let bytes = PackedByteArray::from(&[0x0A, 0x0B, 0x0C]);
let ints = PackedInt32Array::from(&[1, 2, 3]);

// Get/set individual elements using Index and IndexMut operators.
ints[1] = 5;
assert_eq!(ints[1], 5);

// Access as Rust shared/mutable slices.
let bytes_slice: &[u8] = b.as_slice();
let ints_slice: &mut [i32] = i.as_mut_slice();

// Access sub-ranges of the array using the same type.
let part: PackedByteArray = bytes.subarray(1, 3); // 1..3, or 1..=2
assert_eq!(part.as_slice(), &[0x0B, 0x0C]);
```

Unlike `Array`, packed arrays use copy-on-write instead of reference counting. When you clone a packed array, you get a new independent instance. Cloning is cheap as long as you don't modify either instance. Once you use a write operation (anything with `&mut self`), the packed array will allocate its own memory and copy the data.

Footnotes

[1] `PackedVector4Array` is only available since Godot version 4.3; added in PR #85474.

[2] Godot's `int` and `float` types are canonically mapped to `i64` and `f64` in Rust. However, some Godot APIs specify the domain of these types more specifically, so it's possible to encounter `i8`, `u64`, `f32` etc.

[3] String types `GString`, `StringName`, and `NodePath` can be passed into Godot APIs as string literals, hence the `"string"` syntax in this example. To assign to your own value, e.g. of type `GString`, you can use `GString::from("string")` or `"string"`.

[4] When constructing `StringName` from `&str` or `String`, the conversion is rather expensive, since UTF-8 is re-encoded as UTF-32. As Rust recently introduced C-string literals (`c"hello"`), we can now directly construct from them in case of ASCII. This is more efficient, but keeps memory allocated until shutdown, so don't use it for rarely used temporaries. See API docs and issue #531 for more information.

Objects

This chapter covers the most central mechanism of the Rust bindings -- one that will accompany you from the Hello-World example to a sophisticated Rust game.

We're talking about *objects* and the way they integrate into the Godot engine.

Table of contents

- Terminology
  - Inheritance
- The `Gd` smart pointer
- Object management and lifetime
  - Construction
  - Instance API
  - Conversions
  - Destruction
- Conclusion

Terminology

To avoid confusion, whenever we talk about objects, we mean *instances of Godot classes*. This amounts to `Object` (the hierarchy's root) and all classes inheriting directly or indirectly from it: `Node`, `Resource`, `RefCounted`, etc.

In particular, the term "class" also includes user-provided types that are declared using `#[derive(GodotClass)]`, even if Rust technically calls them structs. In the same vein, *inheritance* refers to the conceptual relation ("`Player` inherits `Sprite2D`"), not any technical language implementation.

Objects do **not** include built-in types such as `Vector2`, `Color`, `Transform3D`, `Array`, `Dictionary`, `Variant` etc. These types, although sometimes called "built-in classes", are not real classes, and we generally do not refer to their instances as *objects*.

Inheritance

Inheritance is a central concept in Godot. You likely know it already from the node hierarchy, where derived classes add specific functionality. This concept extends to Rust classes, with inheritance being emulated via composition.

Each Rust class has a Godot base class.

- Typically, a base class is a node type, i.e. it (indirectly) inherits from the class `Node`. This makes it possible to attach instances of the class to the scene tree. Nodes are manually managed, so you need to either add them to the scene tree or free them manually.
- If not explicitly specified, the base class is `RefCounted`. This is useful to move data around, without interacting with the scene tree. "Data bundles" (collection of multiple fields without much logic) should generally use `RefCounted`.
- `Object` is the root of the inheritance tree. It is rarely used directly, but it is the base class of `Node` and `RefCounted`. Use it only when you really need it; it requires manual memory management and is harder to handle.

> **Inheriting custom base classes**
>
> You cannot inherit other Rust classes or user-defined classes declared in GDScript.
>
> To create relations between Rust classes, use composition and traits. The library still undergoes some exploration in this area, so best practices for abstracting over Rust classes might change in the future.

The `Gd` smart pointer

`Gd<T>` is the type you will encounter the most when working with godot-rust.

It is also the most powerful and versatile type that the library provides.

In particular, its responsibilities include:

- Holding references to *all* Godot objects, whether they are engine types like `Node2D` or your own `#[derive(GodotClass)]` structs in Rust.
- Tracking memory management of types that are reference-counted.
- Safe access to user-defined Rust objects through interior mutability.
- Detecting destroyed objects and preventing UB (double-free, dangling pointer, etc.).
- Providing FFI conversions between Rust and engine representations, for engine-provided and user-exposed APIs.

A few practical examples (don't worry if you don't fully understand them yet, they will be explained later on):

1. Retrieve a node relative to current -- type inferred as `Gd<Node3D>` :

```
// Retrieve Gd<Node3D>.
let child = self.base().get_node_as::<Node3D>("Child");
```

2. Load a scene and instantiate it as a `RigidBody2D` :

```
// mob_scene is declared as a field of type Gd<PackedScene>.
self.mob_scene = load("res://Mob.tscn");

// instanced is of type Gd<RigidBody2D>.
let mut instanced = self.mob_scene.instantiate_as::<RigidBody2D>();
```

3. A signal handler for the `body_entered` signal of a `Node3D` in your custom class:

```
#[godot_api]
impl Player {
    #[func]
    fn on_body_entered(&mut self, body: Gd<Node3D>) {
        // body holds the reference to the Node3D object that triggered the
signal.
    }
}
```

Object management and lifetime

When working with Godot objects, it is important to understand how long they live and how or when they are destroyed.

Construction

Not all classes in Godot are constructible; for example, singletons do not provide a constructor.

For all others, the constructor's name depends on the memory management of the class:

- For reference-counted classes, the constructor is called `new_gd` (e.g. `TcpServer::new_gd()`)
- For manually managed classes, it is called `new_alloc` (e.g. `Node2D::new_alloc()`).

The `new_gd()` and `new_alloc()` functions are imported via extension traits `NewGd` and `NewAlloc`, respectively. Those always return the type `Gd<Self>`. If you type `::` after a class name, your IDE should suggest the correct constructor for it.

Instance API

Once alive, Godot objects can be accessed to interact with the engine.

Functionality to query and manage the object's lifetime is directly available on the `Gd<T>` type. Examples include:

- `instance_id()` to obtain Godot's object ID.
- `clone()` to create a new reference to the same object.
- `free()` to manually destroy objects.
- `==` and `!=` to compare objects for identity.

Conversions

You can up- and downcast objects if they stand in an inheritance relation. godot-rust will statically ensure that the cast makes sense.

Downcasts are done via `cast::<U>()`. If the cast fails, the method will panic. You can also use `try_cast::<U>()` to get a `Result`.

```
let node: Gd<Node> = ...;

// "I know this downcast will succeed" -> use cast().
let node2d = node.cast::<Node2D>();
// Alternative syntax:
let node2d: Gd<Node2D> = node.cast();

// Fallible downcast -> use try_cast().
let sprite = node.try_cast::<Sprite2D>();
match sprite {
    Ok(sprite) => { /* access converted Gd<Sprite2D> */ },
    Err(node) => { /* access previous Gd<Node> */ },
}
```

Upcasts are always infallible. You can use `upcast::<U>()` to consume the value.

```
let node2d: Gd<Node2D> = ...;
let node = node2d.upcast::<Node>();
// or, equivalent:
let node: Gd<Node> = node2d.upcast();
```

If you just need a reference, use `upcast_ref()` or `upcast_mut()`.

```
let node2d: Gd<Node2D> = ...;
let node: &Node = node2d.upcast_ref();

let mut refc: Gd<RefCounted> = ...;
let obj: &mut Object = refc.upcast_mut();
```

Destruction

Reference-counted classes, instantiated via `new_gd()`, are automatically destroyed when the last reference goes out of scope. This includes references that have been shared with the Godot engine (e.g. held by GDScript code).

Classes instantiated via `new_alloc()` require manual memory management. This means that you either have to explicitly call `Gd::free()` or let a Godot method such as `Node::queue_free()` take care of it.

> **Safety around the dead**
>
> Accessing destroyed objects is a common source of bugs in Godot, and can occasionally cause undefined behavior (UB). Not so in godot-rust! We have designed the `Gd<T>` type to be safe even in the presence of mistakes.
>
> If you try to access a destroyed object, the Rust code will panic. There are also APIs to query for validity, although we generally recommend to fix bugs rather than defensive programming.

Conclusion

Objects are a central concept in the Rust bindings. They represent instances of Godot

classes, both engine- and user-defined. We have seen how to construct, manage and destroy them.

But we still have to *use* objects, i.e. access functionality their class exposes. The next chapter will go into calling Godot functions.

Calling functions

In general, the godot-rust library maps Godot functions in a way that feels as idiomatic as possible in Rust. Sometimes, signatures differ from GDScript, and this page will go into such differences.

Table of contents

- Godot classes
- Godot functions
- Singletons
- Default parameters
- Dynamic calls

Godot classes

Godot classes are located in the `godot::classes` module. Some often-used ones like `Node`, `RefCounted`, `Node3D` etc. are additionally re-exported in `godot::prelude`.

The majority of Godot's functionality is exposed via functions inside classes. Please don't hesitate to check out the API docs.

Godot functions

As usual in Rust, functions are split into *methods* (with a `&self` / `&mut self` receiver) and *associated functions* (called "static functions" in Godot).

To access Godot APIs on a `Gd<T>` pointer, simply call the method on the `Gd` object directly. This works due to `Deref` and `DerefMut` traits, which give you an object reference through `Gd`. In a later chapter, we'll also see how to call from and into functions defined in Rust.

```
// Call with &self receiver.
let node = Node::new_alloc();
let path = node.get_path();

// Call with &mut self receiver.
let mut node = Node::new_alloc();
let other: Gd<Node> = ...;
node.add_child(&other);
```

Whether a method requires a shared reference (`&T`) or an exclusive one (`&mut T`) depends on how the method is declared in the GDExtension API (`const` or not). Note that this distinction is **informational** only and bears no safety implications, but it is useful in practice to detect accidental modification. Technically, you could always just create another pointer via `Gd::clone()` .

Associated functions (called "static" in GDScript) are invoked on the type itself.

```
Node::print_orphan_nodes();
```

Singletons

Singleton classes (not to be confused with *autoloads*, which are sometimes called singletons, too) provide a `singleton()` function to access the one true instance. Methods are then invoked on that instance:

```
let input = Input::singleton();
let jump = input.is_action_pressed("jump");
let mouse_pos = input.get_mouse_position();

// Mutable actions need mut:
let mut input = input;
input.set_mouse_mode(MouseMode::CAPTURED);
```

There are discussions about providing methods directly on the singleton type instead of requiring the `singleton()` call. This would however lose the mutability information, among a few other things.

Default parameters

GDScript supports default values for parameters. If no argument is passed, then the default value is used. As an example, we can use `AcceptDialog.add_button()`. The GDScript signature is:

```
Button add_button(String text, bool right=false, String action="")
```

So you can call it in the following ways from GDScript:

```
var dialog = AcceptDialog.new()
var button0 = dialog.add_button("Yes")
var button1 = dialog.add_button("Yes", true)
var button2 = dialog.add_button("Yes", true, "confirm")
```

In Rust, we still have a base method `AcceptDialog::add_button()`, which takes no default arguments. It can be called in the usual way:

```
let dialog = AcceptDialog::new_alloc();
let button = dialog.add_button("Yes");
```

Because Rust does not support default parameters, we have to emulate the other calls differently. We decided to use the builder pattern.

Builder methods in the library receive **the `_ex` suffix**. Such a method takes all required parameters, like the base method. It returns a builder object, which offers methods to set the optional parameters by their name. Eventually, a `done()` method concludes the builder and returns the result of the Godot function call.

For our example, we have the `AcceptDialog::add_button_ex()` method. These two calls are exactly equivalent:

```
let button = dialog.add_button("Yes");
let button = dialog.add_button_ex("Yes").done();
```

You can additionally pass optional arguments using methods on the builder object. Just specify the arguments you need. The nice thing here is that you can use any order, and skip any parameters -- unlike GDScript, where you can only skip ones at the end.

```
// Equivalent in GDScript: dialog.add_button("Yes", true, "")
let button = dialog.add_button_ex("Yes")
    .right(true)
    .done();

// GDScript: dialog.add_button("Yes", false, "confirm")
let button = dialog.add_button_ex("Yes")
    .action("confirm")
    .done();

// GDScript: dialog.add_button("Yes", true, "confirm")
let button = dialog.add_button_ex("Yes")
    .right(true)
    .action("confirm")
    .done();
```

Dynamic calls

Sometimes, you want to invoke functions that are not exposed in the Rust API. These could be functions you wrote inside custom GDScript code, or methods from other GDExtensions.

When you don't have the static information available, you can use Godot's reflection APIs. Godot provides `Object.call()` among others, which is exposed in two ways in Rust.

If you expect a call to succeed (since you know the GDScript code you wrote), use `Object::call()`. This method will panic if the call fails, providing a detailed message.

```
let node = get_node_as::<Node2D>("path/to/MyScript");

// Declare arguments as a slice of variants.
let args = &["string".to_variant(), 42.to_variant()];

// Or better, use the vslice! macro for this:
let args = vslice!["string", 42];

// Call the method dynamically.
let val: Variant = node.call("my_method", args);

// Convert to a known type (may panic; try_to() doensn't).
let vec2 = val.to::<Vector2>();
```

If instead you want to handle the failure case, use `Object::try_call()`. This method returns a `Result` with the result or a `CallError` error.

```
let result: Result<Variant, CallError> = node.try_call("my_method", args);

match result {
    Ok(val) => {
        let vec2 = val.to::<Vector2>();
        // ...
    }
    Err(err) => {
        godot_print!("Error calling method: {}", err);
    }
}
```

Registering Rust symbols

This chapter teaches how you make your own Rust code available to Godot. You do this by *registering* individual symbols (classes, functions etc.) in the engine.

Starting with class registration, the chapter then goes into the details of registering functions, properties, signals and constants.

Proc-macro API

The proc-macro API is currently the only way to register Rust symbols. A variety of procedural macros (derive and attribute macros) are provided to decorate your Rust items, such as `struct`s or `impl` blocks. Behind the scenes, these macros generate the necessary glue code to register each item with Godot.

The library is designed in a way that you can use all your existing knowledge and simply extend it with macro syntax, rather than having to learn a completely new way of doing things. We try to avoid foreign DSLs (domain-specific languages) and instead build on top of Rust's existing syntax.

This approach does a respectable job at limiting the amount of boilerplate code you have to write, and thus makes it much easier for you to focus on the important bits. For example, you will rarely have to repeat yourself more than necessary or register one thing in multiple places (e.g. declare a method, mention it in another `register` method and then repeat its name yet again as a string literal).

"Exporting"

The term "exporting" is sometimes erroneously used. Please avoid talking about "exporting classes" or "exporting methods" if you mean "registering". This can often cause confusion, especially among beginners.

Export already has two well-defined meanings in the context of Godot:

1. Exporting a property. This does not *register* the property with Godot, but renders it visible in the editor.
    - GDScript uses the `@export` annotation for this, we use `#[export]`.

- See also GDScript exported properties.

2. Exporting projects, meaning bundling them for release.
    - The editor provides a UI to build release versions of your game or application, so they can run as a standalone executable. This process of building the executable is called "exporting".
    - See also Exporting projects.

Registering classes

Classes are the backbone of data modeling in Godot. If you want to build complex user-defined types in a type-safe way, you won't get around classes. Arrays, dictionaries and simple types only get you so far, and overusing them defeats the purpose of using a statically typed language.

Rust makes class registration straightforward. As mentioned before, Rust syntax is used as a baseline, with godot-rust specific additions.

See also GDScript reference for classes.

Table of contents

- Defining a Rust struct
- Selecting a base class
- The base field
- Conclusion

Defining a Rust struct

In Rust, Godot classes are represented by structs. Structs are defined as usual and can contain any number of fields. To register them with Godot, you need to derive the `GodotClass` trait.

> **GodotClass trait**
>
> The `GodotClass` trait marks all classes known in Godot. It is already implemented for engine classes, for example `Node` or `Resource`. If you want to register your own classes, you need to implement `GodotClass` as well.
>
> `#[derive(GodotClass)]` streamlines this process and takes care of all the boilerplate. See API docs for detailed information.

Let's define a simple class named `Monster`:

```
#[derive(GodotClass)]
#[class(init)] // more about this later.
struct Monster {
    name: String,
    hitpoints: i32,
}
```

That's it. Immediately after compiling, this class becomes available in Godot through hot reloading (before Godot 4.2, after restart). It won't be very useful yet, but the above definition is enough to register `Monster` in the engine.

> **Auto-registration**
>
> `#[derive(GodotClass)]` *automatically* registers the class -- you don't need an explicit `add_class()` registration call or a central list mentioning all classes.
>
> The proc-macro internally registers the class in such a list at startup time.

Selecting a base class

By default, the base class of a Rust class is `RefCounted`. This is consistent with GDScript when you omit the `extends` keyword.

`RefCounted` is quite useful for data bundles. As implied by the name, it allows sharing instances tracked by a reference counter; as such, you don't need to worry about memory management. `Resource` is a subclass of `RefCounted` and is useful for data that needs to be serialized to the filesystem.

However, if you want your class to be part of the scene tree, you need to use `Node` (or one of its derived classes) as a base class.

Here, we use a more concrete node type, `Node3D`. This is done by specifying `#[class(base=Node3D)]` on the struct definition:

```
#[derive(GodotClass)]
#[class(base=Node3D)]
struct Monster {
    name: String,
    hitpoints: i32,
}
```

The base field

Since Rust does not have inheritance, we need to use composition to achieve the same effect. godot-rust provides a `Base<T>` type, which lets us store the instance of the Godot superclass (base class) as a field in our `Monster` class.

```
#[derive(GodotClass)]
#[class(base=Node3D)]
struct Monster {
    name: String,
    hitpoints: i32,
    base: Base<Node3D>,
}
```

The important part is the `Base<T>` type. `T` must match the base class you specified in the `#[class(base=...)]` attribute. You can also use the associated type `Self::Base` for `T`.

When you declare a base field in your struct, the `#[derive]` procedural macro will automatically detect the `Base<T>` type.[1] This lets you access the `Node` API through provided methods `self.base()` and `self.base_mut()`, but more on this later.

Conclusion

You have learned how to define a Rust class and register it with Godot. You now know that different base classes exist and how to select one.

The next chapters cover functions and constructors.

---

[1] You can tweak the type detection using the `#[hint]` attribute, see the corresponding docs.

Registering functions

Functions are essential in any programming language to execute logic. The godot-rust library allows you to register functions, so that they can be called from the Godot engine and GDScript.

Registration of functions happens always inside `impl` blocks that are annotated with `#[godot_api]`.

See also GDScript reference for functions.

Table of contents

- Godot special functions
- User-defined functions
  - Methods
  - Associated functions
- Methods and object access
  - Calling Rust methods (binds)
  - Base access from `self`
  - Obtaining `Gd<Self>` from within
- Conclusion

Godot special functions

> **Interface traits**
>
> Each engine class comes with an associated trait, which has the same name but is prefixed with the letter `I`, for "Interface". The trait has no required functions, but you can override any functions to customize the behavior towards Godot.
>
> Any `impl` block for the trait must be annotated with the `#[godot_api]` attribute macro.

> **godot_api macro**
>
> The attribute proc-macro `#[godot_api]` is applied to `impl` blocks and marks their items for registration. It takes no arguments.
>
> See API docs for detailed information.

Functions provided by the interface trait (beginning with `I`) are called *Godot special functions*. These can be overridden and allow you to influence the behavior of an object. Most common is a hook into the *lifecycle* of your object, defining some logic that is run upon certain events like creation, scene-tree entering, or per-frame updates.

In our case, the `Node3D` comes with the `INode3D` trait. Here is a small selection of its lifecycle methods. For a complete list, see `INode3D` docs.

```
#[godot_api]
impl INode3D for Monster {
    // Instantiate the object.
    fn init(base: Base<Node3D>) -> Self { ... }

    // Called when the node is ready in the scene tree.
    fn ready(&mut self) { ... }

    // Called every frame.
    fn process(&mut self, delta: f64) { ... }

    // Called every physics frame.
    fn physics_process(&mut self, delta: f64) { ... }

    // String representation of the object.
    fn to_string(&self) -> GString { ... }

    // Handle user input.
    fn input(&mut self, event: Gd<InputEvent>) { ... }

    // Handle lifecycle notifications.
    fn on_notification(&mut self, what: Node3DNotification) { ... }
}
```

As you see, some methods take `&mut self` and some take `&self`, depending on whether they typically mutate the object or not. Some also have return values, which are passed back into the engine. For example, the `GString` returned from `to_string()` is used if you print an object in GDScript.

So let's implement `to_string()`, here again showing the class definition for quick reference.

```
#[derive(GodotClass)]
#[class(base=Node3D)]
struct Monster {
    name: String,
    hitpoints: i32,

    base: Base<Node3D>,
}

#[godot_api]
impl INode3D for Monster {
    fn to_string(&self) -> GString {
        let Self { name, hitpoints, .. } = &self;
        format!("Monster(name={name}, hp={hitpoints})").into()
    }
}
```

User-defined functions

Methods

Besides Godot special functions, you can register your own functions. You need to declare them inside an inherent `impl` block, also annotated with `#[godot_api]`.

Each function needs a `#[func]` attribute to register it with Godot. You can omit `#[func]` as well, but functions defined like that are only visible to Rust code.

Let's add two methods to our `Monster` class: one that deals damage to the monster, and one that returns its name.

```
#[godot_api]
impl Monster {
    #[func]
    fn damage(&mut self, amount: i32) {
        self.hitpoints -= amount;
    }

    #[func]
    fn get_name(&self) -> GString {
        self.name.clone()
    }
}
```

The above methods are now available in GDScript. You can call them as follows:

```
var monster = Monster.new()
# ...
monster.damage(10)
print("A monster called ", monster.get_name())
```

As you see, the Rust types are automatically mapped to their GDScript counterparts. In this case, `i32` becomes `int` and `GString` becomes `String`. Sometimes there are multiple possible mappings, e.g. Rust `u16` would also be mapped to `int` in GDScript.

Associated functions

In addition to **methods** (taking `&self` or `&mut self`), you can also register **associated functions** (without a receiver). In GDScript, the latter are known as "static functions".

For example, we can add an associated function which generates a random monster name:

```
#[godot_api]
impl Monster {
    #[func]
    fn random_name() -> GString {
        // ...
    }
}
```

The above can then be called from GDScript as follows:

```
var name: String = Monster.random_name()
```

Of course, it is also possible to declare parameters.

Associated functions are sometimes useful for user-defined constructors, as we will see in the next chapter.

Methods and object access

When you define your own Rust functions, there are two use cases that occur very frequently:

- You want to invoke your Rust methods from outside, through a `Gd` pointer.
- You want to access methods of the base class (e.g. `Node3D`).

This section explains how to do both.

Calling Rust methods (binds)

If you now have a `monster: Gd<Monster>`, which stores a `Monster` object as defined above, you won't be able to simply call `monster.damage(123)`. Rust is stricter than C++ and requires that only one `&mut Monster` reference exists at any point in time. Since `Gd` pointers can be freely cloned, direct access through `DerefMut` wouldn't be sufficient to ensure non-aliasing.

To approach this, godot-rust uses the interior mutability pattern, which is quite similar to how `RefCell` works.

In short, whenever you need shared (immutable) access to a Rust object from a `Gd` pointer, use `Gd::bind()`. Whenever you need exclusive (mutable) access, use `Gd::bind_mut()`.

```
let monster: Gd<Monster> = ...;

// Immutable access with bind():
let name: GString = monster.bind().get_name();

// Mutable access with bind_mut() -- we rebind the object first:
let mut monster = monster;
monster.bind_mut().damage(123);
```

Regular Rust visibility rules apply: if your function should be visible in another module, declare it as `pub` or `pub(crate)`.

> **The need for #[func]**
>
> The `#[func]` attribute *only* makes a function available to the Godot engine. It is orthogonal to Rust visibility (`pub`, `pub(crate)`, ...) and does not influence whether a method can be accessed through `Gd::bind()` and `Gd::bind_mut()`.
>
> If you only need to call a function in Rust, do not annotate it with `#[func]`. You can always add this later.

`bind()` and `bind_mut()` return *guard objects*. At runtime, the library verifies that the borrow rules are upheld, and panics otherwise. It can be beneficial to reuse guards across multiple statements, but make sure to keep their scope limited to not unnecessarily constrain access to objects (especially when using `bind_mut()`).

```
fn apply_monster_damage(mut monster: Gd<Monster>, raw_damage: i32) {
    // Artificial scope:
    {
        let guard = monster.bind_mut(); // locks object -->
        let armor = guard.get_armor_multiplier();

        let damage = (raw_damage as f32 * armor) as i32;

        guard.damage(damage)
    } // <-- until here, where guard lifetime ends.

    // Now you can pass the pointer on to other routines again.
    check_if_dead(monster);
}
```

Base access from `self`

Within a class, you don't directly have a `Gd<T>` pointing to the own instance with base class methods. So you cannot use the approach explained in the *Calling functions* chapter, where you would simply use `gd.set_position(...)` or similar.

Instead, you can access base class APIs via `base()` and `base_mut()`. This requires that your class defines a `Base<T>` field. Let's say we add a `velocity` field and two new methods:

```
#[derive(GodotClass)]
#[class(base=Node3D)]
struct Monster {
    // ...
    velocity: Vector2,
    base: Base<Node3D>,
}

#[godot_api]
impl Monster {
    pub fn apply_movement(&mut self, delta: f32) {
        // Read access:
        let pos = self.base().get_position();

        // Write access (mutating methods):
        self.base_mut().set_position(pos + self.velocity * delta)
    }

    // This method has only read access (&self).
    pub fn is_inside_area(&self, rect: Rect2) -> String
    {
        // We can only call base() here, not base_mut().
        let node_name = self.base().get_name();

        format!("Monster(name={}, velocity={})", node_name, self.velocity)
    }
}
```

Both `base()` and `base_mut()` are defined in an extension trait `WithBaseField`. They return *guard objects*, which prevent other access to `self` in line with Rust's borrow rules. You can reuse a guard across multiple statements, but make sure to keep its scope limited to not unnecessarily constrain access to `self`:

```
    pub fn apply_movement(&mut self, delta: f32) {
        // Artificial scope:
        {
            let guard = self.base_mut(); // locks `self` -->
            let pos = guard.get_position();

            guard.set_position(pos + self.velocity * delta)
        } // <-- until here, where guard lifetime ends.

        // Now can invoke other self methods again.
        self.on_position_updated();
    }
```

Instead of an extra scope, you can of course also just call `drop(guard)`.

> **Do not combine bind/bind_mut + base/base_mut**
>
> Code like `object.bind().base().some_method()` is unnecessarily verbose and slow.
> If you have a `Gd<T>` pointer, use `object.some_method()` directly.
>
> Combining `bind()` / `bind_mut()` immediately with `base()` / `base_mut()` is a mistake.
> The latter two should only be called from within the class `impl`.

Obtaining `Gd<Self>` from within

In some cases, you need to get a `Gd<T>` pointer to the current instance. This can occur if you want to pass it to other methods, or if you need to store a pointer to `self` in a data structure.

`WithBaseField` offers a method `to_gd()`, returning a `Gd<Self>` with the correct type.

Here's an example. The `monster` is passed a hash map, in which it can register/unregister itself, depending on whether it's alive or not.

```
#[godot_api]
impl Monster {
    // Function that registers each monster by name, or unregisters it if dead.
    fn update_registry(&self, registry: &mut HashMap<String, Gd<Monster>>) {
        if self.is_alive() {
            let self_as_gd: Gd<Self> = self.to_gd();
            registry.insert(self.name.clone(), self_as_gd);
        } else {
            registry.remove(&self.name);
        }
    }
}
```

> **⚠ Don't bind to_gd() inside class methods**
>
> The methods `base()` and `base_mut()` use a clever mechanism that "re-borrows" the current object reference. This enables re-entrant calls, such as `self.base().notify(...)`, which may e.g. call `ready(&mut self)`. The `&mut self` here is a reborrow of the call-site `self`.
>
> When you use `to_gd()`, the borrow checker will treat this as an independent object. If you call `bind_mut()` on it, while inside the class impl, you will immediately get a double-borrow panic. Intead, use `to_gd()` to hand out a pointer and don't access until the current method has ended.

Conclusion

This page gave you an overview of registering functions with Godot:

- Special methods that hook into the lifecycle of your object.
- User-defined methods and associated functions to expose a Rust API to Godot.

It also showed how methods and objects interact: calling Rust methods through `Gd<T>` and working with base class APIs.

These are just a few use cases, you are very flexible in how you design your interface between Rust and GDScript. In the next page, we will look into a special kind of functions: constructors.

Constructors

While Rust does not have constructors as a language feature (like C++ or C#), associated functions that return a new object are commonly called "constructors". We extend the term to include slightly deviating signatures, but conceptually *constructors* are always used to construct new objects.

Godot has a special constructor, which we call the *Godot default constructor* or simply `init`. This is comparable to the `_init` method in GDScript.

Table of contents

- Default constructor
  - Library-generated `init`
  - Manually defined `init`
  - Disabled `init`
- Custom constructors
  - Objects with a base field
  - Objects without a base field
- Destructors
- Conclusion

Default constructor

The constructor of any `GodotClass` object is called `init` in godot-rust. This constructor is necessary to instantiate the object in Godot. It is invoked by the scene tree or when you write `Monster.new()` in GDScript.

There are two options to define the constructor: let godot-rust generate it or define it manually. It is also possible to opt out of `init` if you don't need Godot to default-construct your object.

> **No parameters in `init`**
>
> Unlike GDScript, GDExtension does not support additional parameters in the `init` constructor. However, you can achieve the same with Custom constructors below.

Library-generated `init`

You can use `#[class(init)]` to generate a constructor for you. This is limited to simple cases, and it calls `Default::default()` for each field (except the `Base<T>` one, which is correctly wired up with the base object).

```
#[derive(GodotClass)]
#[class(init, base=Node3D)]
struct Monster {
    name: String,           // initialized to ""
    hitpoints: i32,         // initialized to 0
    base: Base<Node3D>,     // wired up
}
```

To provide another default value, use `#[init(val = value)]`. This should only be used for simple cases, as it may lead to difficult-to-read code and error messages. This API may also still change.

```
#[derive(GodotClass)]
#[class(init, base=Node3D)]
struct Monster {
    name: String,           // initialized to ""

    #[init(val = 100)]
    hitpoints: i32,         // initialized to 100

    base: Base<Node3D>,     // wired up
}
```

Manually defined `init`

We can provide a manually-defined constructor by overriding the trait's associated function `init`:

```rust
#[derive(GodotClass)]
#[class(base=Node3D)] // No init here, since we define it ourselves.
struct Monster {
    name: String,
    hitpoints: i32,
    base: Base<Node3D>,
}

#[godot_api]
impl INode3D for Monster {
    fn init(base: Base<Node3D>) -> Self {
        Self {
            name: "Nomster".to_string(),
            hitpoints: 100,
            base,
        }
    }
}
```

As you can see, the `init` function takes a `Base<Node3D>` as its one and only parameter. This is the base class instance, which is typically just forwarded to its corresponding field in the struct, here `base`.

The `init` method always returns `Self`. You may notice that this is currently the only way to construct a `Monster` instance. As soon as your struct contains a base field, you can no longer provide your own constructor, as you can't provide a value for that field. This is by design and ensures that *if* you need access to the base, that base comes from Godot directly.

Disabled `init`

You don't always need to provide a default constructor to Godot. Reasons to not have a constructor include:

- Your class is not a node that should be added to the tree as part of a scene file.
- You require custom parameters to be provided for your object invariants -- a default value is not meaningful.
- You only need to construct objects from Rust code, not from GDScript or the Godot editor.

To disable the `init` constructor, you can use `#[class(no_init)]`:

```
#[derive(GodotClass)]
#[class(no_init, base=Node3D)]
struct Monster {
    name: String,
    hitpoints: i32,
    base: Base<Node3D>,
}
```

Not providing/generating an `init` method and forgetting to use `#[class(no_init)]` will result in a compile-time error.

Custom constructors

The default constructor `init` is not always useful, as it may leave objects in an incorrect state.

For example, a `Monster` will always have the same values for `name` and `hitpoints` upon construction, which may not be desired. Let's provide a more suitable constructor, which accepts those attributes as parameters.

```
// Default constructor from before.
#[godot_api]
impl INode3D for Monster {
    fn init(base: Base<Node3D>) -> Self { ... }
}

// New custom constructor.
#[godot_api]
impl Monster {
    #[func] // Note: the following is incorrect.
    fn from_name_hp(name: GString, hitpoints: i32) -> Self {
        ...
    }
}
```

But now, how to fill in the blanks? `Self` requires a base object, how to obtain it? In fact, we cannot return `Self` here.

> **Passing around objects**
>
> When interacting with Godot from Rust, all objects (class instances) need to be transported inside the `Gd` smart pointer -- whether they appear as parameters or return types.
>
> The return types of `init` and a few other gdext-provided functions are an exception, because the library requires at this point that you have a *value* of the raw object. You never need to return `Self` in your own defined `#[func]` functions.
>
> For details, consult the chapter about objects or the `Gd<T>` API docs.

So we need to return `Gd<Self>` instead of `Self`.

Objects with a base field

If your class `T` contains a `Base<...>` field, you cannot create a standalone instance -- you must encapsulate it in `Gd<T>`. You can also not extract a `T` from a `Gd<T>` smart pointer anymore; since it has potentially been shared with the Godot engine, this would not be a safe operation.

To construct `Gd<Self>`, we can use `Gd::from_init_fn()`, which takes a closure. This closure accepts a `Base` object and returns an instance of `Self`. In other words, it has the same signature as `init` -- this presents an alternative way of constructing Godot objects, while allowing to pass in addition context.

The result of `Gd::from_init_fn()` is a `Gd<Self>` object, which can be directly returned by `Monster::from_name_hp()`.

```
#[godot_api]
impl Monster {
    #[func]
    fn from_name_hp(name: GString, hitpoints: i32) -> Gd<Self> {
        // Function contains a single statement, the `Gd::from_init_fn()` call.

        Gd::from_init_fn(|base| {
            // Accept a base of type Base<Node3D> and directly forward it.
            Self {
                name: name.into(), // Convert GString -> String.
                hitpoints,
                base,
            }
        })
    }
}
```

That's it! The just added associated function is now registered in GDScript and effectively works as a constructor:

```
var monster = Monster.from_name_hp("Nomster", 100)
```

Objects without a base field

For classes that don't have a base field, you can simply use `Gd::from_object()` instead of `Gd::from_init_fn()`.

This is often useful for *data bundles*, which don't define much logic but are an object-oriented way to bundle related data in a single type. Such classes are typically subclasses of `RefCounted` or `Resource`.

```
#[derive(GodotClass)]
#[class(no_init)] // We only provide a custom constructor.
// Since there is no #[class(base)] key, the base class will default to
RefCounted.
struct MonsterConfig {
    color: Color,
    max_hp: i32,
    tex_coords: Vector2i,
}

#[godot_api]
impl MonsterConfig {
    // Not named 'new' since MonsterConfig.new() in GDScript refers to default.
    #[func]
    fn create(color: Color, max_hp: i32, tex_coords: Vector2i) -> Gd<Self> {
        Gd::from_object(Self {
            color,
            max_hp,
            tex_coords,
        })
    }
}
```

Destructors

You do not typically need to declare your own destructors, if you manage memory through RAII. If you do however need custom cleanup logic, simply declare the `Drop` trait for your type:

```
impl Drop for Monster {
    fn drop(&mut self) {
        godot_print!("Monster '{}' is being destroyed!", self.name);
    }
}
```

`Drop::drop()` is invoked as soon as Godot orders the destruction of your `Gd<T>` smart pointer -- either if it is manually freed, or if the last reference to it goes out of scope.

Conclusion

Constructors allow to initialize Rust classes in various ways. You can generate, implement, or disable the default constructor `init`, and you can provide as many custom constructors

with different signatures as you like.

Registering properties

So far, you learned how to register classes and functions. This is already powerful enough to create simple applications with godot-rust, however you might want to give Godot more direct access to the state of your object.

This is where properties come into play. In Rust, properties are typically defined as fields of a struct.

See also GDScript reference for properties.

Table of contents

- Registering variables
- Exporting variables
- Enums
- Advanced usage
- Custom types with `#[var]` and `#[export]`

Registering variables

Previously, we defined a function `Monster::get_name()`. This works to fetch the name, but requires you to write `obj.get_name()` in GDScript. Sometimes, you do not need this extra encapsulation and would like to access the field directly.

The godot-rust library provides an attribute `#[var]` to annotate fields that should be exposed as variables. This works like the `var` keyword in GDScript.

Starting with the earlier struct declaration, we now add the `#[var]` attribute to the `name` field. We also change the type from `String` to `GString`, since this field is now directly interfacing Godot.

```
#[derive(GodotClass)]
#[class(init, base=Node3D)]
struct Monster {
    #[var]
    name: GString,
    hitpoints: i32,
}
```

The effect of this is that `name` is now registered as a *property* in Godot:

```
var monster = Monster.new()

# Write the property.
monster.name = "Orc"

# Read the property.
print(monster.name) # prints "Orc"
```

In GDScript, properties are syntactic sugar for function calls to getters and setters. You can also do so explicitly:

```
var monster = Monster.new()

# Write the property.
monster.set_name("Orc")

# Read the property.
print(monster.get_name()) # prints "Orc"
```

The `#[var]` attribute also takes parameters to customize whether both getters and setters are provided, and what their names are. You can also write Rust methods acting as getters and setters, if you have more involved logic. See the API documentation for details.

> **Visibility**
>
> Like `#[func]` functions, `#[var]` fields do not need to be `pub`. This separates visibility towards Godot and towards Rust.
>
> In practice, you can still access `#[var]` fields from Rust, but via detours (e.g. Godot's reflection APIs). But this is then a deliberate choice; private fields are primarily preventing *accidental* mistakes or encapsulation breaches.

Exporting variables

The `#[var]` attribute exposes a field to GDScript, but does not display it in the Godot editor UI.

Making a property available to the editor is called *exporting*. Like the GDScript annotation `@export`, godot-rust provides exports through the `#[export]` attribute. You might see a pattern with naming here.

The following code not only makes the `name` field available to GDScript, but it also adds a property UI in the editor. This allows you to name every `Monster` instance individually, without any code!

```
#[derive(GodotClass)]
#[class(init, base=Node3D)]
struct Monster {
    #[export]
    name: GString,
    hitpoints: i32,
}
```

You may have noticed that there is no longer a `#[var]` attribute. This is because `#[export]` always implies `#[var]` -- the name is still accessible from GDScript like before.

You can also declare both attributes on the same field. This is in fact necessary as soon as you provide arguments to customize them.

Enums

You can export Rust enums as properties, as long as they are C style enums (no associated data per variant). An exported enum appears as a drop-down field in the editor, with all available options. In order to do that, you need to derive three traits:

- `GodotConvert` to define how the type is converted from/to Godot.
- `Var` to allow using it as a `#[var]` property, so it can be accessed from Godot.
- `Export` to allow using it as a `#[export]` property, so it appears in the editor UI.

Additionally, it can make sense to derive following standard traits:

- `Clone`: it is possible without, but then you need to implement `Var` manually. The reason is that `Var` also provides Rust getters when used with `#[var(pub)]`, which

require cloning the value.
- `Default`: if there is a meaningful default value. If not, use `#[init(val = ...)]` on the field declarations to provide an initial value.

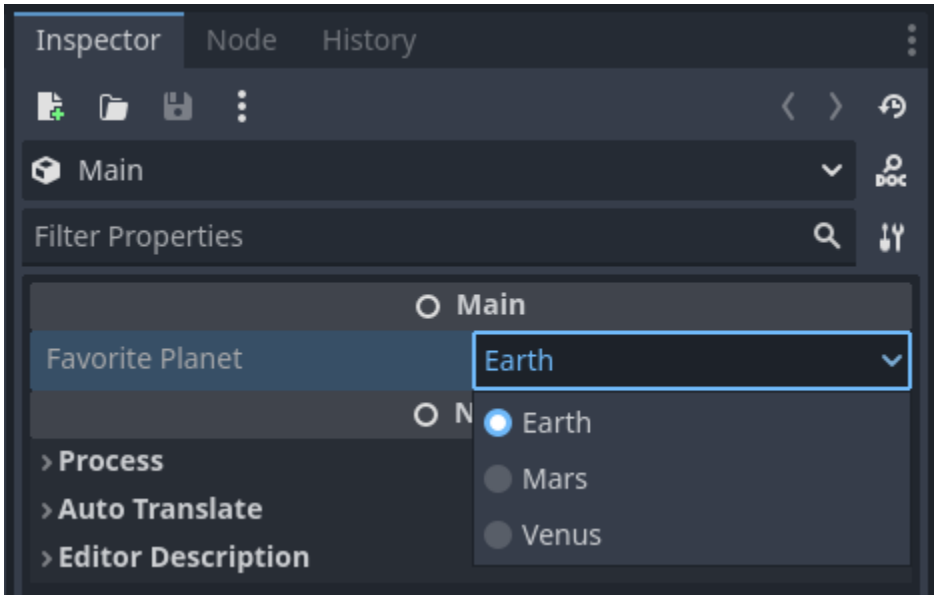
Godot does not have dedicated enum types, so you can map them either as integers (e.g. `i64`) or strings (`GString`). This can be configured using the `via` key of the `#[godot]` attribute.

Exporting an enum can be done as follows:

```
#[derive(GodotConvert, Var, Export, Default, Clone)]
#[godot(via = GString)]
pub enum Planet {
    #[default] // Rust standard attribute, not godot-rust.
    Earth,
    Mars,
    Venus,
}

#[derive(GodotClass)]
#[class(base=Node)]
pub struct SpaceFarer {
    #[export]
    favorite_planet: Planet,
}
```

The above will show up as follows in the editor UI:



Refactoring the Rust enum may impact already serialized scenes, so be mindful if you want to choose integers or strings as the underlying representation:

- Integers enable renaming variants without breaking existing scenes, however new ones must be strictly added at the end, and existing ones cannot be removed or reordered.
- Strings allow free reordering and removing (if unused) and make debugging easier. However, you cannot rename them, and they take slightly more space (only relevant if you have tens of thousands).

Of course, it is always possible to adjust existing scene files, but this involves manual search&replace and is generally error-prone.

⚠ Enums in GDScript

Enums are not first-class citizens in Godot. Even if you define them in GDScript, they are mostly syntactic sugar for constants. This declaration:

```
enum Planet {
    EARTH,
    VENUS,
    MARS,
}

@export var favorite_planet: Planet
```

is roughly the same as:

```
const Planet: Dictionary = {
    EARTH = 0,
    VENUS = 1,
    MARS = 2,
}

@export_enum("EARTH", "VENUS", "MARS") var favorite_planet = Planet.EARTH
```

However, the enum is not type-safe, you can just do this:

```
var p: Planet = 5
```

Furthermore, you can also not easily retrieve the name `"EARTH"` from the expression `Planet.EARTH`.[1]

See GDScript enums for more details.

Advanced usage

Both `#[var]` and `#[export]` attributes accept parameters to further customize how properties are registered in Godot. Consult the API documentation for details.

> **PackedArray mutability**
>
> `Packed*Array` types use copy-on-write semantics, meaning every new instance can be considered an independent copy. When a Rust-side packed array is registered as a property, GDScript will create a new instance of the array when you mutate it, making changes invisible to Rust code. There is a GitHub issue with more details.
>
> Instead, use `Array<T>` or register designated `#[func]` methods that perform the mutation on Rust side.

Custom types with `#[var]` and `#[export]`

If you want to register properties of user-defined types, so they become accessible from GDScript code (`#[var]`) or additionally from the editor (`#[export]`), then you can implement the `Var` and `Export` traits, respectively.

These traits also come with derive macros, `#[derive(Var)]` and `#[derive(Export)]` .

> **Performance**
>
> Enabling all sorts of types for `Var` and `Export` seems convenient, but keep in mind that your conversion functions are invoked every time the engine accesses the property, which may sometimes be behind the scenes. Especially for `#[export]` fields, interactions with the editor UI or serialization to/from scene files can cause a quite a bit of traffic.
>
> As a general rule, try to stay close to Godot's own types, e.g. `Array` , `Dictionary` or `Gd` . These are reference-counted or simple pointers.

Footnotes

[1] You *can* obtain `"EARTH"` if you iterate the `Planet` dictionary and compare each value (assuming there are no duplicates). That however requires that you know the type (`Planet`); the value itself does not hold this information.

Registering signals

Signals are a Godot mechanism to implement the Observer pattern. You can emit events, which are received by everyone who is subscribed ("connected") to the signal, decoupling sender and receiver. If you haven't worked with Godot signals before, you should definitely read the GDScript tutorial.

Table of contents

- The problem with GDScript signals
- Rust signals
  - Generated code
- Connecting signals
  - Signal + handler on same object `self`
  - Handler on different object
  - Handler without object (associated/static function)
- Disconnecting
- Emitting signals
- Accessing signals outside the class
  - Godot built-in signals
  - Signal visibility
  - Connecting from outside
  - Emitting from outside
- Advanced signal setups
  - Untyped signals
  - Signals in the editor – hot reload interaction
- Conclusion

The problem with GDScript signals

You can define GDScript signals as follows, with optional parameter names and types:

```
signal damage_taken
signal damage_taken(amount)
signal damage_taken(amount: int)
```

However, the difference between the above declarations is purely informational (e.g.

appears in class docs). Let's look at an example:

```
signal damage_taken(amount: int)

func log_damage():
    print("damaged!")

func _ready():
    damage_taken.connect(log_damage)
```

Note how `log_damage()` has no parameters, yet you can connect it without warning, neither at parse time nor at runtime.

This problem isn't limited to `connect()`; let's pass an argument of wrong type to `emit()`:

```
signal damage_taken(amount: int)

func log_damage(amount): # now with parameter
    print("damaged: ", amount)

func _ready():
    damage_taken.connect(log_damage)
    damage_taken.emit(true) # no int, no worries -> prints "damaged: true"
```

Again, GDScript happily passes through `bool`, despite the signal declaring `int`.

> **GDScript signals are not type-safe**
>
> In GDScript, a `signal` parameter list is **not type-checked**.
>
> Mismatching `connect()` or `emit()` calls may or may not be caught at runtime, based on the handler function's own typing. They are never caught at parse time.

While this seems like a minor issue in examples like the above, this becomes hard to track in bigger projects with many similar signals, especially once you start refactoring. A signal is designed to act as an API between the sender and receiver -- but there is no way to verify this interface contract, apart from a high level of manual discipline and testing.

Rust signals

godot-rust provides a type-safe and straightforward API to connect and emit signals, even

though the latter are untyped in GDScript. You can rely on signatures and don't need to fear refactorings, as Rust will catch any mismatches at compile time.

In godot-rust, signals can be defined with the `#[signal]` attribute inside a `#[godot_api]` block. Let's take again our class from earlier and declare a `damage_taken` signal:

```
#[derive(GodotClass)]
#[class(init, base=Node3D)]
struct Monster {
    hitpoints: i32,
    base: Base<Node3D>, // required when declaring signals.
}

#[godot_api]
impl Monster {
    #[signal]
    fn damage_taken(amount: i32);
}
```

Signal syntax is close to `#[func]`, but it needs a semicolon instead of a function body. Receivers (`&self`, `&mut self`) and return types are not supported.

Generated code

As soon as you register at least one signal, godot-rust will implement the `WithUserSignals` trait for your class. This provides the `signals()` method, which can now be accessed inside class methods.

`signals()` returns a *signal collection*, i.e. a struct which exposes all signals as named methods:

```
// Generated code ($ are placeholders, actual names up to implementation):
impl $SignalCollection {
    fn damage_taken(&mut self) -> $Signal {...}
}

#[godot_api]
impl INode3D for Monster {
    fn ready(&mut self) {
        let sig = self.signals().damage_taken();
    }
}
```

The `damage_taken()` method returns a custom-generated *signal type* (referred to as `$Signal` in the snippet), whose API is tailored to the signature of `fn damage_taken(amount:`

`i32)`. Each `#[signal]` attribute generates a distinct signal type.

The signal type is implementation-defined. Besides the `#[signal]`-specific custom API, it also implements `Deref/DerefMut` with target `TypedSignal`, meaning you can additionally use all *those* methods on each signal type.

> **Availability of signal API**
>
> For typed signals to be available, you need:
>
> - A `#[godot_api] impl MyClass {}` block.
>   - This must be an inherent impl, the `I*` trait `impl` won't be enough.
>   - Leave the impl empty if necessary.
> - A `Base<T>` field.
>
> Signals, typed or not, **cannot** be declared in secondary `impl` blocks (those annotated with `#[godot_api(secondary)]` attribute).

Connecting signals

godot-rust offers many ways to connect signals, depending on where the handler function is located.

Signal + handler on same object `self`

Connecting signals to methods of the same class is quite common. This is possible with the `connect_self()` method, which simply takes the method pointer as an argument:

```
impl Monster {
    fn on_damage_taken(&mut self, amount: i32) {
        ... // Update healthbar, play sound, etc.
    }
}

#[godot_api]
impl INode3D for Monster {
    fn ready(&mut self) {
        self.signals()
            .damage_taken()
            .connect_self(Self::on_damage_taken);
    }
}
```

Note how `on_damage_taken` has no `#[func]` attribute, and its surrounding impl block no `#[godot_api]` proc-macro. Signal receivers are regular Rust functions! You can completely hide them from Godot, and only make them accessible via signals.

Since `connect_self()` 's parameter here is essentially `impl FnMut(&mut Self, i32)`, you can also pass a closure:

```
#[godot_api]
impl INode3D for Monster {
    fn ready(&mut self) {
        self.signals()
            .damage_taken()
            .connect_self(|this, amount| {
                //         ^^^^  ^^^^^^
                //         types inferred as &mut Self, i32

                this.update_healthbar(amount);
                this.play_sound(Sfx::MonsterAttacked);
            });
    }
}
```

> **One signal at a time**
>
> If you want to connect multiple signals, call `self.signals()` repeatedly. You cannot store it in a variable for reuse.

Handler on different object

If the handler function should run on an object other than `self`, you can use `connect_other()`, which takes a `&Gd<T>` as first argument:

```
#[godot_api]
impl INode3D for Monster {
    fn ready(&mut self) {
        // Let's say damage is deflected to a shield object.
        // That one is stored as field `shield: OnReady<Gd<Shield>>`.

        // We clone it into a pointer to not have borrow errors later, as
`self`
        // is already borrowed by `self.signals()`. The * is OnReady deref.
        let shield: Gd<Shield> = (*self.shield).clone();

        self.signals()
            .damage_taken()
            .connect_other(&shield, Shield::on_damage_taken);
    }
}
```

Handler without object (associated/static function)

If the handler function does not need access to `self`, simply use `connect()`:

```
impl Monster {
    // Now an associated function, no longer a method.
    fn on_damage_taken(amount: i32) {
        // Does not modify the object itself, but updates
        // some global statistics.
    }
}

#[godot_api]
impl INode3D for Monster {
    fn ready(&mut self) {
        self.signals()
            .damage_taken()
            .connect(Self::on_damage_taken);

        // Or with closures:
        self.signals()
            .damage_taken()
            .connect(|amount| {
                // Update global statistics.
            });
    }
}
```

Disconnecting

Connecting a signal in any aforementioned way returns a `ConnectHandle`, which can be used for disconnecting.

```
#[derive(GodotClass)]
#[class(init, base=Node3D)]
struct Monster {
    handle: Option<ConnectHandle>,
    base: Base<Node3D>,
}

#[godot_api]
impl INode3D for Monster {
    fn ready(&mut self) {
        self.handle = Some(self.signals()
            .damage_taken()
            .connect(Self::on_damage_taken));
    }
}

impl Monster {
    fn make_invulnerable(&mut self) {
        if let Some(connection) = self.handle.take()
        {
            connection.disconnect();
        }
    }
}
```

Disconnecting a signal will result in panic if given connection does not exist. Use `is_connected()` when there is no clear ownership of the signal or connected object is expected to be freed.

```rust
#[derive(GodotClass)]
#[class(init, base=Node3D)]
struct Commentator {
    handle: Option<ConnectHandle>,
    base: Base<Node3D>,
}

impl Commentator {
    fn watch_monster(&mut self, monster: Gd<Monster>) {
        if let Some(connection) = self.handle.take()
            && connection.is_connected()
        {
            connection.disconnect();
        }

        monster
            .signals()
            .damage_taken()
            .connect_other(&self.to_gd(), Self::comment);
    }

    fn comment(&mut self, damage_taken: i32) {
        match damage_taken {
            ..0 => {
                return;
            }
            0..3 => {
                godot_print!("Oh!");
            }
            3..10 => {
                godot_print!("That must hurt!");
            }
            10.. => {
                self.handle
                    .take()
                    .expect("Signal mustn't be connected without the handle!")
                    .disconnect();
            }
        }
    }
}
```

Emitting signals

We already saw that `#[signal]` attributes generate a signal type with several methods: `connect()`, `connect_self()` and `connect_other()`. This same signal type also provides an `emit()` method, which you can use to trigger the signal:

```
impl Monster {
    // Can be invoked by other game systems.
    pub fn deal_damage(&mut self, amount: i32) {
        self.hitpoints -= amount;
        self.signals().damage_taken().emit(amount);
    }
}
```

Like `connect*()` methods, `emit()` is fully type-safe. You can only pass a single `i32`. If you update your signal definition, e.g. to take a `bool` or `enum` value for the type of damage, the compiler will catch all `connect*` and `emit` calls. You'll sleep well after refactorings.

The nice thing about `emit()` is that it also comes with parameter names, as provided in the `#[signal]` attribute. This lets IDEs provide more context, e.g. show parameter inlay hints in `emit()` calls. The parameter types use the `AsArg<T>` trait, which follows engine APIs and provides flexibility in the argument types. For example, `"string"` can be passed for `impl AsArg<GString>`.

In addition to the specific `emit()` method, the `TypedSignal` (deref target of the custom signal type) also provides a generic method `emit_tuple()`, which takes a tuple of all arguments, by value. This is rarely needed, but can be useful in situations where you want to pass multiple arguments as a "bundle". Just for completeness, the above call is equivalent to:

```
self.signals().damage_taken().emit_tuple((amount,));
```

Accessing signals outside the class

As your game grows in interactions, you may want to configure or emit signals not just within `impl Monster` blocks, but also from other parts of your codebase. The trait method `WithUserSignals::signals()` allows direct access from `&mut self`, but outside you often only have a `Gd<Monster>`. You could technically `bind_mut()` that object, but there's a better way without borrow-checking.

For this reason, `Gd` itself *also* provides a `signals()` method, returning the exact same *signal collection* API:

```
let monster: Gd<Monster> = ...;
let sig = monster.signals().damage_taken();
```

Godot built-in signals

Godot provides many built-in signals to hook into lifecycles and events. All engine-provided classes implement the `WithSignals` trait, which is a supertrait of `WithUserSignals`.

Every class `T` has its own signal collection, accessible by `Gd<T>::signals()`. Like class methods, signals are inherited, so you can do the following:

```
// tree_entered is a signal declared on Node.
let node: Gd<Node> = ...;
let sig = node.signals().tree_entered();

// You can also access it from a derived class.
let node: Gd<Node3D> = ...;
let sig = node.signals().tree_entered();
```

This works also in user-defined classes. This means we can extend our previous `ready()` implementation to connect Godot signals:

```
#[godot_api]
impl INode3D for Monster {
    fn ready(&mut self) {
        // Previous code.
        self.signals()
            .damage_taken()
            .connect_other(&*self.shield, Shield::on_damage_taken);

        // Connect to the `Node::renamed` signal, which is invoked
        // when a node name changes.
        self.signals()
            .renamed()
            .connect_self(|this| {
                let new_name = this.base().get_name();
                println!("Monster node renamed to {new_name}.");
            });
    }
}
```

> **Disabling typed signals**
>
> The generated API for typed signals usually does no harm even if you don't use it. However, it is possible to disable code generation with:
>
> ```
> #[godot_api(no_typed_signals)]
> impl MyClass { ... }
> ```
>
> This still allows you to use `#[signal]` and will register each signal declared as such, but it won't generate a `signals()` collection.

Signal visibility

Like all items in Rust, signals are private by default, i.e. only visible in their module and submodules. You can make them public by adding `pub` to the `#[signal]` attribute:

```
#[godot_api]
impl Monster {
    #[signal]
    pub fn damage_taken(amount: i32);
}
```

Of course, `pub(crate)`, `pub(super)` or `pub(in path)` are also possible for more fine-grained control.

> **Exceeding visibility**
>
> `#[signal]` visibility **must not exceed** class visibility.
>
> If you get errors such as "can't leak private type", then you violated this rule.

So, if your class is declared as `struct Monster` (private), then you cannot declare signals as `pub` or `pub(crate)`. This is due to a technical limitation resulting from signals being separate types, which refer to the class type in their APIs. Making them "more public" than the class would thus circumvent Rust's privacy rules.

Semantically, it makes sense though: the only situation where you'd need outside access is through `Gd<SomeClass>::signals()`, and this implies that `SomeClass` is visible at that point. But unlike other Rust items such as `fn`, wider visibility isn't automatically limited to "at most

struct visibility", but causes a compile error.

Note that you cannot separate the visibility of connect and emit APIs. If you want to make sure that outsiders can only emit, keep the signal private and provide a public wrapper function in your class that forwards the call to the signal.

Connecting from outside

Let's say you have a sound system which should play a sound effect whenever a monster takes damage. You can connect to the signal from there:

```
impl SoundSystem {
    fn connect_sound_system(&self, monster: &Gd<Monster>) {
        monster.signals()
            .damage_taken()
            .connect_other(self, |this, _amount| {
                this.play_sound(Sfx::MonsterAttacked);
            });
    }
}
```

Emitting from outside

Like connecting, emitting can also happen through `Gd::signals()`. The rest remains the same.

```
fn load_map() {
    // All the loading.
    ...

    // Notify player that the world around is now loaded.
    let player: Gd<Player> = ...;
    player.signals().world_loaded().emit();
}
```

Advanced signal setups

The `TypedSignal::connect*()` methods are designed to be straightforward, while covering common use cases. If you need more advanced setups, a high degree of customization is provided by `TypedSignal::builder()`.

The returned `ConnectBuilder` provides several dimensions of configurability:

- Receiver parameter: `function(args)`, `method(&mut T, args)`, `method(Gd<T>, args)`
- Provided object: none, `self` or other instance
- Connection flags: `DEFERRED`, `ONESHOT`, `PERSIST`
- Single-threaded (default) or thread-crossing ("sync")

Some example setups:

```
// Connect -> Self::log_event(&mut self, event: String)
signal.builder()
    .flags(ConnectFlags::DEFERRED | ConnectFlags::ONESHOT)
    .connect_self_mut(Self::log_event); // receive &mut self

// Connect -> Logger::log_event(&mut self, event: String)
signal.builder()
    .connect_other_mut(some_gd, Logger::log_event);

// Connect -> Logger::log_event(this: Gd<Self>, event: String)
signal.builder()
    .connect_other_gd(some_gd, Logger::log_event);

// Connect -> Logger::log_thread_safe(event: String)
signal.builder()
    .connect_sync(Logger::log_thread_safe); // associated fn, no receiver
```

The builder methods need to be called in the correct order ("stages"). See API docs for more information.

Untyped signals

Godot's low-level APIs for dealing with untyped signals are still available:

- `Object::connect()`
- `Object::connect_flags()`
- `Object::emit_signal()`
- `Signal::connect()`
- `Signal::emit()`

The new typed-signal API should cover the full functionality, but there are situations where information is only available at runtime, making the untyped reflection APIs a good fit. We might also combine the two in the future.

One way to connect signals the old-school is passing in the signal name and the `Callable`:

```
let monster: Gd<Monster> = ...;
let damage_taken: Callable = monster.callable("damage_taken");
monster.connect("damage_taken", &damage_taken);
```

To emit an untyped signal, call the `Object::emit_signal()` method by (mutably) accessing the base class. Staying with the `Monster` struct from the previous examples, you can emit its signal with:

```
self.base_mut().emit_signal(
    "damage_taken",
    vslice![amount_damage_taken],
);
```

See `vslice!` docs for passing multiple variants in a slice.

Certain untyped-signal functionality may still be ported typed signals, while others such as `Callable::bind()` will likely not be available. Just use closures instead. Generally speaking, the `TypedSignal` and `ConnectBuilder` APIs are designed to be extensible for your own workflows.

Signals in the editor – hot reload interaction

Typed signals use custom callables for connections, which contain a vtable – a set of function pointers pointing to Godot FFI functions. After hot reload, all these function pointers become invalid, and invoking any of them results in undefined behavior, usually leading to crash.

To prevent unsoundness, godot-rust automatically disconnects all typed signal connections (i.e. created using anything other than the untyped signals API) before a hot reload. These connections can be recreated afterwards, typically using the `EXTENSION_RELOADED` notification.

```
#[derive(GodotClass)]
#[class(init, base = Node)]
struct InEditorMonster {
    // Properties are being preserved by the hot reload,
    // allowing us to recrate signals afterward.
    #[var]
    managed_editor_objects: Array<Gd<Object>>,

    base: Base<Node>
}

#[godot_api]
impl INode for InEditorMonster {
    fn on_notification(&mut self, what: NodeNotification) {
        if what == NodeNotification::EXTENSION_RELOADED {
            // Clone the pointer to the array to avoid partial borrow issues -
`signals()` require &mut reference.
            // Do it only when you can ensure that nothing will modify the
array during iteration.
            for obj in self.managed_editor_objects.clone().iter_shared() {
                self.signals().my_signal().builder().connect_other_gd(&obj,
some_fn);
            }
        }
    }
}
```

Alternatively, untyped signals – which are preserved during hot reload – with **standard** (non-custom) callables can be used. At the moment `Callable::from_object_method` or `Gd::callable()` can be used to create standard callables.

```
let monster: Gd<InEditorMonster> = ...;
let non_custom_callable: Callable = monster.callable("damage_taken");

// Will not cause any problems upon hot reload.
monster.connect("damage_taken", &non_custom_callable);

let custom_callable: Callable = Callable::from_fn(
    "my custom callable",
    |_| godot_print!("hello world!")
);

// Invoking this signal after hot reload will lead to UB!
monster.connect("damage_taken", &custom_callable);
```

Conclusion

In this chapter, we saw how godot-rust's **typed signals** provide an intuitive and resilient way to deal with Godot's observer pattern and avoid certain pitfalls of GDScript. Rust function references or closures can be directly connected to signals, and emitting is achieved with regular function calls.

Registering constants

Constants can be used to share fixed values from Rust code to the Godot engine.

See also GDScript reference for constants.

Constant declaration

Constants are declared as `const` items in Rust, inside the inherent `impl` block of a class. `static` declarations cannot be used.

The attribute `#[constant]` makes the constant available to Godot.

```
#[godot_api]
impl Monster {
    #[constant]
    const DEFAULT_HP: i32 = 100;

    #[func]
    fn from_name_hp(name: GString, hitpoints: i32) -> Gd<Self> { ... }
}
```

Usage in GDScript would look as follows:

```
var nom = Monster.from_name_hp("Nomster", Monster.DEFAULT_HP)
var orc = Monster.from_name_hp("Orc", 200)
```

(This particular example might be better suited for default parameters once they are implemented, but it illustrates the point.)

Limitations

Godot supports **only integers** to be registered as constants via GDExtension API.

You can work around this by registering a static function, called as `Monster.DEFAULT_NAME()` in GDScript.

```
#[godot_api]
impl Monster {
    #[func(rename = "DEFAULT_NAME")]
    fn default_name() -> GString {
        "Monster_001".into()
    }
}
```

While you could technically use read-only properties, this is problematic because:

- You need an existing instance of the class.
- Every object occupies space for the constant.[1]

It's really just syntax, an extra `()` will not derail your game. If you have a heavier value that you don't want to recompute (e.g. array), you can always store it in a `thread_local!` in Rust.

Footnotes

[1] In the future, we may have properties that don't occupy space, see #1151.

Script-virtual functions

The GDExtension API allows you to define virtual functions in Rust, which can be overridden in scripts attached to your objects.

Note that these are conceptually different from virtual functions like `ready()`, which are defined *by Godot* and overridden *by you* (in Rust). Hence the emphasis on "script-virtual".

> **Compatibility**
>
> This feature is available from Godot 4.3 onwards.

Table of contents

- A motivating example
- Step by step
  - Rust default behavior
  - Overriding in GDScript
  - Dynamic behavior
- Trade-offs
- Limitations
- Types of scripts
- Conclusion

A motivating example

To stay with our `Monster` example, let's say we have different monster types and would like to customize their behavior. We can write the logic common to all monsters in Rust, and for quick prototyping use GDScript for the specific parts.

For example, we can experiment with two monsters: `Orc` and `Goblin`. Each of them comes with a different behavior, which is encoded in a respective GDScript file. The project structure might look like this:

```
project_dir/
│
├── godot/
│   ├── .godot/
│   ├── project.godot
│   ├── MonsterGame.gdextension
│   └── Scenes
│       ├── Monster.tscn
│       ├── Orc.gd
│       └── Goblin.gd
│
└── rust/
    ├── Cargo.toml
    └── src/
        ├── lib.rs
        └── monster.rs
```

The `Monster.tscn` encodes a simple scene with the node `Monster` (our Rust class inheriting `Node3D`) at the root. This node would be the one to attach scripts to.

Step by step

Rust default behavior

Let's start from this class definition:

```
use godot::prelude::*;

#[derive(GodotClass)]
#[class(init, base=Node3D)]
struct Monster {
    base: Base<Node3D>
}
```

We can now implement a Rust function to calculate the damage a monster deals per hit. Traditionally, we would write this:

```
#[godot_api]
impl Monster {
    #[func]
    fn damage(&self) -> i32 {
        10
    }
}
```

That method will always return `10`, no matter what. To customize this behavior in scripts that are attached to the `Monster` node, we can define a *virtual method* in Rust, which can be *overridden* in GDScript. The Rust code is called the *default* implementation.

> **Early vs. late binding**
>
> *Virtual* (also called *late-binding*) means that dynamic dispatch is involved: the actual method to call is determined at runtime, depending on whether a script is attached to the `Monster` node -- and if yes, which one.
>
> This stands in contrast to *early-binding*, which is resolved at compile time, using static dispatch.

While traditional Rust might use trait objects (`dyn Trait`) for late binding, godot-rust provides a more direct way. Making a method virtual is very easy: just add the `virtual` key to the `#[func]` attribute.

```
#[godot_api]
impl Monster {
    #[func(virtual)]
    fn damage(&self) -> i32 {
        10
    }
}
```

That's it. Your monster can now be customized in scripts.

Overriding in GDScript

In the GDScript files, you can now override the Rust `damage` method as `_damage`. The method is prefixed with an underscore, following Godot convention for virtual methods such as `_ready` or `_process`.

Here's an example for the `Orc` and `Goblin` scripts:

```
# Orc.gd
extends Monster

func _damage():
    return 20


# Goblin.gd
extends Monster

# Random damage between 5 and 15.
# Type annotations are possible, but not required.
func _damage() -> int:
    return randi() % 11 + 5
```

If your signature in GDScript does not match the Rust signature, Godot will cause an error.

Dynamic behavior

Now, let's call `damage()` in Rust code:

```
fn monster_attacks_player(monster: Gd<Monster>, player: Gd<Player>) {
    // Compute the damage.
    let damage_points: i32 = monster.bind().damage();

    // Apply the damage to the player.
    player.bind_mut().take_damage(damage_points);
}
```

What value does `damage_points` have in the above example?
The answer depends on the circumstances:

- If the `Monster` node has no script attached, `damage_points` will be `10` (the default implementation in Rust).
- If the `Monster` node has the `Orc.gd` script attached, `damage_points` will be `20`.
- If the `Monster` node has the `Goblin.gd` script attached, `damage_points` will be a random number between `5` and `15`.

Trade-offs

You might ask: what's the point of all this, if one can achieve the same with a simple `match` statement?

And you're right; if a `match` in Rust is all you need, then use that. However, the script-based approach has a few advantages, especially when it comes to more complex scenarios than just computing a single damage number:

- You can prepare a variety of scripts with different behaviors, e.g. for different levels or enemy AI behavior. In the Godot editor, you can then simply swap out scripts as needed, or have different `Monster` instances with different scripts, to compare them side-by-side.
- Switching behaviors does not require recompiling Rust code. This can be useful if you work with game designers, modders or artists who are less familiar with Rust, but want to experiment nonetheless.

That said, if your compile times are short (godot-rust itself is quite lightweight) and you prefer having the logic in Rust, that is of course also a valid choice. To retain the option to quickly switch behaviors, you could use an `#[export]` 'ed enum to select the behavior, and then dispatch on that in Rust.

Ultimately, `#[func(virtual)]` is just one extra tool that godot-rust offers among a variety of abstraction mechanisms. Since Godot's paradigm revolves heavily around attaching scripts to nodes, this feature integrates very well with the engine.

Limitations

> **⚠ Warning**
>
> Godot script-virtual functions do not behave like OOP virtual functions in every aspect. Make sure you understand the limitations.

In contrast to virtual methods from OOP languages (C++, C#, Java, Kotlin, PHP, ...), there are some important differences to be aware of.

1. **The default implementation is unreachable from Godot.**

   In Rust, calling `monster.bind().damage()` will automatically look for script overrides, and fall back to the Rust default if no script is attached. In GDScript however, you cannot call the default implementation. Calling `monster._damage()` will fail without a

script. The same is true for reflection calls from Rust (e.g. `Object::call()`).

The `_` prefix underlines that: ideally, you don't call virtual functions directly from scripts.

To work around this, you can declare a separate `#[func] fn default_damage()` in Rust, which will be registered as a regular method and thus can be called from scripts. To keep Rust's convenient fallback behavior, just invoke `default_damage()` inside the Rust `damage()` method.

2. **No access to `super` methods.**

   In OOP languages, you can call the base method from the overriding method, typically using `super` or `base` keywords.

   As a consequence of point 1), this default method is also not visible to the script overriding it. The same workaround can be used though.

3. **Limited re-entrancy.**

   If you call a virtual method from Rust, it may dispatch to a script implementation. The Rust side holds either a shared (`&self`) or exclusive borrow (`&mut self`) to the object -- an implicit `Gd::bind()` or `Gd::bind_mut()` guard. If the script implementation then accesses the same object (e.g. by setting a `#[var]` property), panics can occur due to double-borrow errors.

   For now, you can work around this by declaring the method with `#[func(gd_self, virtual)]`. The `gd_self` requires the first parameter to be of type `Gd<Self>`, which avoids the bind call and thus the borrow.

We are observing how virtual functions are used by the community and plan to mitigate the limitations where possible. If you have any inputs, feel free to let us know!

Types of scripts

While this page focuses on GDScript, Godot also provides other scripting capabilities. Notably, C# can be used for scripting, if you run Godot with the Mono runtime.

The library also provides a dedicated trait `ScriptInstance`, which allows users to provide Rust-based "scripts". Consult its docs for detailed information.

You can also configure scripts entirely programmatically, using the `classes::Script` API

and its inherited classes, such as `classes::GDScript`. This typically defeats the purpose of scripting, but is mentioned here for completeness.

Conclusion

In this chapter, we have seen how to define virtual functions in Rust, and how to override them in GDScript. This provides an additional integration layer between the two languages and allows to effortlessly experiment with swappable behaviors from the editor.

Toolchain

Beyond Rust, there are quite a few things that are handy to know when working with Godot. This chapter goes into more detail about them, covering topics such as versioning, compatibility or debugging.

Check out the subchapters for more information.

Compatibility and stability

The godot-rust library supports multiple stable Godot releases at a time.

- Compatibility with Godot
  - Current guarantees
  - Compatibility matrix
  - Philosophy
  - Out of scope
- Rust API stability

Compatibility with Godot

When developing extension libraries (or just "extensions"), you need to consider which engine version you want to target. There are two conceptually different versions:

- **API version** is the version of GDExtension against which your extension is **compiled**.
- **Runtime version** is the version of Godot in which the library built with godot-rust is **run**.

The two versions can be different, with some constraints elaborated below.

Current guarantees

Latest godot-rust requires at least **Godot 4.2**.

Starting from that version's official release, extensions can be loaded by any Godot version, as long as *runtime version* **>=** *API version*. In other words, you can run existing extensions in newer Godot versions without needing to change anything.

- You **can** run a `4.2` extension in Godot `4.2.1` or `4.3`.
- You **cannot** run a `4.3` extension in Godot `4.2.1`.

As long as the GDExtension API evolves in a backward-compatible manner -- which it has remarkably achieved since Godot 4.1 -- we will try our best to keep up this guarantee. If you notice any discrepancies, please report them to us.

Compatibility matrix

We typically provide support for Godot versions for 1-2 years after their release, depending on feature set and maintenance effort. For example, Godot 4.0 extensions are binary-incompatible with newer versions and thus provide very little value. Godot 4.1 also lacks foundational features necessary for Rust callables, typed signals, hot reloading and much more.

If you need to support an older Godot version, you can fall back to older godot-rust releases. These won't receive any more updates however, not even for critical bugs.

godot-rust version	minimum Godot version	Godot release date[1]
0.4+	4.2	November 2023
0.2, 0.3	4.1	July 2023
0.1	4.0[2]	March 2023

Make sure to use the appropriate `api-4-*` feature flag, see *Selecting a Godot version*.

Philosophy

We take compatibility with the engine seriously, in an attempt to build an ecosystem of extensions that are interoperable with multiple Godot versions. Nothing is more annoying than updating the engine and recompiling 10 plugins/extensions.

This is sometimes difficult, because:

- Godot may introduce subtle breaking changes of which we are not aware.
- Some changes that are non-breaking in C++ and GDScript are breaking in Rust (e.g. providing a default value for a previously required parameter).
- Using newer features needs to come with a fallback/polyfill for older Godot versions.

We run CI jobs against multiple Godot versions, to get a certain level of confidence that updates do not break compatibility. Nevertheless, the number of possible combinations is large and only growing, so we may miss certain issues. If you find incompatibilities or violations of the rules stated below, please let us know.

Out of scope

We do **not** invest effort in maintaining compatibility with:

1. Godot in-development versions, except for the latest `master` branch.

    - Note that we may take some time to catch up with the latest changes, so please don't report issues within a few days after upstream changes have landed.

2. Non-stable releases (alpha, beta, RC).

3. Third-party bindings or GDExtension APIs (C#, C++, Python, ...).

    - These may have their own versioning guarantees and release cycles; and there may be specific bugs to such an integration. If you find an issue with godot-rust and another binding, reproduce it in GDScript to make sure it's relevant for us.
    - We do however maintain compatibility with Godot, so if integrations go through the engine (e.g. Rust calls a method whose implementation is in C#), this should work.

4. Godot with non-standard build flags (e.g. disabled modules).

5. Godot forks or engines running third-party modules.

Rust API stability

A lot of godot-rust's foundation has been be built and is in a production-ready state. However, we still regularly add new features, and sometimes refine existing APIs.

As such, **expect occasional breaking changes**. These are usually minor and will be announced in both changelog and migration guides. We additionally work with deprecations in our API, allowing smooth transitions.

Note that if breaking changes occur, they are externally motivated, for example:

- GDExtension changes in a way that cannot be abstracted from the user.
- There are subtleties in the type system or runtime guarantees that can be modeled in a better, safer way (e.g. typed arrays, RIDs).
- We get feedback from game developers and other users stating that certain workflows are very cumbersome.

Our crates.io releases adhere to SemVer, but may lag behind the master branch.

Footnotes

[1] See *Release history* on Wikipedia.

[2] Every extension developed with API version `4.0.x` **MUST** be run with the same runtime version. In particular, it is not possible to run an extension compiled with API version `4.0.x` in Godot 4.1 or later. This is due to breaking changes in Godot's GDExtension API.

Selecting a Godot version

Supporting multiple Godot versions is a key feature of godot-rust. Especially if you plan to share your extension with others (as a library or an editor plugin), this page elaborates your choices and their trade-offs in detail. The previous chapter about compatibility is expected as a prerequisite.

Table of contents

- Motivation
  - Why support multiple versions?
  - Cutting edge vs. compatibility
- Selecting the API version in godot-rust
  - Default version
  - Lower minor version
  - Lower or higher patch version
- Custom Godot versions
  - Setting `GODOT4_BIN` to a relative path

Motivation

To refresh, you have two Godot versions to consider:

- **API version**, against which godot-rust compiles.
  - Affects Rust symbols (classes, methods, etc.) you have available at compile time.
  - This sets a lower bound on the Godot binary you can run your extension in.
- **Runtime version**, the Godot engine version, in which you run the Rust extension.
  - Affects the runtime behavior, e.g. newer versions may fix some bugs.
  - It is advised to stay up-to-date with Godot releases, to benefit from new improvements.

GDExtension is designed to be backward-compatible, so an extension built with a certain API version can be run in all Godot binaries greater than that version.[1] Therefore, the lower your API version, the more Godot versions you support.

> **In other words:**
>
> API version <= runtime version

Why support multiple versions?

The choice you have in the context of godot-rust is the **API version**. If you just make a game on your own, the defaults are typically good enough.

Explicitly selecting an API version can be helpful in the following scenarios:

1. You run/test your application on different Godot **minor** versions.
2. You are collaborating in a team, or you want to give your Godot project to friends to experiment with.
3. You work on a library or plugin to share with the community, either open-source (distributed as code) or closed-source (distributed as compiled dynamic library).

Especially in the last case, you may want your extension to be compatible with as many Godot versions as possible, to reach a broader audience.

> **Building an ecosystem**
>
> At first glance, it may not seem obvious why a plugin would support anything but the latest Godot version. After all, users can just update, right?
>
> However, sometimes users cannot update their Godot version due to regressions, incompatibilities or project/company constraints.
>
> Furthermore, imagine you want to use two GDExtension plugins: **X** (API level 4.3) and **Y** (4.2). Unfortunately, Y contains a bug that causes some issues with Godot 4.3. This means you cannot use both together, and you are left with some suboptimal choices:
>
> - Only use X on 4.3.
> - Only use Y on 4.2.
> - Help the author of Y to patch the bug. But they may just sail the Caribbean and not respond on their repo. Or worse, Y might even be a closed-source plugin that you paid for.
>
> Not only are you now left with a less-than-ideal situation, but you cannot build *your own tool* Z which uses both X and Y, either. Had X declared API 4.2, people could stick to that version until Y is fixed, and you too could release Z with API 4.2.
>
> A longer compatibility range gives users more flexibility regarding *when* they update *what*. It accounts for the fact that developers iterate at varying pace, and enables projects to depend on each other. At scale, this enables a vibrant ecosystem of extensions around Godot.

Cutting edge vs. compatibility

Lower API versions allow supporting a wider range of Godot versions. For example, if you set the API version to 4.2, you can run it in Godot 4.2, 4.2.2 or 4.3, but not Godot 4.1.

On the flip side, lower API versions reduce the API surface that you can statically[2] use in your Rust extension. If you select 4.2, you will not see classes and functions introduced in 4.3.

This is the core trade-off, and you need to decide based on your use case. If you are unsure, you can always start with a conservatively low API version, and bump it when you find yourself needing more recent features.

Selecting the API version in godot-rust

Now that the *why* part is clarified, let's get into *how* you can choose the Godot API version.

Default version

By default, godot-rust uses the **current minor release** of Godot 4, with patch 0. This ensures that it can be run with all Godot patch versions for that minor release.

Example: if the current release is Godot 4.3.5, then godot-rust will use API version 4.3.0.

Lower minor version

To change the API level to a lower version, simply turn on the Cargo feature `api-4-x`, where `x` is the minor version you want to target.

Example in Cargo.toml:

```
[dependencies]
# API level 4.2
godot = { ..., features = ["api-4-2"] }
```

You can also explicitly set the current minor version (the same as the default). This has the advantage that you keep that compatibility, even once godot-rust starts targeting a newer version by default.

> **Mutual exclusivity**
>
> Only one `api-*` feature can be active at any time.

Lower or higher patch version

godot-rust supports API version granularity on a patch level, if absolutely needed. This is rarely necessary and can cause confusion to users, so only select a patch-level API if you have a very good reasons. Also note that GDExtension itself is only updated in minor releases.

Reasons to want this might be:

- Godot ships a bugfix in a patch version that is vital for your extension to function properly.
- A new API is introduced in a patch version, and you would like its class/function definitions. This happens quite rarely.

To require a minimum patch level, use a `api-4-x-y` feature:

```
[dependencies]
# API level 4.2.1
godot = { ..., features = ["api-4-2-1"] }
```

Custom Godot versions

If you want to freely choose a Godot binary on your local machine from which the GDExtension API is generated, you can use the Cargo feature `api-custom`. If enabled, this will look for a Godot binary in two locations, in this order:

1. The environment variable `GODOT4_BIN`.
2. The binary `godot4` in your `PATH`.

Generated code inside the `godot::builtin`, `godot::classes` and `godot::global` modules may now look different from stable releases. Note that we do not give any support or compatibility guarantees for custom-built GDExtension APIs.

Working with the `api-custom` feature requires the `bindgen` crate, as such you may need to install the LLVM toolchain. Consult the setup page for more information.

Setting `GODOT4_BIN` to a relative path

If you have multiple Godot workspaces on a machine, you may want a workspace-independent method of setting the `GODOT4_BIN` environment variable. This way, the matching Godot editor binary for that workspace is always used in the build process, without having to set `GODOT4_BIN` differently for each location.

You can do this by configuring Cargo to set `GODOT4_BIN` to a relative path for you, in `.cargo/config.toml`.

In the root of your Rust project, create `.cargo/config.toml` with the example content shown below, modifying the editor path as needed to find your binary. The path you set will

be resolved relatively to the location of the `.cargo` directory.

```
[env]
GODOT4_BIN = { value = "../godot/bin/godot.linuxbsd.editor.x86_64", relative =
true, force = true }
```

(If you want to override `config.toml` by setting `GODOT4_BIN` in your environment, remove `force = true`.)

Test your change by running `cargo build`.

See The Cargo Book for more information on customizing your build environment with `config.toml`.

Footnotes

[1] Godot 4.0 has been released before the GDExtension API committed to stability, so no single 4.0.x release is compatible with any other release (not even patch versions among each other). We provide 4.0 API levels, but due to their limited utility, we will phase out support very soon.

[2] Even if your API level is 4.2, it is possible to access 4.3 features, but you need to do so dynamically. This can be achieved using reflection APIs like `Object::call()`, but you lose the type safety and convenience of the statically generated API. To obtain version information, check out the `GdextBuild` API.

Debugging

Extensions written in godot-rust can be debugged using LLDB, in a similar manner to other Rust programs. The primary difference is that LLDB will launch or attach to the Godot C++ executable: either the Godot editor or your custom Godot application. Godot then loads your extension (itself a dynamic library), and with it your Rust code.

The process for launching or attaching LLDB varies based on your IDE and platform. Unless you are using a debug version of Godot itself, you will only have symbols for stack frames in Rust code.

Launching with VS Code

Here is an example debugger setup for Visual Studio Code. Launch configurations and tasks should be placed in `./.vscode/launch.json` and `./.vscode/tasks.json` respectively, relative to your project's root.

This example assumes you have the CodeLLDB extension installed, which is common for Rust development.

For commands like `-w` (windowed mode) or `-e` (editor), refer to Godot's command line tutorial.

```
// ./.vscode/launch.json
{
    "configurations": [
        {
            "name": "Debug Project (Godot 4)",
            "type": "lldb", // type provided by CodeLLDB extension
            "request": "launch",
            "cwd": "${workspaceFolder}/godot",
            "preLaunchTask": "build-rust",
            "args": [
                "-w" // windowed mode
                // "-e": editor does not keep breakpoints; thus not listed
here.
            ],
            "program": "PATH/TO/GODOT"
        }
    ]
}
```

```
// ./.vscode/tasks.json
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "build-rust",
            "command": "cargo",
            "args": [
                "build"
            ],
            "options": {
                "cwd": "${workspaceFolder}/rust"
            }
        }
    ]
}
```

Debugging on macOS

Attaching a debugger to an executable that wasn't compiled locally (the Godot editor, in this example) requires special considerations on macOS due to its *System Integrity Protection* (SIP) security feature. Even though your extension is compiled locally, LLDB will be unable to attach to the Godot *host process* without manual re-signing.

In order to re-sign, simply create a file called `editor.entitlements` with the following contents. Be sure to use the `editor.entitlements` file below rather than the one from the Godot Docs, as it includes the required `com.apple.security.get-task-allow` key not currently present in Godot's instructions.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist
  PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/
PropertyList-1.0.dtd">
<plist version="1.0">
    <dict>
        <key>com.apple.security.cs.allow-dyld-environment-variables</key>
        <true/>
        <key>com.apple.security.cs.allow-jit</key>
        <true/>
        <key>com.apple.security.cs.allow-unsigned-executable-memory</key>
        <true/>
        <key>com.apple.security.cs.disable-executable-page-protection</key>
        <true/>
        <key>com.apple.security.cs.disable-library-validation</key>
        <true/>
        <key>com.apple.security.device.audio-input</key>
        <true/>
        <key>com.apple.security.device.camera</key>
        <true/>
        <key>com.apple.security.get-task-allow</key>
        <true/>
    </dict>
</plist>
```

Once this file is created, you can run

```
codesign -s - --deep --force --options=runtime \
    --entitlements ./editor.entitlements /Applications/Godot.app
```

in Terminal to complete the re-signing process. It is recommended to check this file into version control, since each developer needs to re-sign their local installation if you have a team. This process should only be necessary once per Godot installation though.

Export to Android

Exporting with godot-rust for Godot requires some of the same pieces that are required for building Godot from source. Specifically, the Android SDK Command Line Tools and JDK 17 as mentioned in Godot's documentation here.

Once you have those installed, you then need to follow Godot's instructions for setting up the build system here.

To find the JDK and NDK versions that are needed, reference the Godot configuration that your version of Godot is using. For example:

- master branch
- 4.2.2-stable tag

Compiling

The environment variable `CLANG_PATH` is used by bindgen's clang-sys dependency. See also clang-sys documentation

Set the environment variable `CLANG_PATH` to point to Android's build of clang. Example:

```
export CLANG_PATH=\
"{androidCliDirectory}/{androidCliVersion}/ndk/{ndkVersion}/toolchains/llvm/
prebuilt/{hostMachineOs}/bin/clang"
```

Then set the `CARGO_TARGET_{shoutTargetTriple}_LINKER` to point to the Android linker for the Android triple you are targeting. The `{shoutTargetTriple}` should be in `SHOUT_CASE` so that a triple such as `aarch64-linux-android` becomes `AARCH64_LINUX_ANDROID`. You need to compile your Rust extension library for each Android triple individually. Possible targets can be found by running:

```
rustup target list
```

You can find the linkers in the Android CLI directory at:

```
{androidCliDirectory}/{androidCliVersion}/ndk/{ndkVersion}/toolchains/llvm/
prebuilt/
{hostMachineOs}/bin/{targetTriple}{androidVersion}
```

As of writing this, the tested triples are:

Triple	Environment Variable	Godot Arch	GD
`aarch64-linux-android`	`CARGO_TARGET_AARCH64_LINUX_ANDROID_LINKER`	`arm64`	andr
`x86_64-linux-android`	`CARGO_TARGET_X86_64_LINUX_ANDROID_LINKER`	`x86_64`	andr
`armv7-linux-androideabi`	`CARGO_TARGET_ARMV7_LINUX_ANDROID_LINKER`	`arm32`	andr -v7a
`i686-linux-android`	`CARGO_TARGET_I686_LINUX_ANDROID_LINKER`	`x86_32`	andr

Notice how the environment variables are in all-caps and the triple's "-" is replaced with "_".

Make sure to add all of the triples you want to support to `rustup` via:

```
rustup target add {targetTriple}
```

Example:

```
rustup target add aarch64-linux-android
```

A complete example

Putting it all together, here is an example compiling for `aarch64-linux-android`. This is also probably the most common Android target, as of the writing of this.

Assuming the following things:

1. Android CLI is installed in the `$HOME` folder.
2. Godot is still relying on Android NDK version 23.2.8568313. Check here.
3. The downloaded Android CLI version is: 11076708_latest (update this to be the version you downloaded).
4. This is being run on Linux. Change the `linux-x86_64` folder in `CLANG_PATH` and

`CARGO_TARGET_AARCH64_LINUX_ANDROID_LINKER` to be your host machine's operating system.

5. You are targeting Android version 34.

And here is what the commands look like running from a bash shell:

```
rustup target add aarch64-linux-android

export CLANG_PATH="$HOME/android-cli/11076708_latest/ndk/23.2.8568313/
toolchains/llvm/prebuilt/linux-x86_64/bin/clang"
export CARGO_TARGET_AARCH64_LINUX_ANDROID_LINKER=\
"$HOME/android-cli/11076708_latest/ndk/23.2.8568313/toolchains/llvm/prebuilt/
linux-x86_64/bin/aarch64-linux-android34-clang"

cargo build --target=aarch64-linux-android
```

And then you should find a built version of your GDExtension library in:

```
target/aarch64-linux-android/debug/{YourCrate}.so
```

Make sure to update your `.gdextension` file to point to the compiled lib. Example:

```
android.debug.arm64="res://path/to/rust/lib/target/aarch64-linux-android/debug/
{YourCrate}.so
```

Export to Web

Web builds are a fair bit more difficult to get started with compared to native builds. This will be a complete guide on how to get things compiled. However, setting up a web server to host and share your game is considered out of scope of this guide, and is best explained elsewhere.

> **⚠ Warning**
>
> Web support in godot-rust is experimental and should be understood as such before proceeding.

Installation

Install a nightly build of `rustc`, the `wasm32-unknown-emscripten` target for `rustc`, and `rust-src`. The reason why nightly `rustc` is required is the unstable flag to build `std` (`-Zbuild-std`). Assuming that Rust was installed with `rustup`, this is quite simple.

```
rustup toolchain install nightly
rustup component add rust-src --toolchain nightly
rustup target add wasm32-unknown-emscripten --toolchain nightly
```

Next, install Emscripten. The simplest way to achieve this is to install `emsdk` from the git repo.
We recommend version **3.1.74** when targeting Godot 4.3 or later.[1]

```
git clone https://github.com/emscripten-core/emsdk.git
cd emsdk
./emsdk install 3.1.74
./emsdk activate 3.1.74

source ./emsdk_env.sh  # on Linux
run ./emsdk_env.bat    # on Windows
```

It would also be **highly** recommended to follow the instructions in the terminal to add `emcc`[2] to your `PATH`. If not, it is necessary to manually `source` the `emsdk_env.sh` file in every new terminal prior to compilation. This is platform-specific.

Project Configuration

At the moment, you'll need to use `api-custom` to export to the web. In the future, we will provide a "prebuilt" target (like Linux, macOS and Windows) directly for Wasm32. For now, see Custom Godot versions.

Enable the `experimental-wasm` feature on godot-rust in the `Cargo.toml` file. It is also recommended to enable the `lazy-function-tables` feature to avoid long compile times with release builds (this might be a bug and not necessary in the future). Edit the line to something like the following:

```
[dependencies.godot]
git = "https://github.com/godot-rust/gdext"
branch = "master"
features = ["api-custom", "experimental-wasm", "lazy-function-tables"]
```

Next, begin configuring the `emcc` flags and export targets as below. These initial settings will assume that your extension needs multi-threading support, but that's usually not the case, so make sure to check the "Thread support" section below if you're exporting to Godot 4.3 or later.

If you do not already have a `.cargo/config.toml` file, do the following:

- Create a `.cargo` directory at the same level as your `Cargo.toml`.
- Inside that directory, create a `config.toml` file.

Start by adding the following contents to that file:

```
[target.wasm32-unknown-emscripten]
rustflags = [
    "-C", "link-args=-pthread", # /!\ Read 'Thread support' below regarding
this flag
    "-C", "target-feature=+atomics", # /!\ Read 'Thread support' below
regarding this flag
    "-C", "link-args=-sSIDE_MODULE=2",
    "-C", "llvm-args=-enable-emscripten-cxx-exceptions=0",
    "-Z", "default-visibility=hidden",
    "-Z", "link-native-libraries=no",
    "-Z", "emscripten-wasm-eh=false",
]
```

A bit of background: The compiled Wasm binary is a *side module* which is dynamically loaded by Godot's *main module* at runtime. This is very similar to how dynamic libraries and executables work.

The `default-visibility=hidden` flag ensures that symbols of each side module are private to that module. Without it, it's possible that global variables of multiple extensions are merged together, causing very confusing bugs. This flag is essential when you have multiple Rust extensions loaded in the same Godot game. If you need to selectively disable this flag (not recommended), you can remove it from the `rustflags` list in `.cargo/config.toml` or `RUSTFLAGS`. See also this GitHub discussion for more information.

Edit the project's `.gdextension` file to include support for web exports. This file will probably be at `godot/{YourCrate}.gdextension`. The format will be similar to the following:

```
[libraries]
...
web.debug.wasm32 = "res://../rust/target/wasm32-unknown-emscripten/debug/
{YourCrate}.wasm"
web.release.wasm32 = "res://../rust/target/wasm32-unknown-emscripten/release/
{YourCrate}.wasm"
```

Compile the Project

Verify `emcc` is in the `PATH`. This can be as simple as doing the following:

```
emcc --version
```

Now, try to compile your code. It is necessary to both use the nightly compiler and specify to build std[3], along with specifying the Emscripten target.

```
cargo +nightly build -Zbuild-std --target wasm32-unknown-emscripten
```

Note that you may have to use a different build command in order to let the extension work in single-threaded web export in Godot 4.3+ (see the "Thread support" section below for more information).

Thread support (Godot 4.3 or later)

> **Note**
>
> The following section assumes your extension targets **Godot 4.3 or later**. If your extension will only target Godot 4.2 or 4.1, you may keep the initial configuration from Project Configuration without any changes.
>
> In addition, this section's instructions require **godot-rust 0.3 or later**.

The above settings assume that multi-threading support is always needed for your extension. However, starting with Godot 4.3, when the end user exports a game to the web, Godot includes an option to disable *Thread Support* in the web export menu (see the image in the "Godot editor setup" section), with the goal of having the exported game run in more environments, including older browsers, as well as webservers without Cross-Origin Isolation support.

With the proposed initial configuration from "Project configuration", if the end user disabled *Thread Support*, your extension would break. If you'd like your extension to support builds without multi-threading as well to avoid this problem, you will need to update your build setup in one of the two following ways.

> **Note**
>
> Ensure you're using recommended versions of Emscripten and nightly Rust (at least Rust 1.85 is recommended).
>
> This is because earlier versions of Emscripten expected `link-args=-sUSE_PTHREADS=1` instead of `link-args=-pthread`, but this flag has been deprecated.
>
> In addition, earlier Rust versions required additional `+bulk-memory,+mutable-globals` target features, but they appear to not be needed anymore.

Building without multi-threading support

In this scenario, you'd like to build your extension without any multi-threading support, that is, to have your extension only work when *Thread Support* is disabled.

To do that, you must remove the lines with the `-pthread` and `target-feature=+atomics` flags from `.cargo/config.toml`, as well as enable the `experimental-wasm-nothreads` feature in `Cargo.toml`.

The remaining configuration and build command do not require further changes.

This setup, by itself, isn't very common. We recommend following the instructions below to accept both multi-threaded and single-threaded exports for your extension.

Building both with and without multi-threading support

This is the recommended approach and allows your extension to work in both multi-threaded and single-threaded exports.

For that to happen, your extension will need to have two separate builds, one for each mode (with and without multi-threading).

Afterwards, Godot will automatically pick the correct build depending on whether the user chooses to enable or disable *Thread Support* when exporting to the web.

Here's how this can be done:

1. Remove `"-C", "link-args=-pthread"` and `"-C", "target-feature=+atomics"` from `.cargo/config.toml` so that you may conditionally enable them afterwards, resulting in the following updated `.cargo/config.toml` file:

   ```
   [target.wasm32-unknown-emscripten]
   rustflags = [
       "-C", "link-args=-sSIDE_MODULE=2",
       "-C", "llvm-args=-enable-emscripten-cxx-exceptions=0",
       "-Z", "default-visibility=hidden",
       "-Z", "link-native-libraries=no",
       "-Z", "emscripten-wasm-eh=false",
   ]
   ```

2. Create a feature for your main crate which enables `experimental-wasm-nothreads` when used. You can do this by creating a `[features]` section in your crate's `Cargo.toml` as follows:

   ```
   [features]
   nothreads = ["godot/experimental-wasm-nothreads"]
   ```

   This feature should be enabled on any crates depending on the `godot` crate, so if you have more than one crate in your workspace, you should add the same `[features]` section above to each other crate, and then enable each crate's `nothreads` feature

from the main crate (which provides the extension's entrypoint).

For example, if you have a workspace with one main crate called `extension` and two other crates called `lib1` and `lib2`, each depending on `godot`, then you may add the `[features]` section above to `crates/lib1/Cargo.toml` and `crates/lib2/Cargo.toml`, and then add the following to `crates/extension/Cargo.toml`:

```
[features]
# Ensure that enabling `nothreads` for the main crate also enables
# that feature for other crates.
nothreads = [
    "lib1/nothreads",
    "lib2/nothreads",
    "godot/experimental-wasm-nothreads"
]
```

3. Edit your `.gdextension` file to list two separate Wasm binary paths - one for the threaded build with the `.threads.wasm` suffix and one for the `nothreads` build without a different suffix, as follows:

```
[libraries]
...
web.debug.threads.wasm32 = "res://../rust/target/wasm32-unknown-emscripten/
debug/{YourCrate}.threads.wasm"
web.release.threads.wasm32 = "res://../rust/target/wasm32-unknown-
emscripten/release/{YourCrate}.threads.wasm"
web.debug.wasm32 = "res://../rust/target/wasm32-unknown-emscripten/debug/
{YourCrate}.wasm"
web.release.wasm32 = "res://../rust/target/wasm32-unknown-emscripten/
release/{YourCrate}.wasm"
```

4. Have two separate build commands, executed in the following order:

    1. **Building with multi-threading support:** you must add the `-pthread` and `target-feature=+atomics` flags back manually through the `RUSTFLAGS` environment variable, but NOT enable the `nothreads` feature yet.

> **Warning**
>
> Specifying `RUSTFLAGS` will cause flags in `.cargo/config.toml` to be ignored, so all flags in it must be specified again.

Afterwards, you should rename the generated Wasm binary, such that it can be picked up by the modified `.gdextension` file as a threaded build:

```
RUSTFLAGS="-C link-args=-pthread \
-C target-feature=+atomics \
-C link-args=-sSIDE_MODULE=2 \
-C llvm-args=-enable-emscripten-cxx-exceptions=0 \
-Z default-visibility=hidden \
-Z link-native-libraries=no \
-Z emscripten-wasm-eh=false" cargo +nightly build -Zbuild-std --target
wasm32-unknown-emscripten

mv target/wasm32-unknown-emscripten/debug/{YourCrate}.wasm target/
wasm32-unknown-emscripten/debug/{YourCrate}.threads.wasm
# On Batch (Windows), use instead: REN target\wasm32-unknown-
emscripten\debug\{YourCrate}.wasm {YourCrate}.threads.wasm
```

For a release mode build, you'd replace `debug` with `release` in the last command.

2. **Building without multi-threading support:** build without the `-pthread` or `target-feature=+atomics` flags, but this time enabling your `nothreads` feature created in the second step.

   No further renaming is needed, but make sure the previous build's resulting binary was renamed to avoid accidentally overwriting it.

   The build command for this step will then look as follows:

```
cargo +nightly build --features nothreads -Zbuild-std --target wasm32-
unknown-emscripten
```

5. Optionally, if you'd like to disable certain functionality in your extension for `nothreads`

builds (e.g. disable a certain multi-threaded function call), you can use `#[cfg(feature = "nothreads")]` and its variants to conditionally compile certain code under single-threaded builds, thanks to the `nothreads` feature created in step 2. For example:

```
fn maybe_threaded_function() {
    #[cfg(feature = "nothreads")]
    {
        /* single-threaded code */
    }

    #[cfg(not(feature = "nothreads"))]
    {
        std::thread::spawn(|| { /* multi-threaded code */
}).join().unwrap();
    }
}
```

Warning

If your extension is meant to be distributed to other users beside you, the developer, don't forget to ship BOTH binaries (with and without multi-threading support) to your end users.

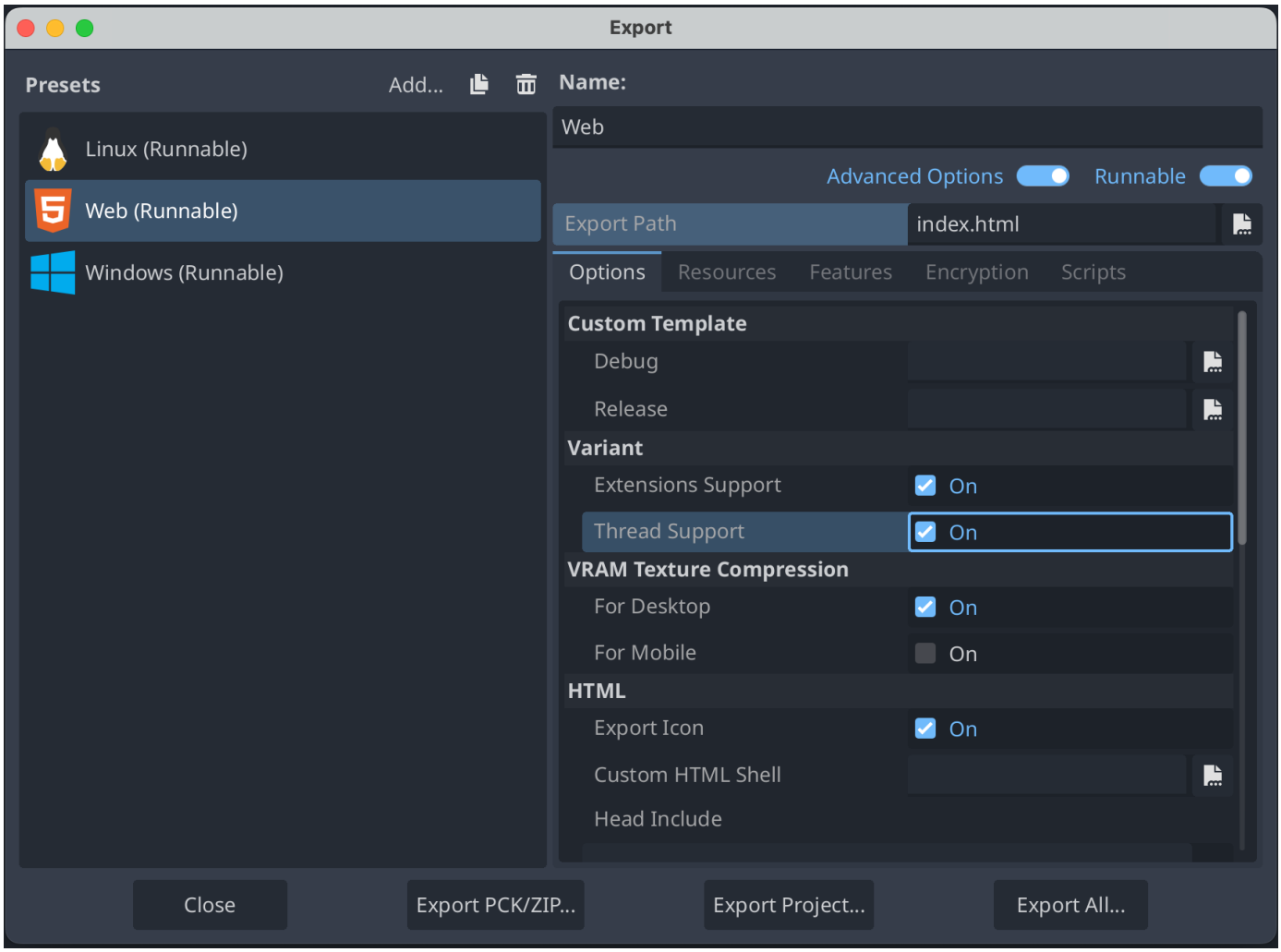
With those steps, you may successfully compile your extension with and without multi-threading support, and let you and your end users choose either option when exporting games to the web.

To not have to remember the multiple build commands, it is advised to add them to a single shell script file called `build.sh` which invokes both builds in order (including the binary file renaming before the second build and any other steps), or store them in a Justfile (useful if you need to build from Windows), Makefile or similar.

Godot editor setup

To export your godot-rust game to the web, add a web export in the Godot Editor. It can be configured at `Project > Export...` in the top menu bar. Make sure to turn on the *Extensions Support* checkbox.

In Godot 4.3 or above, you should also make sure to turn on the *Thread Support* checkbox, unless your extension has a `nothreads` build, which can be made by following the steps in the "Thread Support" section.



If the error below appears in red at the bottom of the export popup instead:

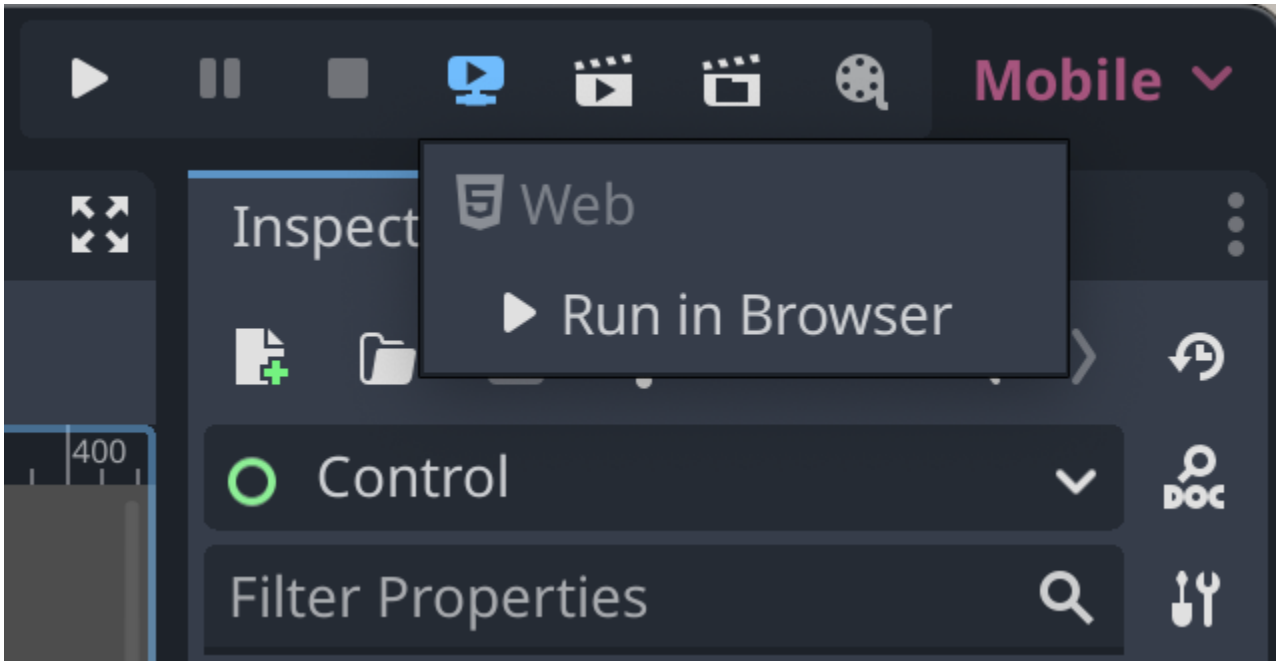
> No export template found at expected path:

Then click on *Manage Export Templates* next to the error message, and then on the next screen select *Download and Install*. See Godot tutorial for further information.

Running the webserver

Back at the main editor screen, there is an option to run the web debug build (*not* a release build) locally without needing to run an export or manually set up a web server.

At the top right, choose `Remote Debug > Run in Browser`. Afterwards, Godot will automatically open up a web browser running your game.



> **Known Caveats**
>
> - Godot 4.1.3+ or 4.2+ is necessary.
> - GDExtension support for Firefox requires Godot 4.3+, and can be more limited compared to Chromium-based browsers (such as Google Chrome, Microsoft Edge or Brave).

If you face problems when testing with Firefox, you may need to copy the URL of the server created by the editor, which is usually `http://localhost:8060/tmp_js_export.html`, and open it in a Chromium-based browser such as Google Chrome, Microsoft Edge or Brave to verify whether it's a problem with your game or with Firefox.

Troubleshooting

1. Make sure *Extensions Support* is turned on when exporting.
2. Make sure you are using the recommended compiler flags.
3. When using Godot 4.3+, *Thread Support* has to be turned on during export unless your extension supports a `nothreads` build, as described in the "Thread Support" section.

4. If the game was exported with *Thread Support* enabled (or targeting Godot 4.1 or 4.2), make sure the webserver you use to host your game supports Cross-Origin Isolation. Web games hosted on itch.io, for example, should already support this out of the box.

   To test it locally, you can either use the Godot editor's built-in web game runner (shown in "Running the webserver"), or a third-party HTTP server program. For example, if you have `npm` and `npx` installed, you may use `npx serve --cors` to quickly host your web game locally with Cross-Origin Isolation support, enabling multi-threading.

   - Note that Godot 4.3 games exported to the web without *Thread Support* are not subject to this restriction, making them compatible with more environments, which is the main advantage of disabling that option. You may even have success in running those games by simply double-clicking the generated HTML file. The main caveat is that they may only run single-threaded.

5. Make sure your Rust library and Godot project are named differently (for example, `cool-game-extension` and `cool-game`), as otherwise your extension's `.wasm` file may be overwritten, leading to confusing runtime errors.

6. Try using exactly the recommended `emcc` version in the guide.

Understanding common errors

1. `RuntimeError: Aborted(undefined). Build with -sASSERTIONS for more info.`

   The game aborted unexpectedly. This likely means some Rust code called `panic!()` or unsuccessful `assert!(condition)`.

   Unfortunately, godot-rust cannot catch panics in Wasm yet due to limitations in the Rust compiler, so your game will abort.

   Please fix any panics indicated in the browser console, perhaps using debugging tools. The suggested `-sASSERTIONS` flag will likely not help at all.

   Some common panic causes include:

   - Attempting to call certain multi-threaded code in a `nothreads` build, such as `std::thread::spawn(...)`;
   - Using panicking variants of methods, such as `Array::at` instead of `Array::get`;
   - Calling `.unwrap()` on `Option::None` or `Result::Err`.

2. `TypeError: resolved is not a function`

   This likely indicates you specified the `-sASSERTIONS` emscripten flag, which is not entirely supported at the moment. Try removing it.

3. `Wasm module '{YourCrate}.wasm' not found. Check the console for more information.`

   This indicates the extension's Wasm binary filename is using a name that godot-rust doesn't expect.

   By default, on game startup (only on the Wasm target), godot-rust will look for binaries named `{YourCrate}.wasm` or `{YourCrate}.threads.wasm`. If your GDExtension is using a different Wasm filename, please either rename it to one of those names, or tell godot-rust the name of the Wasm binary you are using as below. Don't forget to update the binary name in your `.gdextension` file to match.

   ```
   // lib.rs of your main crate:
   struct MyExtension;

   #[gdextension]
   unsafe impl ExtensionLibrary for MyExtension {
       fn override_wasm_binary() -> Option<&'static str> {
           // Explicitly use a custom name for our Wasm binary.
           Some("some-different-name.wasm")
       }
   }
   ```

   In addition, please note that **godot-rust 0.3 or later** is required to fix this error.

4. `LinkError: WebAssembly.instantiate(): Import #6 "env" "__cpp_exception": tag import requires a WebAssembly.Tag`

   This is caused by Rust's WASM exception handling system, used for panics, which was enabled by default for Emscripten targets in `rustc` v1.93.0 (see the relevant PR). It unfortunately seems to be incompatible with Godot.

   To fix, disable the feature by adding `-Z emscripten-wasm-eh=false` to `RUSTFLAGS` or to `.cargo/config.toml` as suggested in the previous steps (copying the suggested list of flags should be enough). This requires using a nightly compiler.

   Further discussion can be found at godot-rust issue #1119.

Customizing `emcc` flags

If you keep running into unknown errors and none of the solutions above worked, first and foremost consider letting us know by opening an issue, especially if you're using a newer Godot version, as it's possible some new information is missing from this documentation.

Make sure to also check or comment on the WebAssembly thread on GitHub, as new information is continually added to that thread over time.

Besides that, it's possible that you may have to enable additional `emcc` flags during compilation for your extension to work properly, which are specified at build time as `-C link-args=-FLAG_HERE` either in the `RUSTFLAGS` environment variable or in the `.cargo/config.toml` file (note that using `RUSTFLAGS` causes all flags in `.cargo/config.toml` to be ignored).

If that's the case, you may check out the Emscripten documentation for a list of some of the accepted flags. For example, `-C link-args=-g` enables linking back to Rust code while debugging.

This additional list also contains useful `emcc` flags which may be specified only with the `-s` prefix.

If you found a set of flags that worked for your case, please share it in the WebAssembly GitHub thread to help others in a similar situation.

Debugging

Currently, the only option for Wasm debugging is the "C/C++ DevTools Support" extension for Chrome. It adds support for breakpoints and a memory viewer into the F12 menu.

If Rust source code doesn't appear in the browser's debug panel, you should compile your extension in debug mode and add `-g` to linker flags. For example, in a multi-threaded build:

```
RUSTFLAGS="-C link-args=-g \
-C link-args=-pthread \
-C target-feature=+atomics \
-C link-args=-sSIDE_MODULE=2 \
-C llvm-args=-enable-emscripten-cxx-exceptions=0 \
-Z default-visibility=hidden \
-Z link-native-libraries=no \
-Z emscripten-wasm-eh=false" cargo +nightly build -Zbuild-std --target wasm32-
unknown-emscripten
```

Footnotes

[1] Note: Due to a bug with `emscripten`, web export templates for Godot 4.2 and earlier versions could only be compiled with `emcc` versions up to `3.1.39`. If you're targeting those older Godot versions, it could be safer to use `emcc` version `3.1.39` to compile your extension as well, but newer `emcc` versions might still work regardless (just make sure to test your extension in all targeted Godot versions).

[2] `emcc` is the name of Emscripten's compiler.

[3] The primary reason for this is it is necessary to compile with `-pthread` and `-Ctarget-feature=+atomics` enabled for multi-threaded builds. The shipped `std` does not, and may also build with other flags we don't use, so building `std` is a requirement. Related info on about WASM support can be found here.

Export to macOS and iOS

Mac libraries that are intended to be shared with other people require *Code Signing* and *Notarization*. This page will introduce you to the process of building a macOS universal library and an iOS library, which you can distribute to other people.

- Building a redistributable library
  - Building a macOS universal lib
  - Building an iOS library
- Code Signing and Notarizing (macOS only)
  - `APPLE_DEV_ID` - Apple ID
  - `APPLE_DEV_TEAM_ID` - Apple Team ID
  - `APPLE_DEV_PASSWORD` - Apple App-Specific Password
  - `APPLE_CERT_BASE64`, `APPLE_CERT_PASSWORD` and `APPLE_DEV_APP_ID`
- Godot export

Building a redistributable library

For this tutorial, you will need:

- a Mac Computer
- an Apple ID enrolled in Apple Developer Program (99 USD per year).

Without Code Signing and Notarization, the other person can still use the built library, but either needs to:

- rebuild the whole thing locally
- re-sign it
- accept that it may contain malicious code.

Prerequisites:

- Download and install Xcode on your Mac computer.

Building a macOS universal lib

Add both x64 and arm64 targets. This is needed in order to create a universal build.

```
rustup target add x86_64-apple-darwin
rustup target add aarch64-apple-darwin
```

Build the library for both target architectures:

```
cargo build --target=x86_64-apple-darwin --release
cargo build --target=aarch64-apple-darwin --release
```

Run the lipo tool to merge the two in one universal library.

```
lipo -create -output target/release/lib{YourCrate}.macos.dylib \
    target/aarch64-apple-darwin/release/lib{YourCrate}.dylib \
    target/x86_64-apple-darwin/release/lib{YourCrate}.dylib
```

The result of this will be the file `target/release/lib{YourCrate}.macos.dylib` that will now have support for both x64 and arm64 platforms.

The user would need to replace `{YourCrate}` with the crate name. The name of your library will be the one you provided in `Cargo.toml` file, prefixed with `lib` and followed by `.dylib`:

```
[package]
name = "{YourCrate}"
```

Next, you will need to create the `.framework` folder.

```
mkdir target/release/lib{YourCrate}.macos.framework
cp target/release/lib{YourCrate}.macos.dylib \
    target/release/lib{YourCrate}.macos.framework/lib{YourCrate}.macos.dylib
```

Next, create the `Info.plist` file inside the `Resources` folder:

```
mkdir target/release/lib{YourCrate}.macos.framework/Resources
```

File contents:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/
DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>CFBundleExecutable</key>
    <string>lib{YourCrate}.macos.dylib</string>
    <key>CFBundleIdentifier</key>
    <string>org.mywebsite.myapp</string>
    <key>CFBundleInfoDictionaryVersion</key>
    <string>6.0</string>
    <key>CFBundleName</key>
    <string>My App Name</string>
    <key>CFBundlePackageType</key>
    <string>FMWK</string>
    <key>CFBundleShortVersionString</key>
    <string>1.0.0</string>
    <key>CFBundleSupportedPlatforms</key>
    <array>
        <string>MacOSX</string>
    </array>
    <key>NSHumanReadableCopyright</key>
    <string>Copyright (c)...</string>
    <key>CFBundleVersion</key>
    <string>1.0.0</string>
    <key>LSMinimumSystemVersion</key>
    <string>10.12</string>
</dict>
</plist>
```

> **XML format**
>
> The `CFBundleExecutable` name **must** match the dylib file name. Some of the contents in the XML file **must** not contain some characters. Generally avoid using anything other than letters and numbers. Related StackOverflow issue.

Edit the project's `.gdextension` file to include support for macOS. This file will probably be at `godot/{YourCrate}.gdextension`. The format will be similar to the following:

```
[libraries]
...
macos.release = "res://../rust/target/release/lib{YourCrate}.macos.framework"
```

Building an iOS library

Add as target arm64 iOS.

```
rustup target add aarch64-apple-ios
```

Build the library:

```
cargo build --target=aarch64-apple-ios --release
```

The result of this will be the file `target/aarch64-apple-ios/release/lib{YourCrate}.dylib`.

Next, you will need to create the `.framework` folder.

```
mkdir target/release/lib{YourCrate}.ios.framework
cp target/release/lib{YourCrate}.ios.dylib \
    target/release/lib{YourCrate}.ios.framework/lib{YourCrate}.ios.dylib
```

Next, create the `Info.plist` file inside the `.framework` folder, with the following contents:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple Computer//DTD PLIST 1.0//EN" "http://
www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>CFBundleInfoDictionaryVersion</key>
    <string>6.0</string>
    <key>CFBundleDevelopmentRegion</key>
    <string>en</string>
    <key>CFBundleExecutable</key>
    <string>lib{YourCrate}.ios.dylib</string>
    <key>CFBundleName</key>
    <string>My App Name</string>
    <key>CFBundleDisplayName</key>
    <string>My App Name</string>
    <key>CFBundleIdentifier</key>
    <string>org.my-website.my-app</string>
    <key>NSHumanReadableCopyright</key>
    <string>Copyright (c) ...</string>
    <key>CFBundleVersion</key>
    <string>0.12.0</string>
    <key>CFBundleShortVersionString</key>
    <string>0.12.0</string>
    <key>CFBundlePackageType</key>
    <string>FMWK</string>
    <key>CSResourcesFileMapped</key>
    <true/>
    <key>DTPlatformName</key>
    <string>iphoneos</string>
    <key>MinimumOSVersion</key>
    <string>12.0</string>
</dict>
</plist>
```

See XML format requirements above.

Edit the project's `.gdextension` file to include support for iOS. This file will probably be at `godot/{YourCrate}.gdextension`. The format will be similar to the following:

```
[libraries]
...
ios.release = "res://../rust/target/release/lib{YourCrate}.ios.framework"
```

Code Signing and Notarizing (macOS only)

> **Optional Step**
>
> This step is only needed if you want to share the library. If you are building the whole game, you will sign everything and don't need to sign the library. You can skip to Godot export step.

In order to code-sign and notarize your app, you will first need to gather some information from your enrolled Apple Developer account. We will create corresponding environment variables and use a script to sign, so it's easier to run. Here are the environment variables needed:

- `APPLE_CERT_BASE64`
- `APPLE_CERT_PASSWORD`
- `APPLE_DEV_ID`
- `APPLE_DEV_TEAM_ID`
- `APPLE_DEV_PASSWORD`
- `APPLE_DEV_APP_ID`

Firstly, make sure to enroll your Apple ID to the Developer Program:

- Create an Apple ID if you don't have one already.
- Use your Apple ID to register in the Apple Developer Program by going to developer.apple.com.
- Accept all agreements from the Apple Developer Page.

`APPLE_DEV_ID` - Apple ID

Your email used for your Apple ID.

```
APPLE_DEV_ID = email@provider.com
```

`APPLE_DEV_TEAM_ID` - Apple Team ID

Go to developer.apple.com. Go to account.

Go to membership details. Copy Team ID.

```
APPLE_DEV_TEAM_ID = 1ABCD23EFG
```

`APPLE_DEV_PASSWORD` - Apple App-Specific Password

Create Apple App-Specific Password. Copy the password.

```
APPLE_DEV_PASSWORD = abcd-abcd-abcd-abcd
```

`APPLE_CERT_BASE64`, `APPLE_CERT_PASSWORD` and `APPLE_DEV_APP_ID`

Go to developer.apple.com. Go to account.

Go to certificates.

Click on + at Certificates tab. Create Developer ID Application. Click Continue.

Leave profile type as is. Create a certificate signing request from a Mac. You can use your own name and email address. Save the file to disk. You will get a file called `CertificateSigningRequest.certSigningRequest`. Upload it to the Developer ID Application request. Click Continue.

Download the certificate. You will get a file `developerID_application.cer`.

On a Mac, right click and select open. Add it to the login keychain. In the Keychain Access app that opened, log into Keychain tab, go to Keys, sort by date modified, and expand your key (the key should have the name you entered at *Common Name*). Right click the expanded certificate, get info, and copy the text at *Details -> Subject Name -> Common Name*. For example:

```
APPLE_DEV_APP_ID = Developer ID Application: Common Name (1ABCD23EFG)
```

Then, select the certificate, right click and click export. At file format select `p12`. When exporting, set a password for the certificate. This will be the value of `APPLE_CERT_PASSWORD`. You will get a `Certificates.p12` file.

For example:

```
APPLE_CERT_PASSWORD = <password_set_when_exporting_p12>
```

Then you need to make a base64 file out of it, by running:

```
base64 -i Certificates.p12 -o Certificates.base64
```

Copy the contents of the generated file, e.g.:

```
APPLE_CERT_BASE64 = ...(A long text file)
```

After these secrets are obtained, all that remains is to set them as environment variables. Afterwards you can use the following script for signing ci-sign-macos.ps1. In order to run this script you will need to install powershell on your Mac.

```
ci-sign-macos.ps1 target/release/{YourCrate}.framework
```

> **External script disclaimer**
>
> The user is responsible for the security and up-to-dateness of the script.

Godot export

After building the libraries, you can now distribute them as they are, or build the whole game using Godot. For that, follow Godot's *How to export* guide:

- Exporting for macOS
- Exporting for iOS

Recipes

Custom resources

With godot-rust, you are able to define custom `Resource` classes which are then available to the end user.

Editor plugins

`EditorPlugin` types are loaded during editor and runtime and are able to access the editor as well as the scene tree. This type follows the same functionality that a typical `EditorPlugin` class written in GDScript would, but crucially with access to the *entire Rust ecosystem*.

Engine singletons

An Engine Singleton is a class instance that is always globally available (following the Singleton pattern). However, it cannot access the `SceneTree` through any reliable means.

`ResourceFormatSaver` and `ResourceFormatLoader`

Provide custom logic for saving and loading your `Resource` derived classes.

Custom icons

Adding custom icons to your classes is actually fairly simple!

Custom resources

Custom `Resource` s are exposed to the end user to use within their development. `Resource` s can store data that is easily edited from within the editor GUI. For example, you can create a custom `AudioStream` type that handles a new and interesting audio file type.

Registering a `Resource`

This workflow is similar to the Hello World example:

```
#[derive(GodotClass)]
#[class(init, base=Resource)]
struct ResourceType {
    base: Base<Resource>,
}
```

The above resource does not export any variables. While not all resources require exported variables, most do.

To execute code within the Godot Editor (e.g., overriding `set_property` to run custom logic when editing Inspector properties), you should annotate the class with `#[class(tool)]` .

```rust
#[derive(GodotClass)]
#[class(tool, init, base=Resource)]
struct ResourceType {
    base: Base<Resource>,
}

#[godot_api]
impl IResource for ResourceType {
    // String representation of the object.
    fn to_string(&self) -> GString { ... }

    // Called when the object receives a Godot notification.
    fn on_notification(&mut self, what: ObjectNotification) { ... }

    // Called whenever get() is called or Godot gets the value of a property.
    fn get_property(&self, property: StringName) -> Option<Variant> { ... }

    // Called whenever Godot set() is called or Godot sets the value of a
property.
    fn set_property(&mut self, property: StringName, value: Variant) -> bool {
... }

    // Called whenever Godot get_property_list() is called,
    // the returned vector here is appended to the existing list of properties.
    fn get_property_list(&mut self) -> Vec<PropertyInfo> { ... }

    // Called whenever Godot retrieves value of property. Allows to customize
existing properties.
    // Every property info goes through this method, except properties added
with get_property_list().
    fn validate_property(&self, property: &mut PropertyInfo) { ... }

    // Called by Godot to tell if a property has a custom revert or not.
    fn property_get_revert(&self, property: StringName) -> Option<Variant> {
... }

    fn setup_local_to_scene(&mut self) { ... }
}
```

It is important that similar to defining custom resources in GDScript, marking this class as a "tool class" is required to be usable within the editor.

The systems for registering functions, properties, and more are described in detail in the Registering Rust symbols section.

Editor plugins

Using `EditorPlugin` types is very similar to the process used when writing plugins in GDScript. Unlike GDScript plugins, godot-rust plugins are registered automatically and cannot be enabled/disabled in the Project Settings plugins pane.

Plugins written in GDScript are automatically disabled if they have a code error, but because Rust is a compiled language, you cannot introduce compile-time errors.

Creating an `EditorPlugin`

```
#[derive(GodotClass)]
#[class(tool, init, base=EditorPlugin)]
struct MyEditorPlugin {
    base: Base<EditorPlugin>,
}

#[godot_api]
impl IEditorPlugin for MyEditorPlugin {
    fn enter_tree(&mut self) {
        // Perform typical plugin operations here.
    }

    fn exit_tree(&mut self) {
        // Perform typical plugin operations here.
    }
}
```

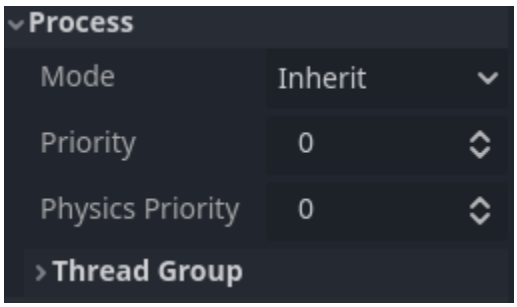
Since this is an `EditorPlugin`, it will be automatically added to the scene tree root. This means it can access the scene tree at runtime. Additionally, it is safe to access the `EditorInterface` singleton through this node, which allows adding different GUI elements to the editor directly. This can be helpful if you have an advanced GUI you want to implement.

Inspector plugins

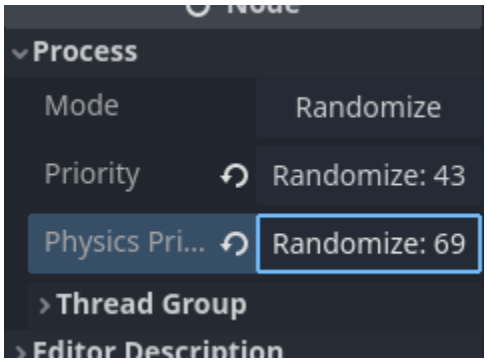
The inspector dock allows you to create custom widgets to edit properties through plugins. This can be beneficial when working with custom datatypes and resources, although you can use the feature to change the inspector widgets for built-in types. You can design custom controls for specific properties, entire objects, and even separate controls associated with particular datatypes. For more info, see docs.godotengine.org.

The example in the Godot docs in Rust. It will replace integer input with a button that creates a random value.

Before (int input):



After (button):



Add this dependency to Rust with the shell in the same directory as `Cargo.toml`.

```
cargo add rand
```

Add file `addon.rs` and import it in `lib.rs`:

```
// file: lib.rs
mod addon;
```

Add the following imports at the beginning of the file:

```
use godot::classes::{
    Button, EditorInspectorPlugin, EditorPlugin, EditorProperty,
IEditorInspectorPlugin,
    IEditorPlugin, IEditorProperty,
};
use godot::global;
use godot::prelude::*;
use rand::Rng;
```

Since Rust is a statically typed language, we will proceed in reverse order unlike in Godot documentation, to avoid encountering errors unnecessarily.

Add Property Editor

To begin with, let's define the editor for properties:

```
#[derive(GodotClass)]
#[class(tool, init, base=EditorProperty)]
struct RandomIntEditor {
    base: Base<EditorProperty>,
    button: Option<Gd<Button>>,
}
```

After that, we need to add an implementation for the trait `IEditorProperty`:

```
#[godot_api]
impl IEditorProperty for RandomIntEditor {
    fn enter_tree(&mut self) {
        // Create button element.
        let mut button = Button::new_alloc();

        // Add handler for this button, handle_press will be define in another
impl.
        button.connect("pressed", self.base().callable("handle_press"));
        button.set_text("Randomize");

        // Save pointer to the button into struct.
        self.button = Some(button.clone());
        self.base_mut().add_child(button.upcast());
    }

    fn exit_tree(&mut self) {
        // Remove element from inspector when this plugin unmount:
        if let Some(button) = self.button.take() {
            self.base_mut().remove_child(button.upcast());
        } else {
            // Log error if button disappeared before
            godot_error!("Button wasn't found in exit_tree");
        }
    }
}
```

Let's add a handler for the button:

```
#[godot_api]
impl RandomIntEditor {
    #[func]
    fn handle_press(&mut self) {
        // Update value by button click:
        // - Take property name, randomize number.
        // - Send property name and random number to Godot engine to update
value.
        // - Update button text.
        let property_name = self.base().get_edited_property();
        let num = rand::thread_rng().gen_range(0..100);

        godot_print!("Randomize! {num} for {property_name}");

        self.base_mut()
            .emit_changed(property_name, num.to_variant());

        if let Some(mut button) = self.button.clone() {
            let text = format!("Randomize: {num}");
            button.set_text(&text);
        } else {
            // Print error of something went wrong
            godot_error!("Button wasn't found in handle_press");
        }
    }
}
```

Add Inspector plugin

Now we need to connect this editor to fields with an integer type. To do this, we need to create an `EditorInspectorPlugin`.

```
#[derive(GodotClass)]
#[class(tool, init, base=EditorInspectorPlugin)]
struct RandomInspectorPlugin {
    base: Base<EditorInspectorPlugin>,
}
```

To add a property editor (which we implemented earlier), you need to implement the `IEditorInspectorPlugin` trait:

```
#[godot_api]
impl IEditorInspectorPlugin for RandomInspectorPlugin {
      fn parse_property(
        &mut self,
        _object: Gd<Object>, // object that is being inspected
        value_type: VariantType,
        name: GString,
        _hint_type: global::PropertyHint,
        _hit_string: GString,
        _flags: global::PropertyUsageFlags,
        _wide: bool,
    ) -> bool {
        if value_type == VariantType::INT {
            self.base_mut()
                .add_property_editor(name,
RandomIntEditor::new_alloc().upcast());
            return true;
        }

        false
    }

    // This method says Godot that this plugin handle the object if it returns
true
    fn can_handle(&self, object: Gd<Object>) -> bool {
        // This plugin handle only Node2D and object that extends it
        object.is_class("Node2D")
    }
}
```

If `parse_property` returns `true`, the editor plugin will be created and replace the current representation; if not, it's necessary to return `false`. This allows you to control where and how processing is done by this plugin.

Adding an editor plugin

Only one thing left to do: define the editor plugin that will kick off all this magic! This can be a generic `EditorPlugin` or a more specific `InspectorEditorPlugin`, depending on what you want to achieve.

```
#[derive(GodotClass)]
#[class(tool, init, base=EditorPlugin)]
struct RustEditorPlugin {
    base: Base<EditorPlugin>,
    random_inspector: Gd<RandomInspectorPlugin>,
}
```

```
#[godot_api]
impl IEditorPlugin for RustEditorPlugin {
    fn enter_tree(&mut self) {
        // Create our inspector plugin and save it.
        let plugin = RandomInspectorPlugin::new_gd();
        self.random_inspector = plugin.clone();
        self.base_mut().add_inspector_plugin(plugin.upcast());
    }

    fn exit_tree(&mut self) {
        // Remove inspector plugin when editor plugin leaves scene tree.
        let plugin = self.random_inspector.clone();
        self.base_mut().remove_inspector_plugin(plugin.upcast());
    }
}
```

Troubleshooting

Sometimes after compilation, you may encounter errors or panic. Most likely, all you need to do is simply **restart** the Godot Editor.

Example error:

```
Initialize godot-rust (API v4.2.stable.official, runtime
v4.2.2.stable.official)
ERROR: Cannot get class 'RandomInspectorPlugin'.
   at: (core/object/class_db.cpp:392)
ERROR: Cannot get class 'RandomInspectorPlugin'.
   at: (core/object/class_db.cpp:392)
```

User singletons

It is important for you to understand the Singleton pattern to properly utilize this system.

> **Controversy**
>
> The "Singleton pattern" is often referred to as an anti-pattern, because it violates several good practices for clean, modular code. However, it is also a tool that can be used to solve certain design problems. As such, it is used internally by Godot, and is available to godot-rust users as well.
>
> Read more about criticisms here.

Custom singletons in Godot:

- are `Object` types
- always run in the editor (implied `#[class(tool)]`)
- are always accessible to GDScript and GDExtension languages
- must be registered and unregistered in the `InitStage::Scene` step

Godot provides *many* built-in singletons in its API. You can find a full list here.

Table of contents

- Registering custom singleton
- Using a singleton with `on_main_loop_frame`
- Registering custom singleton without proc macro
- Singletons and the `SceneTree`

Registering custom singleton

You can register given class as a Singleton with `#[class(singleton)]`.

```
#[derive(GodotClass)]
#[class(init, singleton)]
struct MySingleton {
    // For `#[class(singleton)]`, the default base is Object, not RefCounted.
    base: Base<Object>,
}

// Can be accessed like any other singleton from the main thread.
let val = MySingleton::singleton().bind().foo();
```

Now that your singleton is available (and once you've recompiled and reloaded), you should be able to access it from GDScript like so:

```
extends Node

func _ready() -> void:
    MySingleton.foo()
```

Using a singleton with `on_main_loop_frame`

Since Godot4.5+ `on_main_loop_frame` can be used to invoke an user singleton:

```
fn global_delta() -> f64 {
    let ticks = ProjectSettings::singleton()
        .get("physics/common/physics_ticks_per_second")
        .to::<i64>();
    1.0 / (ticks as f64)
}

#[derive(GodotClass)]
#[class(init, singleton)]
struct MySingleton {
    #[init(val = global_delta())]
    delta: f64,

    #[init(val = true)]
    paused: bool,

    #[init(val = Instant::now())]
    time: Instant,

    base: Base<Object>,
}

#[gdextension]
unsafe impl ExtensionLibrary for MyExtension {
    fn on_main_loop_frame() {
        if Engine::singleton().is_editor_hint() {
            return;
        }
        MySingleton::singleton().bind_mut().frame();
    }
}

impl MySingleton {
    pub fn frame(&mut self) {
        if self.paused {
            return;
        }

        let elapsed = self.time.elapsed().as_secs_f64();
        self.elapsed += elapsed;

        if self.elapsed >= self.delta {
            let time_scale = Engine::singleton().get_time_scale();
            self.run_simulation(self.elapsed * time_scale);
            self.elapsed = 0.0;
        }
        self.time = Instant::now();
    }
}
```

Registering custom singleton without proc macro

Custom singleton can be registered through `godot::classes::Engine`. Additionally, implementing `UserSingleton` allows accessing a registered singleton instance through `singleton()`.

User singletons should be registered under their class name – otherwise some Godot components (for example GDScript before 4.4) might have trouble handling them, and the editor might crash when using `T::singleton()`.

There should be only one instance of a given singleton class in the engine, valid as long as the library is loaded. Therefore, user singletons are limited to classes with manual memory management (ones not inheriting from RefCounted).

```
#[derive(GodotClass)]
#[class(init, base = Object)]
struct MySingleton {}

// Provides blanket implementation allowing to use MySingleton::singleton().
// Ensures that `MySingleton` is a valid singleton
// (i.e., a non-refcounted GodotClass).
impl UserSingleton for MySingleton {}

struct MyExtension;

#[gdextension]
unsafe impl ExtensionLibrary for MyExtension {
    fn on_stage_init(stage: InitStage) {
        // Singleton should be registered before the MainLoop startup;
        // otherwise it won't be recognized by the GDScriptParser.
        if stage == InitStage::Scene {
            let obj = MySingleton::new_alloc();
            Engine::singleton()
                .register_singleton(
                    &MySingleton::class_id().to_string_name(),
                    &obj
                );
        }
    }

    fn on_stage_deinit(stage: InitStage) {
        if stage == InitStage::Scene {
            let obj = MySingleton::singleton();
            Engine::singleton()
                .unregister_singleton(
                    &MySingleton::class_id().to_string_name()
                );
            obj.free();
        }
    }
}
```

Singletons and the `SceneTree`

Singletons cannot safely access the scene tree. At any given moment, they may exist without a scene tree being active. While it is technically possible to access the tree through hacky methods, it is **highly recommended** to use a custom `EditorPlugin` for this purpose. Creating an `EditorPlugin` allows for registering an "autoload singleton" which is a `Node` (or derived) type and is automatically loaded into the `SceneTree` by Godot when the game starts.

`Resource` savers and loaders

The `ResourceFormatSaver` and `ResourceFormatLoader` classes allow you to serialize and deserialize your Rust `Resource`-derived classes with a custom procedure, as well as define new recognized file extensions. This is mostly useful if you have resources that contain *pure Rust state*. "Pure" in this context refers to members of your struct that don't have any `#[var]` or similar annotations, i.e. Godot isn't aware of them. This can easily be the case when you work with Rust libraries.

The following example gives you a starting point to copy-and-paste. For advanced use cases, consult the Godot documentation for these classes.

Project Configuration

Enable the `experimental-threads` feature in your `Cargo.toml`. If this feature is not enabled, the application will panic if Godot uses your `ResourceFormatLoader` in a thread other than the main thread, such as in the editor.

Example saver and loader

First of all, you need to call the provided functions in your library entry point at the `InitLevel::Scene`. This ensures proper initialization and cleanup of your loader/saver.

```
// These imports will be needed across the following code samples.
use godot::classes::{
    Engine, IResourceFormatLoader, IResourceFormatSaver, ResourceFormatLoader,
    ResourceFormatSaver, ResourceLoader, ResourceSaver,
};
use godot::prelude::*;

#[gdextension]
unsafe impl ExtensionLibrary for MyGDExtension {
    // Register the singleton when the extension is loading.
    fn on_level_init(level: InitLevel) {
        if level == InitLevel::Scene {
            Engine::singleton().register_singleton(
                &MyAssetSingleton::class_name().to_string_name(),
                &MyAssetSingleton::new_alloc(),
            );
        }
    }

    // Unregister the singleton when the extension is unloaded.
    fn on_level_deinit(level: InitLevel) {
        if level == InitLevel::Scene {
            let mut engine = Engine::singleton();
            let singleton_name =
&MyAssetSingleton::class_name().to_string_name();
            let my_singleton = engine.get_singleton(singleton_name).unwrap();
            engine.unregister_singleton(singleton_name);
            my_singleton.free();
        }
    }
}
```

Define the singleton to keep track of your Loaders and Savers.

```
// The definition of the singleton with all your loader/savers as members,
// to keep the object references for destruction later.
#[derive(GodotClass)]
#[class(base=Object, tool)]
struct MyAssetSingleton {
    base: Base<Object>,
    loader: Gd<MyAssetLoader>,
    saver: Gd<MyAssetSaver>,
}

#[godot_api]
impl IObject for MyAssetSingleton {
    fn init(base: Base<Object>) -> Self {
        let saver = MyAssetSaver::new_gd();
        let loader = MyAssetLoader::new_gd();

        // Register the loader and saver in Godot.
        //
        // If you want your default extension to be the one defined by your
loader,
        // set the `at_front` parameter to true. Otherwise you can also remove
the
        // builder. Godot currently doesn't provide a way to completely
deactivate
        // the built-in loaders.
        //
        // WARNING: The built-in loaders won't work if you have _pure Rust
state_.
        ResourceSaver::singleton().add_resource_format_saver_ex(&saver)
            .at_front(false)
            .done();
        ResourceLoader::singleton().add_resource_format_loader(&loader);

        Self { base, loader, saver }
    }
}

// Unregister the loader and saver when the extension is unloaded.
impl Drop for MyAssetSingleton {
    fn drop(&mut self) {
        ResourceSaver::singleton().remove_resource_format_saver(&self.saver);

ResourceLoader::singleton().remove_resource_format_loader(&self.loader);
    }
}
```

> ⚠ **at_front behavior**
>
> The ordering of `at_front` may currently not work as expected in Godot. For more information, see PR godot#101543 or book discussion #65.

The minimal code for a **saver**, with all required virtual methods defined:

```
#[derive(GodotClass)]
#[class(base=ResourceFormatSaver, init, tool)]
struct MyAssetSaver {
    base: Base<ResourceFormatSaver>,
}

#[godot_api]
impl IResourceFormatSaver for MyAssetSaver {
    // If you want a custom extension name (e.g., resource.myextension),
    // then override this.
    fn get_recognized_extensions(
        &self,
        res: Option<Gd<Resource>>
    ) -> PackedStringArray {
        let mut array = PackedStringArray::new();

        // It is also possible to add multiple extensions per Saver.
        array.push("myextension");
        array
    }

    // All resource types that this saver should handle must return true.
    fn recognize(&self, res: Option<Gd<Resource>>) -> bool {
        // It is also possible to add multible resource types per Saver.
        res.is_some_and(|r| r.is_class("MyResourceType"))
    }

    // This defines your logic for actually saving your resource.
    fn save(
        &mut self,
        // The resource that is currently getting saved.
        resource: Option<Gd<Resource>>,
        // The path that the resource is getting saved at.
        path: GString,
        // Flags for saving (see link below).
        flags: u32,
    ) -> godot::global::Error {
        // TODO: Put your saving logic in here, with the `GFile` API (see link
below).

        godot::global::Error::OK
    }
}
```

Here are direct doc links to `SaverFlags` (Godot, Rust) and `GFile`.

The minimal code for a **loader**, with all required virtual methods defined:

```
#[derive(GodotClass)]
#[class(init, tool, base=ResourceFormatLoader)]
struct MyAssetLoader {
    base: Base<ResourceFormatLoader>,
}

#[godot_api]
impl IResourceFormatLoader for MyAssetLoader {
    // All file extensions you want to be redirected to your loader
    // should be added here.
    fn get_recognized_extensions(&self) -> PackedStringArray {
        let mut arr = PackedStringArray::new();
        arr.push("myextension");
        arr
    }

    // All resource types that this loader handles.
    fn handles_type(&self, ty: StringName) -> bool {
        ty == "MyResourceType".into()
    }

    // The stringified name of your resource should be returned.
    fn get_resource_type(&self, path: GString) -> GString {
        // The extension arg always comes with a `.` in Godot, so don't forget
it ;)
        if path.get_extension().to_lower() == ".myextension".into() {
            "MyResourceType".into()
        } else {
            // In case of not handling the given resource, this function must
            // return an empty string.
            GString::new()
        }
    }

    // The actual loading and parsing of your data.
    fn load(
        &self,

        // The path that should be openend to load the resource.
        path: GString,

        // If the resource was part of a import step you can access the
original file
        // with this. Otherwise this path is equal to the normal path.
        original_path: GString,

        // This parameter is true when the resource is loaded with
        // load_threaded_request().
        // Internal implementations in Godot also ignore this parameter.
        _use_sub_threads: bool,

        // If you want to provide custom caching this parameter is the
```

```
CacheMode enum.
        // You can look into the ResourceLoader docs to learn about the values.
        // When calling the default load() method, cache_mode is
CacheMode::REUSE.
        cache_mode: i32,
    ) -> Variant {
        // TODO: Put your saving logic in here, with the `GFile` API (see link
below).

        // If your loading operation failed and you want to handle errors,
        // you can return a godot::global::Error and cast it to a Variant.
    }
}
```

Direct link to `CacheMode` (Godot, Rust) and `GFile`.

Custom node icons

By default, all your custom types will use the `Node` icon in the editor UI -- e.g. in the scene tree or when selecting a node to create. While this can be serviceable, you may want to add custom icons to distinguish node types, especially if you plan to distribute your extension to others.

All icons must be registered by their class name in your `.gdextension` file. For this, you can add a new `icon` section. Classes are keys and paths to SVG files are values.

```
[icons]

MyClass = "res://addons/your_extension/filename.svg"
```

> **Icon paths**
>
> The path is based off the `res://` scheme, like other Godot resources. It is recommended to use Godot's convention of an `addons` folder, followed by the name of the addon.
>
> Read more about the reasoning behind this in the Godot docs:
>
> - Installing plugins
> - Making plugins

Formatting for custom icons

The Godot docs have a page dedicated to tools and resources for creating custom icons. The long and short of it is:

- Use the SVG format.
- Aspect ratio is a square, 16x16 units is the reference size.
- Refer to the Godot icon colors mappings.
    - Use the light mode colors -- Godot only supports light-to-dark, but not dark-to-light color conversions.

Third-party article

The user *QueenOfSquiggles* wrote an alternative version of this article on her personal blog, which includes color previews for the light and dark themed colors.

Details on how to use her reference page is included here.

Ecosystem

This chapter lists third-party projects that extend godot-rust with additional functionality: tools, libraries, integrations, apps, and more. The projects are grouped by type of project and their respective domain (although such classification is not always clear-cut).

If you'd like to add a project, please read Contributing!

Games made with godot-rust are available on a separate page.

Table of contents

- List of 3rd-party projects
  - Rust libraries
  - Editor plugins
  - Applications
- Contributing

List of 3rd-party projects

Rust libraries

Project	Further links	Activity
🌀 ***Async***		
gdext-coroutines Integrate Rust coroutines with Godot's async/await.	crates.io, Discord	last commit april
godot-tokio Create Tokio runtime for use with godot-rust.	crates.io, Discord	last commit june 2025
gdext-async Compatibility macros between Rust `await` and Godot `await`	crates.io	last commit last tuesday
———		
🏗 ***Project workflow***		

Project	Further links	Activity
godot_rust_tools Tools and utilities for godot-rust development.	Asset Library, Discord	last commit september
gd-rehearse Unit tests for godot-rust code.	Discord	last commit september 2025
gd-props Resource serialization using `serde`.	Discord	last commit august 2024
gdext-generation Auto-generate the `.gdextension` file.	Discord	last commit september 2025
godot-rust-cli CLI scripts for Godot with Rust.	Discord	last commit january 2025
gdext-cli A CLI tool to generate godot-rust projects and scripts.		last commit august 2025
godust Opinionated CLI tool for Godot template generation in gdext.	crates.io	last commit september
_____		
📜 ***Scripting***		
godot-rust-script Allows Rust scripts to be added to nodes.		last commit october
_____		
🎮 ***Game development***		
SpireTween Alternative tweening library for Godot 4.2+.	Discord	last commit august 2025
GridForge Generic abstraction for grid maps.	Discord	last commit april 2025

Editor plugins

Project	Further links	Activity
📐 ***User interface***		
Godot-Tour UI tours/tutorials for editor and in-game.	Discord	last commit may

Project	Further links	Activity
Godot-VSCode VSCode running inside the Godot game engine.	Discord	last commit june
🎨 ***Graphics***		
Godot Trail 3D Adds a `Trail3D` node to Godot.	Discord	last commit december 2025
🧲 ***Physics***		
Godot Rapier Physics Rapier 2D + 3D integration for Godot.	Discord	last commit yesterday
Godot Rapier 3D GDExtension that enables Rapier physics with Godot.	Discord	last commit march 2025
🧙 ***Storytelling***		
nobodywho Interact with local LLMs for interactive storytelling.	Discord	last commit yesterday
🪜 ***Project workflow***		
godot-sandbox Secure modding support for C++, Rust and others.		last commit october
Termdot Terminal built for Godot, enhance your workflow.	Discord	last commit june 2025
Stag Toolkit Procedural terrain and real-time simulations.	Island Builder	last commit september
🌐 ***Localization***		
Fluent Translation Translation using Mozilla's Fluent (FTL).	Asset Library	last commit may 2025

Applications

Project	Further links	Activity

Project	Further links	Activity
🎛️ ***Software platforms***		
Godot Boy Game boy emulator in Godot, written in Rust.	Discord	last commit september
GDScript Transpiler Reimplements parts of GDScript in Rust.	Discord	last commit december 2024
godot_rust_mapgen Procedural 3D map generator.	Discord	last commit november 2025
_____		
🛸 ***Tech demos***		
Godot boids Addon for Godot that adds 2D/3D boids (flocking).	Discord	???

Contributing

If you have a project that might fit this list, great! You don't have to be the author -- if you've come across something that will make other people's lives easier, please share it!

To keep this list useful for visitors, there are a few acceptance criteria:

- The project must be related to godot-rust (not only Rust or only Godot). It should use Godot 4.
- There's already something tangible with at least minimal docs/examples.
  - This could be a usable library on GitHub, a working demo, etc. No need for a crate release or very polished presentation; the idea is that the project is accessible for newcomers.
  - To discuss ideas and WIP prototypes, feel free to start a discussion in `#showcase` on Discord!
- The author should be willing to maintain the project for a while.
  - GDExtension has a very good track record with binary compatibility, and godot-rust supports extensions down to Godot 4.1. So if you integrate via extensions (e.g. as an editor plugin), your project tends to be more future-proof than with source code.
  - That said, we don't have major breaking changes very often.
- If the project is intended for distribution and usage, make sure it comes with a license (e.g. an open-source one for software, or Creative Commons for artworks).

Once that's sorted, please open a pull request directly to the book repository. If you're not sure about the criteria or have other questions, don't hesitate to ask on Discord or the book issue tracker.

> **A thriving ecosystem**
>
> Every single project enriches the space around Godot and Rust, and lets more and more people enjoy game development. Thanks a lot to every contributor!

Games

Godot is primarily a *game* engine, and Rust has a reputation of coming with ~~frustrating~~ interesting engineering challenges when used for game development. Yet, users keep putting godot-rust's pragmatic approach to the test and push the project to its limits, for which we are very thankful!

This page lists some godot-rust games built by the community. This is an early draft, the list will be curated over time. Depending on the number of entries, it may be necessary to remove/archive older or less active entries, or split it into multiple pages.

Table of contents

- In active development
  - Flame Up
  - One More Gem
  - Law of Entropy
  - Allyon: Battlecry
  - Voxel game (MountainLabz)
  - Voxel game (Ayrton)
  - Neon Grand Prix
  - Super Space Crystals
- Commercially released games
  - Solitaire Klondike Classic
  - Graphwar II
- Educational
  - Musical constellations
  - Gleba
  - Sonic Maker
- Web-playable
  - Iced Heist
  - Stickyban
  - Block comprehension
  - Aces' Nim

In active development

Games with regular updates on social media or devlogs. If they are abandoned, they may be removed from this section or archived.

Flame Up

Steam | Discord | *by Charming Moon*



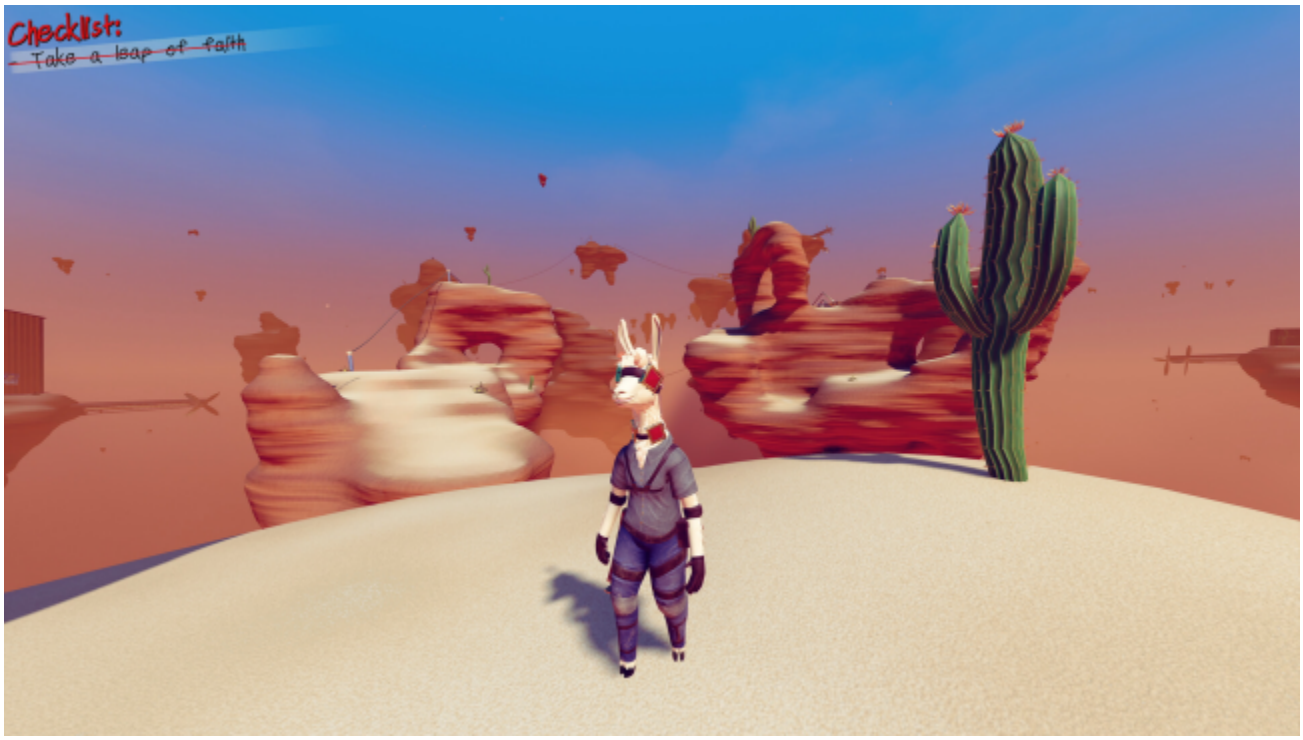
One More Gem

Steam | Discord | *by Cosmin Munteanu*



Law of Entropy

Steam | Discord | *by Stag Math*



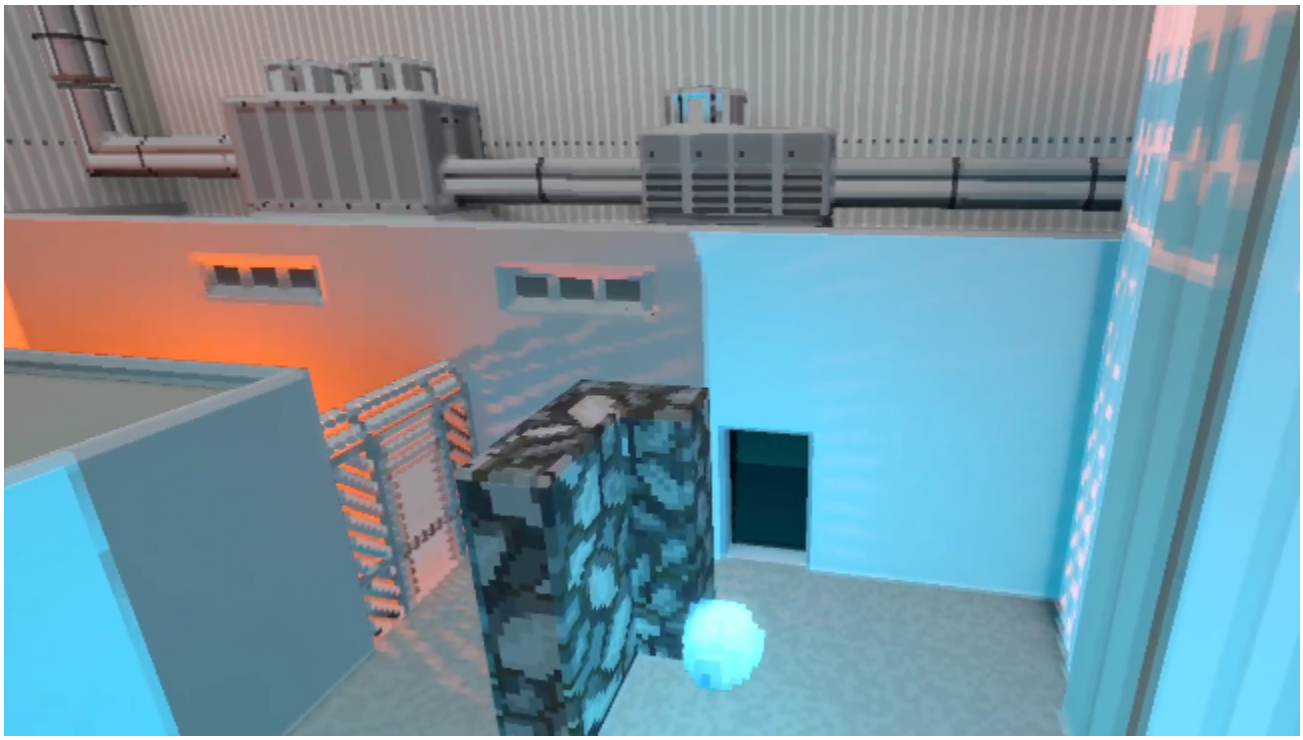
Allyon: Battlecry

Steam | **YouTube** | Discord | *by Clementine Games*



Voxel game (MountainLabz)

YouTube | Discord | *by MountainLabz*



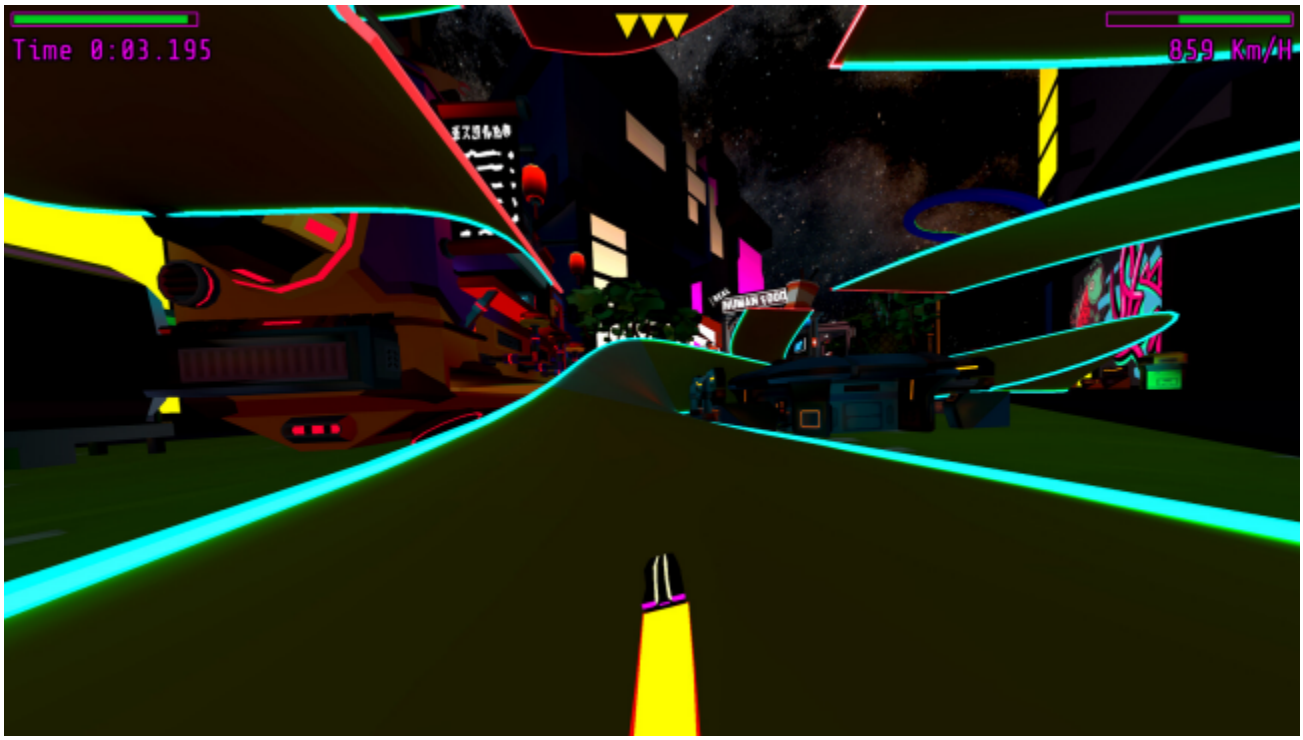
Voxel game (Ayrton)

YouTube | **BlueSky** | Discord | *by Ayrton*



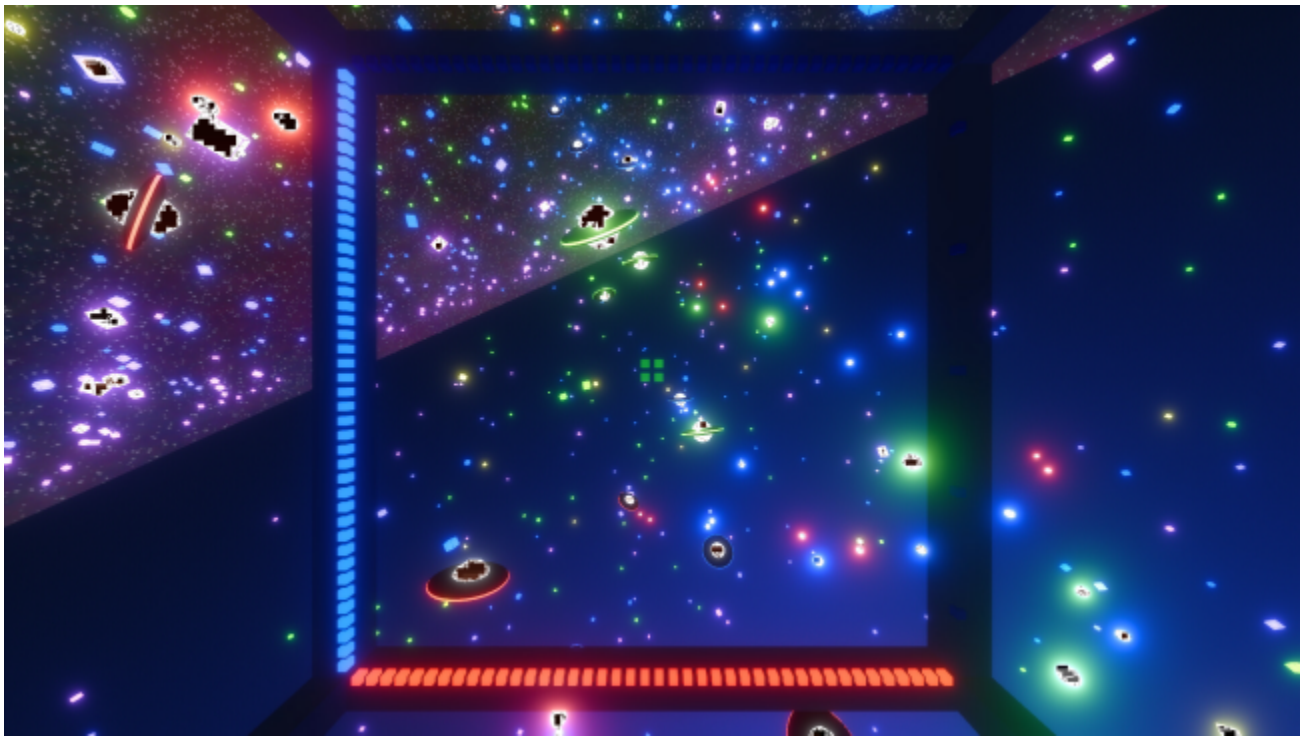
Neon Grand Prix

Steam | Discord | *by DMTE Consulting*



Super Space Crystals

Steam | **YouTube** | Discord | *by Benji Flaming*



Commercially released games

Finished games, available on platforms like Steam, Google Play, Apple Store, etc.

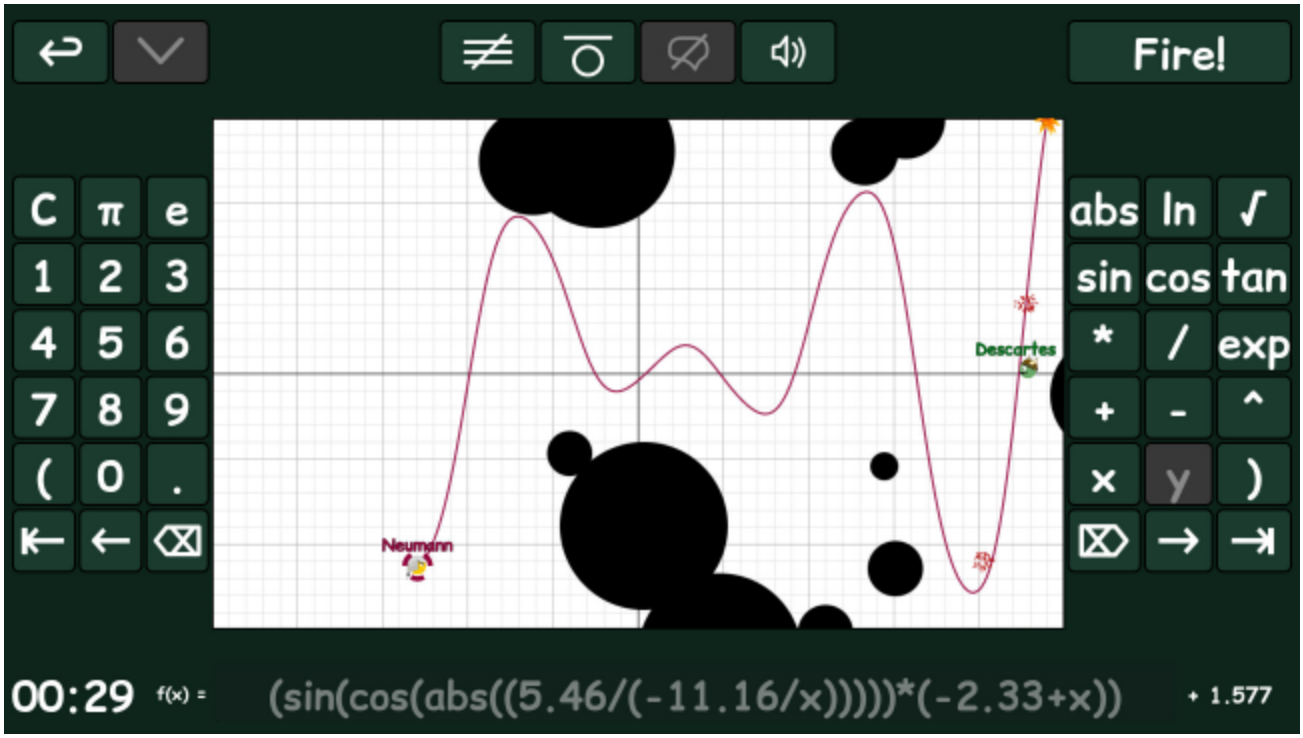
Solitaire Klondike Classic

Google Play | **Apple Store** | Discord | *by IsCool Entertainment*



Graphwar II

Steam | **Google Play** | **Apple Store** | Website | Discord | *by Catacorp*



Educational

Games and simulations from which you can learn, either through the game mechanics or by them being open-source.

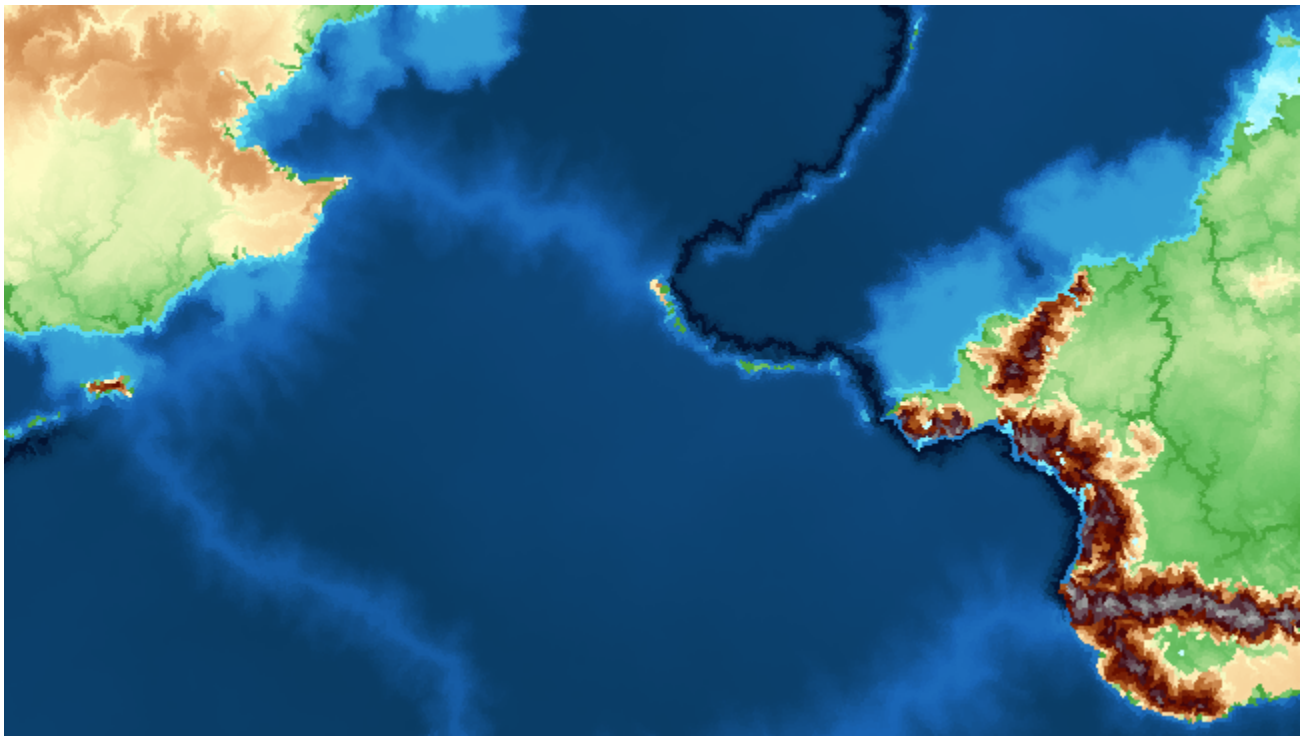
Musical constellations

GitHub (+ Downloads) | **YouTube** | Discord | *by Bauxite*



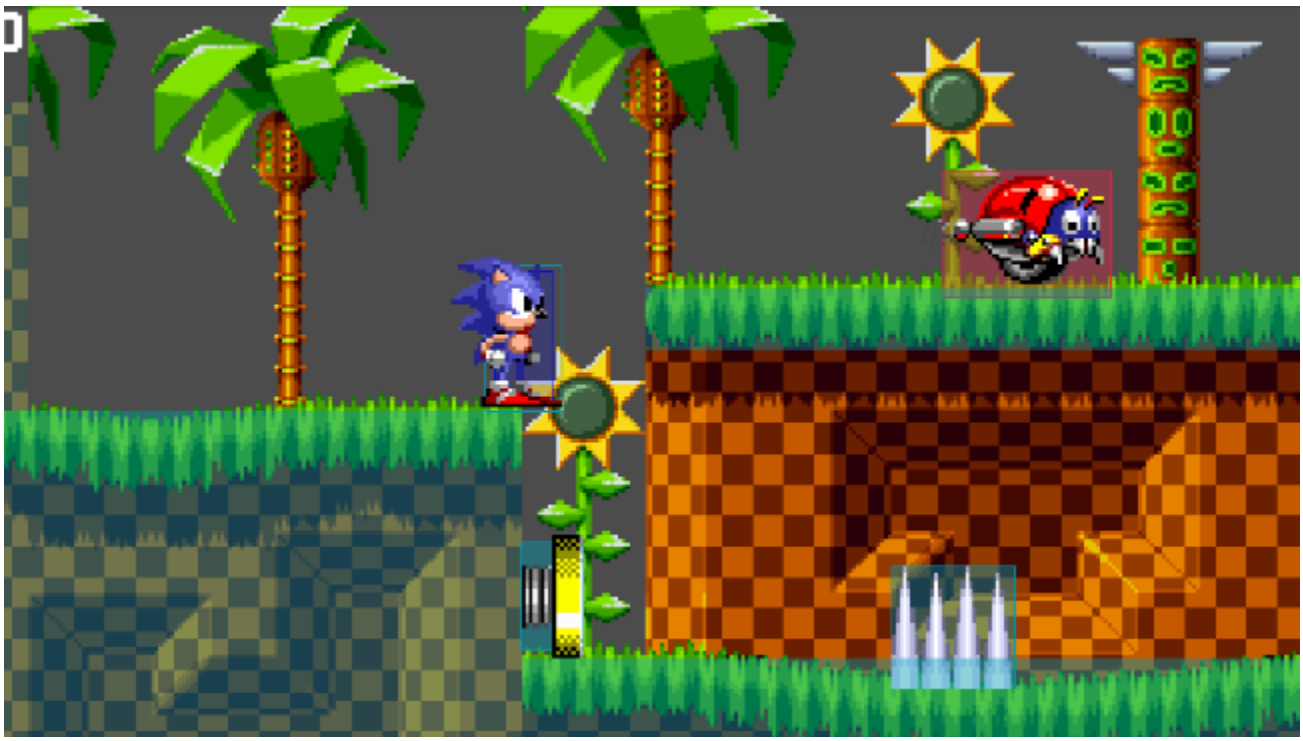
Gleba

Itch.io | Devlog | Discord | *by Calandiel*



Sonic Maker

GitHub | Discord | *by iride lombardi*

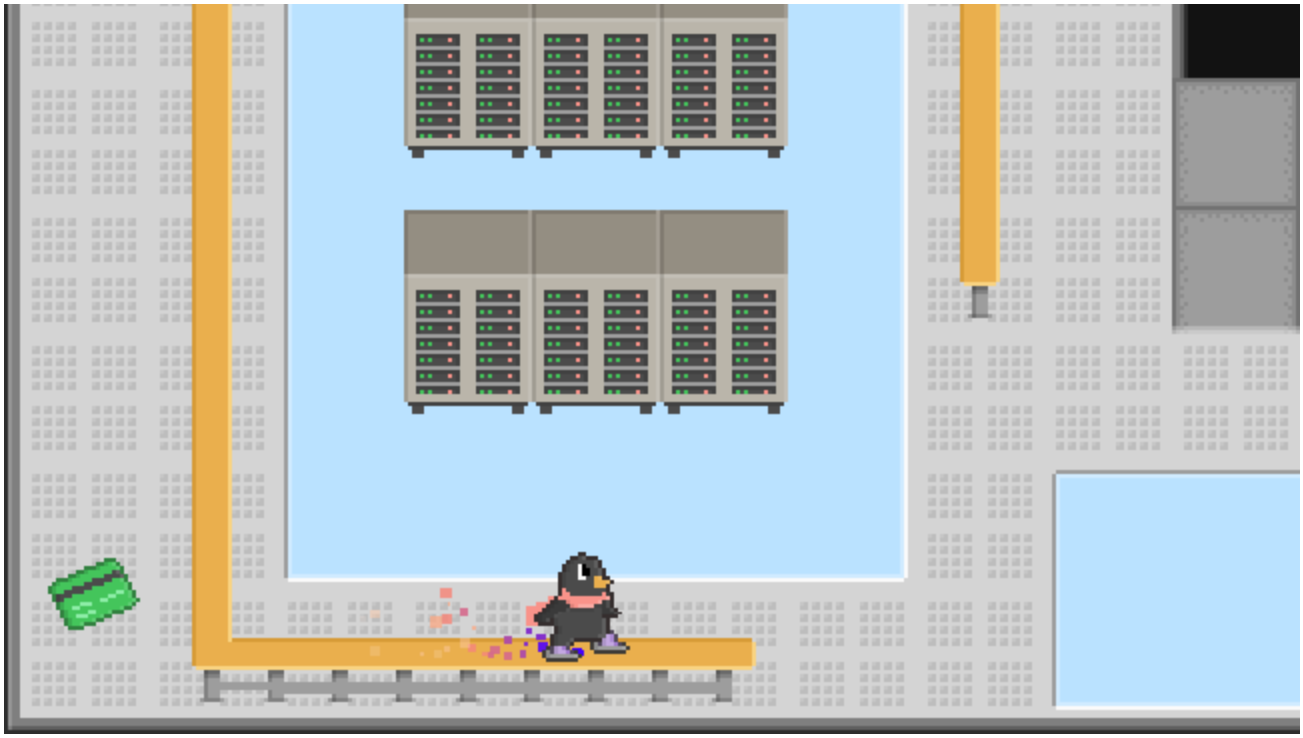


Web-playable

For people with only a few minutes to spare. Run godot-rust directly inside the browser. These games are developed using Godot's HTML5 Export.

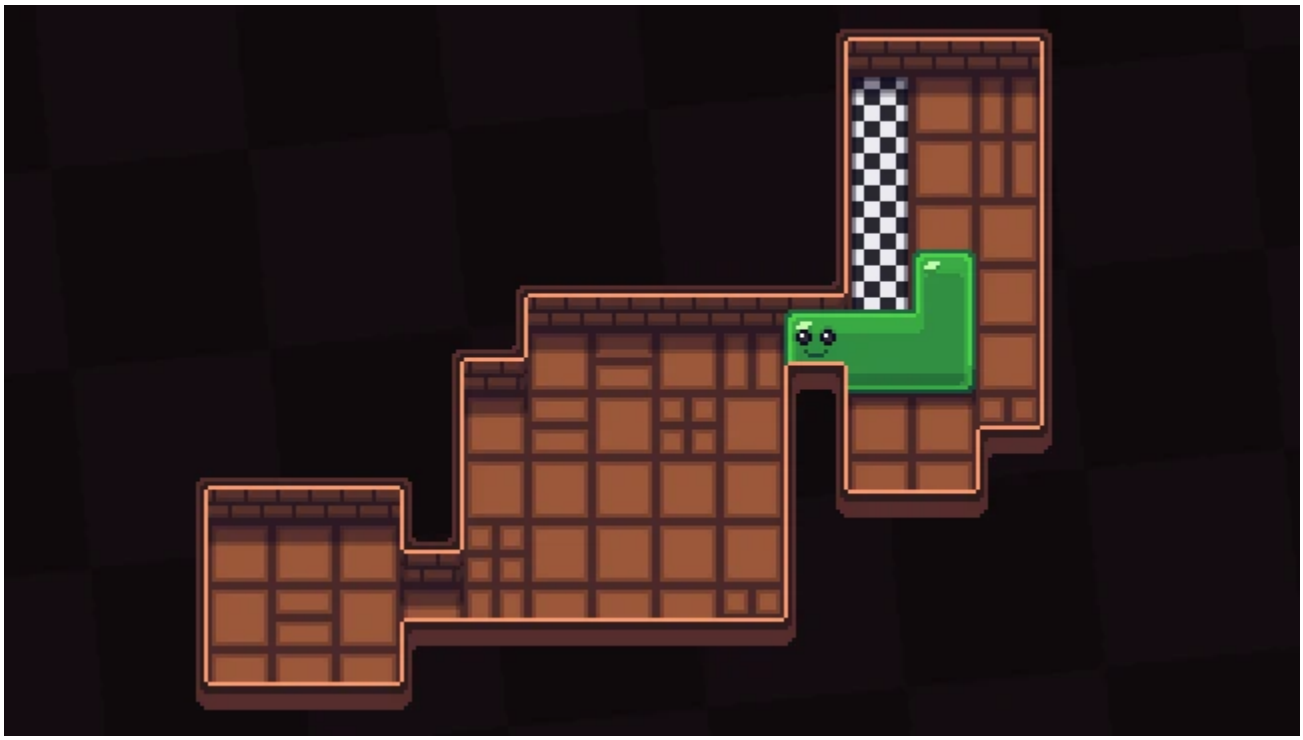
Iced Heist

Itch.io | *by Carrot Games* | 🥇 1st place in Winter Game Jam 2025



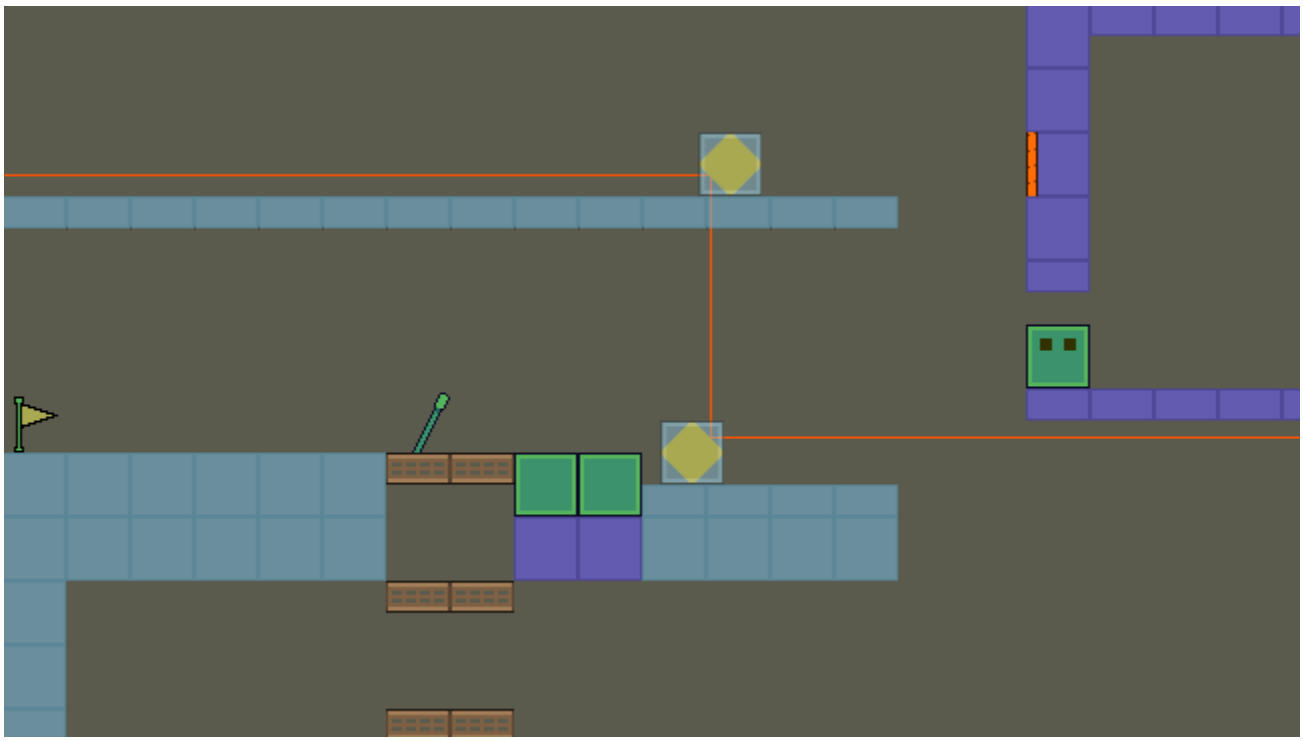
Stickyban

Itch.io | Discord | *by FlowVix*



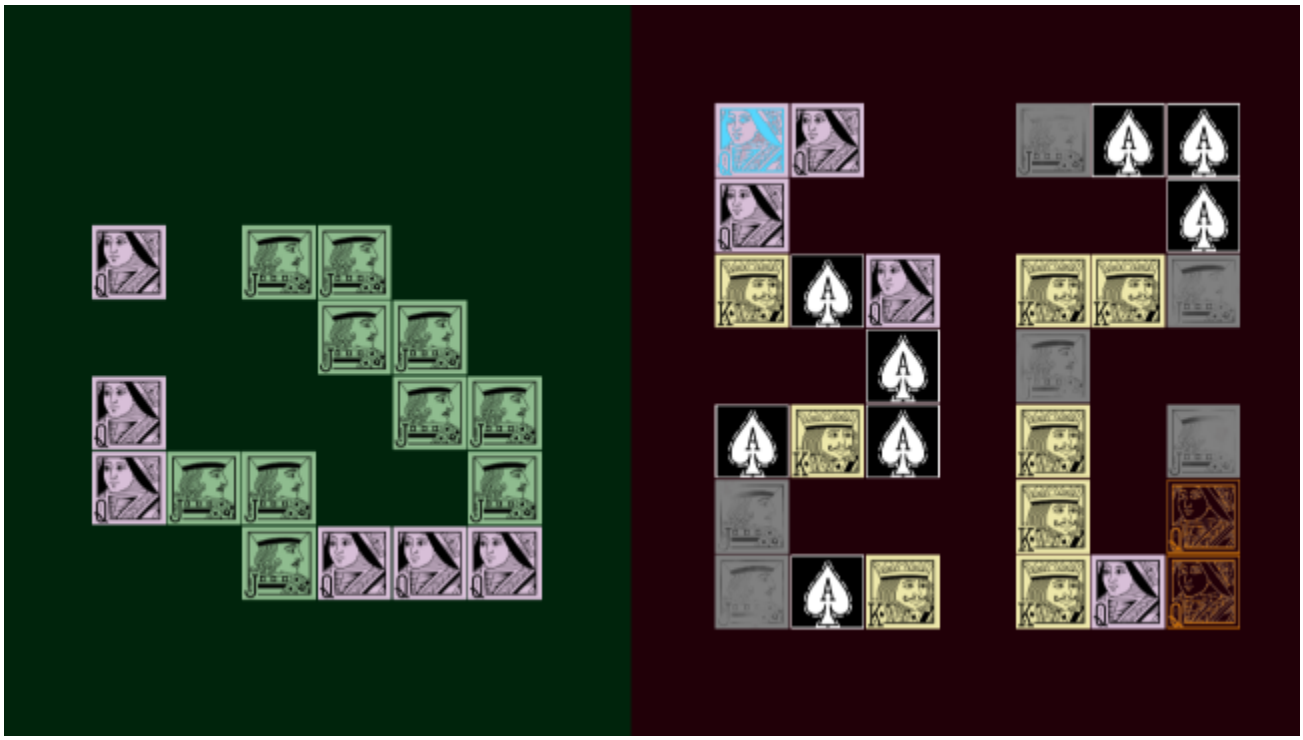
Block comprehension

Itch.io | Discord | *by Art4Aged*



Aces' Nim

Itch.io | Discord | *by Puleo*



Contributing to godot-rust

This chapter provides deeper information for people who are interested in contributing to the library. In case you are simply *using* godot-rust, you can skip this chapter.

If you haven't already, please read the Contributing guidelines in the repository first. The rest of this chapter explains developer tools and workflows in more detail. Check out the respective subchapters.

> **Tip**
>
> We highly appreciate if contributors propose a rough design before spending large effort on implementation. This aligns ideas early and saves time on approaches that may not work.

GitHub issues

We use issues on GitHub as a project management tool, not as a loose collection of ideas. To keep the issue tracker useful for both contributors and maintainers, we have established some rules.

Before opening a new issue, please search existing ones (including closed) and link them if related. Make sure you are using the latest `master` branch -- run `cargo update` if needed.

Every issue should be **actionable**, meaning:

1. It is **reproducible**. We don't expect that reporters go deep into debugging, but we expect a minimal, self-contained example that demonstrates a problem. This is true for both bugs and features (in which case the problem is a current limitation of the API).
2. It has a **well-defined scope**. It's clear under which conditions it is resolved.
    - "Improve error messages" or "my extension is slow" isn't helpful, but pointing to concrete messages or performance bottlenecks is.
3. It is fundamentally **solvable**. Given reasonable time, a motivated contributor could pick it up and make meaningful progress. Of course, some cases require significant knowledge of inner workings, but this is grounded in a technical challenge, not a formal one.

    - We don't track issues outside our control, unless there's a realistic chance to fix something in the near future (for which we have the `upstream` label). For example, potential Rust features 5 years into the future or big Godot changes that haven't even entered discussion stage fall into this category.

We appreciate reporters being available in the initial phase for follow-up clarifications, for example to narrow down when a bug occurs, or to understand the use case behind a suggested change. In rare cases, issues may be closed as `status: wontfix` if they deliver too low return on investment. Important here is not just initial development effort, but ongoing maintenance cost -- which no longer falls on the original contributor, and is thus often overlooked.

godot-rust's approach may differ from other open-source projects. However, we believe that having a huge collection of vague requests doesn't do anyone a favor. It makes it hard to see which issues are truly relevant and planned to be addressed; it discourages contributors from opening new issues (knowing those might just get buried); and it creates extra triage work for maintainers.

Sometimes people use issues to ask questions. This is fine -- we use the `question` label for those and usually close them after an answer.

Philosophy

Different gamedev projects have different goals, which determines how APIs are built and how they support various use cases.

Understanding the vision behind godot-rust allows users to:

- decide whether the library is the right choice for them
- comprehend design decisions that have influenced the library's status quo
- contribute in ways that align with the project.

Mission statement

If the idea behind the godot-rust project had to be summarized in a single word, it would be:

> **Pragmatism**
>
> **godot-rust** offers an ergonomic, safe and efficient way to access Godot functionality from Rust.
>
> It focuses on a productive workflow for the development of games and interactive applications.

In our case, pragmatism means that progress is driven by solutions to real-world problems, rather than theoretical purity. Engineering comes with trade-offs, and godot-rust is rather atypical for a Rust project. As such, we may sometimes deviate from Rust best practices that may apply in a clean-room setting, but fall apart when exposed to the interaction with a C++ game engine.

At the end of the day, people use Godot and Rust to build games, simulations or other interactive applications. The library should be designed around this fact, and Rust should be a tool that helps us achieve this goal -- not an end in itself.

In many ways, we follow similar principles as the Godot engine.

Scope

godot-rust is primarily a *binding* to the Godot engine. A priority is to make Godot functionality accessible for Rust developers, in ways that exploit the strengths of the language, while minimizing the friction.

Since we are not building our own game engine, features need to be related to Godot. We aim to build a robust core for everyday workflows, while avoiding overly niche features. Integrations with other parts of the gamedev ecosystem (e.g. ECS, asset pipelines, GUI) are out of scope and best implemented as extensions.

API design principles

We envision the following core principles as a guideline for API design:

1. **Solution-oriented approach**
   Every feature must solve a concrete problem that users or developers face.
   - We do not build solutions in search of problems. "Idiomatic Rust", "others also do it" or "it would be nice" are not good justifications :)
   - Priority is higher if more people are affected by a problem, or if the problem impacts a daily workflow more severely. In particular, this means that we can't spend much time on rarely used niche APIs, while there are game-breaking bugs in the core functionality.
   - We should always keep the big picture in mind. Rust makes it easy to get lost in irrelevant details. What matters is how a certain change helps end users.
2. **Simplicity**
   Prefer self-explanatory, straightforward APIs.
   - Avoid abstractions that don't add value to the user. Do not over-engineer prematurely just because it's possible; follow YAGNI and avoid premature optimization.
   - Examples to avoid: traits that are not used polymorphically, type-state pattern, many generic parameters, layers of wrapper types/functions that simply delegate logic.
   - Sometimes, runtime errors are better than compile-time errors. Most users are building a game, where fast iteration is key. Use `Option` / `Result` when errors are recoverable, and panics when the user must fix their code. See also Ergonomics and panics.
3. **Maintainability**
   Every line of code added **must be maintained, potentially indefinitely**.

- Consider that it may not be you working with it in the future, but another contributor or maintainer, maybe a year from now.
- Try to see the bigger picture -- how important is a specific feature in the overall library? How much detail is necessary? Balance the amount of code with its real-world impact for users.
- Document non-trivial thought processes and design choices as inline `//` comments.
- Document behavior, invariants and limitations in `///` doc comments.

4. **Consistency**
   As a user, having a uniform experience when using different parts of the library is important. This reduces the cognitive load of learning and using the library, requires less doc lookup and makes users more efficient.

   - Look at existing code and try to understand its patterns and conventions.
   - Before doing larger refactorings or changes of existing systems, get an understanding of the underlying design choices and discuss your plans.

See these as guidelines, not hard rules. If you are unsure, please don't hesitate to ask questions and discuss different ideas :)

Dev tools and testing

The library comes with a handful of tools and tricks to ease development. This page goes into different aspects of the contributing experience.

- Local development
  - API Docs
  - Unit tests
  - Integration tests
  - Formatting
  - Clippy
- Continuous Integration
  - Manually trigger a CI run
  - Trigger CI on push
- Build configurations
  - `real` type

Local development

The script `check.sh` in the project root can be used to mimic a minimal version of CI locally. It's useful to run this before you commit, push or create a pull request:

```
./check.sh
```

At the time of writing, this will run formatting, clippy, unit tests and integration tests. More checks may be added in the future. Run `./check.sh --help` to see all available options.

If you like, you can set this as a pre-commit hook in your local clone of the repository:

```
ln -sf check.sh .git/hooks/pre-commit
```

API Docs

Besides published docs, API documentation can also be generated locally using `./check.sh doc`. Use `dok` instead of `doc` to open the page in the browser.

Unit tests

Because most of godot-rust interacts with the Godot engine, which is not available from the test executable, unit tests (using `cargo test` and the `#[test]` attribute) are pretty limited in scope. They are primarily used for Rust-only logic.

Unit tests also include doctests, which are Rust code snippets embedded in the documentation.

As additional flags might be needed, the preferred way to run unit tests is through the `check.sh` script:

```
./check.sh test
```

Integration tests

The `itest` directory contains a suite of integration tests. It is split into two directories: `rust`, containing the Rust code for the GDExtension library, and `godot` with the Godot project and GDScript tests.

Similar to `#[test]`, the function annotated by `#[itest]` contains one integration test. There are multiple syntax variations:

```
// Use a Godot API and verify the results using assertions.
#[itest]
fn variant_nil() {
    let variant = Variant::nil();
    assert!(variant.is_nil());
}

// TestContext parameter gives access to a node in the scene tree.
#[itest]
fn do_sth_with_the_tree(ctx: &TestContext) {
    let tree: Gd<Node> = ctx.scene_tree.share();

    // If you don't need the scene, you can also construct free-standing nodes:
    let node: Gd<Node3D> = Node3D::new_alloc();
    // ...
    node.free(); // don't forget to free everything created by new_alloc().
}

// Skip a test that's not yet ready.
#[itest(skip)]
fn not_executed() {
    // ...
}

// Focus on a one or a few tests.
// As soon as there is at least one #[itest(focus)], only focused tests are
run.
#[itest(focus)]
fn i_need_to_debug_this() {
    // ...
}
```

You can run the integration tests like this:

```
./check.sh itest
```

Just like when compiling the crate, the `GODOT4_BIN` environment variable can be used to supply the path and filename of your Godot executable. Otherwise, a binary named `godot4` in your PATH is used.

Formatting

`rustfmt` is used to format code. `check.sh` only warns about formatting issues, but does not fix them. To do that, run:

```
cargo fmt
```

Clippy

`clippy` is used for additional lint warnings not implemented in `rustc`. This, too, is best run through `check.sh`:

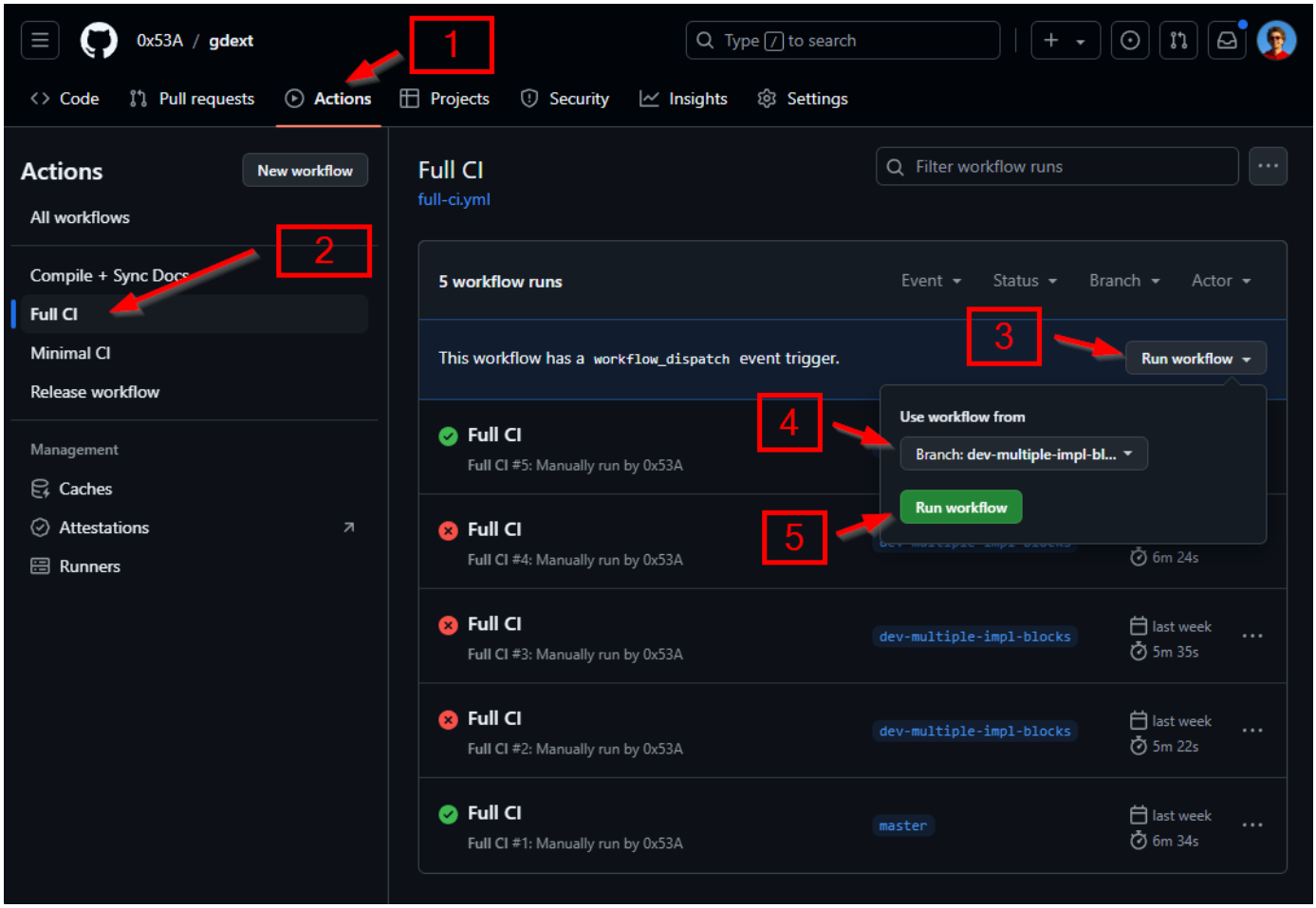
```
./check.sh clippy
```

Continuous Integration

If you want to have the full CI experience, you can experiment as much as you like on your own fork of the repository, before submitting a pull request.

Manually trigger a CI run

For one-off CI runs you can manually trigger it by enabling Actions in the project settings of your fork, then going to the `Actions` tab in the project, selecting the `Full CI` workflow, clicking on `Run Workflow` and selecting the branch you're working on:



Trigger CI on push

If you're working on a bigger feature, you might not want to have to trigger CI manually every time.

For this, navigate to the file `.github/workflows/full-ci.yml` and change the following lines:

```yaml
on:
  push:
    branches:
      - staging
      - trying
```

to:

```yaml
on:
  push:
```

This runs the entire CI pipeline to run on every push. You can then see the results in the

Actions tab in your repository.

Don't forget to undo this before opening a PR! You may want to keep it in a separate commit named "UNDO" or similar.

Build configurations

`real` type

Certain types in Godot use either a single or double-precision float internally, such as `Vector2`. When working with these types, we use the `real` type instead of choosing either `f32` or `f64`. As a result, our code is portable between Godot binaries compiled with `precision=single` and `precision=double`.

To run the testing suite with `double-precision` enabled you may add `--double` to a `check.sh` invocation:

```
./check.sh --double
```

Code and API conventions

- Bikeshed auto-painting
- Technicalities
  - Formatting
  - Code organization
  - Types
  - Functions
  - Attributes

Bikeshed auto-painting

In general, we try to automate as much as possible during CI. This ensures a consistent code style and avoids unnecessary work during pull request reviews.

In particular, we use the following tools:

- **rustfmt** for code formatting (config options).
- **clippy** for lints and style warnings (list of lints).
- Clang's **AddressSanitizer** and **LeakSanitizer** for memory safety.
- Various specialized tools:
  - **skywalking-eyes** to enforce license headers.
  - **cargo-deny** and **cargo-machete** for dependency verification.

In addition, we have unit tests (`#[test]`), doctests and Godot integration tests (`#[itest]`). See Dev tools for more information.

Technicalities

This section lists specific style conventions that have caused some confusion in the past. Following them is nice for consistency, but it's not the top priority of this project. Hopefully, we can automate some of them over time.

Formatting

`rustfmt` is the authority on formatting decisions. If there are good reasons to deviate from

it, e.g. data-driven tables in tests, use `#[rustfmt::skip]`. rustfmt does not work very well with macro invocations, but such code should still follow `rustfmt`'s formatting choices where possible.

Line width is 120-145 characters (mostly relevant for comments).
We use separators starting with `// ---` to visually divide sections of related code.

Code organization

1. Anything that is not intended to be accessible by the user, but must be `pub` for technical reasons, should be marked as `#[doc(hidden)]`.
    - This does **not** constitute part of the public API.
2. We do not use the `prelude` inside the project, except in examples and doctests.
3. Inside `impl` blocks, we *roughly* try to follow the order:
    - Type aliases in traits (`type`)
    - Constants (`const`)
    - Constructors and associated functions
    - Public methods
    - Private methods (`pub(crate)`, private, `#[doc(hidden)]`)
4. Inside files, there is no strict order yet, except `use` and `mod` at the top. Prefer to declare public-facing symbols before private ones.
5. Use flat import statements. If multiple paths have different prefixes, put them on separate lines. Avoid `self`.

```
// Good:
use crate::module;
use crate::module::{Type, function};
use crate::module::nested::{Trait, some_macro};

// Bad:
use crate::module::{self, Type, function, nested::{Trait, some_macro}};
```

Types

1. Avoid tuple-enums `enum E { Var(u32, u32) }` and tuple-structs `struct S(u32, u32)` with more than 1 field. Use named fields instead.

2. Derive order is `#[derive(GdextTrait, ExternTrait, Default, Copy, Clone, Eq, PartialEq, Ord, PartialOrd, Hash, Debug)]`.

    - `GdextTrait` is a custom derive defined by godot-rust itself (in any of the crates).
    - `ExternTrait` is a custom derive by a third-party crate, e.g. `nanoserde`.
    - The standard traits follow order *construction, comparison, hashing, debug display*. More expressive ones (`Copy`, `Eq`) precede their implied counterparts (`Clone`, `PartialEq`).

Functions

1. Getters don't have a `get_` prefix.

2. Use `self` instead of `&self` for `Copy` types, unless they are really big (such as `Transform3D`).

3. For `Copy` types, avoid in-place mutation `vector.normalize()`.
   Instead, use `vector = vector.normalized()`. The past tense indicates a copy.

4. Annotate with `#[must_use]` when ignoring the return value is likely an error.
   Example: builder APIs.

Attributes

Concerns both `#[proc_macro_attribute]` and the attributes attached to a `#[proc_macro_derive]`.

1. Attributes always have the same syntax: `#[attr(key = "value", key2, key_three = 20)]`

    - `attr` is the outer name grouping different key-value pairs in parentheses.
      A symbol can have multiple attributes, but they cannot share the same name.
    - `key = value` is a key-value pair. just `key` is a key-value pair without a value.
        - Keys are always `snake_case` identifiers.
        - Values are typically strings or numbers, but can be more complex expressions.
        - Multiple key-value pairs are separated by commas. Trailing commas are

allowed.

2. In particular, avoid these forms:

    - `#[attr = "value"]` (top-level assignment)
    - `#[attr("value")]` (no key -- note that `#[attr(key)]` is allowed)
    - `#[attr(key(value))]`
    - `#[attr(key = value, key = value)]` (repeated keys)

The reason for this choice is that each attribute maps nicely to a map, where values can have different types. This allows for a recognizable and consistent syntax across all proc-macro APIs. Implementation-wise, this pattern is directly supported by the `KvParser` type in godot-rust, which makes it easy to parse and interpret attributes.

Migration guides

See sidebar for concrete guides.

Migrating to v0.5

This guide covers deprecations and breaking changes when upgrading from godot-rust 0.4 to 0.5.

> **Smooth transition**
>
> To reduce the friction, we recommend first updating to the latest patch release of the previous minor version, v0.4.5. Many changes are announced early in the form of deprecation warnings, which contain instructions on how to switch to newer APIs.
>
> You can update your `Cargo.toml` to the latest patch release by running:
>
> ```
> cargo update -p godot
> ```
>
> Once you have addressed all deprecation warnings, you can update to the new minor version:
>
> ```
> cargo upgrade -p godot
> ```

Table of contents

- Godot versions
- Rust 2024 edition
- Required objects in engine APIs
- Property APIs
  - `Var` trait redesign
  - Const-qualified getters
  - `GodotShape` replaces property hint APIs
- `#[func]` bounds
  - `u64`
  - Raw pointers and `RawPtr<P>`
- `IObject` callback renames
- Typed collections
  - Typed `Dictionary<K, V>`
  - `vdict!` macro uses `=>` separator

    - `iarray!` and `idict!` macros for type inference
    - `AnyArray` -- covariant typed/untyped array
- Core API changes
    - Builder APIs for builtin methods
    - Type-safe `duplicate_node()` and `duplicate_resource()`
    - Geometric API changes
    - Module reorganization
    - Trait renames: `ArrayElement` and `PackedArrayElement`
- Other changes
    - `GFile::read_as_gstring_entire()` parameter removed
    - `GString`, `StringName` and `NodePath`: removed `arg()` method
    - Environment variable renames
- Removed deprecated symbols

Godot versions

#1484: godot-rust v0.5 adds **API level 4.6**, which is now the new default.

This doesn't mean you have to update your Godot version -- just provide Cargo features to use older API levels, e.g. `api-4-5`.

#1505: API level features are now limited to **minor versions** only. If you previously used patch-level features like `api-4-3-1`, you need to change them to the corresponding minor version, e.g. `api-4-3`.

Rust 2024 edition

#1500: godot-rust v0.5 uses the **Rust 2024 edition**.
The minimum supported Rust version (MSRV) is now **1.94**.

Required objects in engine APIs

#1383, #1493: GDExtension 4.6 provides non-null ("required") type information for function parameters and return types. This means that many Rust APIs can now become more type-safe, and your code will not need to deal with `Option` in cases where values are not

supposed to be null.

If you have `Option<Gd>` where Godot guarantees non-null objects, you'll need to change this to `Gd`.

Before (0.4):

```
let maybe_child: Option<Gd<Node>> = ...;
node.add_child(&maybe_child);

let tree = node.get_tree().unwrap();
let tween = node.create_tween().unwrap();
```

After (0.5):

```
// Option no longer compiles.
let definitely_child: Gd<Node> = ...; // non-null
node.add_child(&definitely_child);

let tree = node.get_tree(); // Panics if not in tree; otherwise
get_tree_or_null().
let tween = node.create_tween();
```

Property APIs

`Var` trait redesign

#1454, #1458, #1466, #1469: The `Var` trait has been redesigned to support `#[var(pub)]`, which generates typed Rust getters/setters with clone-based semantics. The trait API changed as follows:

Before (0.4):

```
pub trait Var: GodotConvert {
    fn get_property(&self) -> Self::Via;
    fn set_property(&mut self, value: Self::Via);
    fn var_hint() -> PropertyHintInfo { ... }
}
```

After (0.5):

```
pub trait Var: GodotConvert {
    type PubType;
    fn var_get(field: &Self) -> Self::Via;
    fn var_set(field: &mut Self, value: Self::Via);
    fn var_pub_get(field: &Self) -> Self::PubType;
    fn var_pub_set(field: &mut Self, value: Self::PubType);
}
```

If you have manual `Var` implementations, you need to update them. However, for most cases, you can use the new `SimpleVar` marker trait instead: types that already support `ToGodot + FromGodot + Clone` can get an automatic `Var` implementation via blanket impl.

Before (0.4):

```
impl Var for MyType {
    fn get_property(&self) -> Self::Via {
        self.to_godot_owned()
    }
    fn set_property(&mut self, value: Self::Via) {
        *self = FromGodot::from_godot(value);
    }
}
```

After (0.5):

```
// MyType: ToGodot + FromGodot + Clone.
impl SimpleVar for MyType {}
```

`#[var]` getter/setter semantics

#1454: The `#[var(get, set)]` keys are now independent from each other.

Attribute	Old behavior (0.4)	New behavior (0.5)
`#[var]`	Generates getter + setter	Same as before
`#[var(get)]`	*Disabled* setter	Declares custom `get_{field}`, setter unchanged
`#[var(set)]`	*Disabled* getter	Declares custom `set_{field}`, getter unchanged
`#[var(no_get)]`	New; was implied by `#[var(set)]`	Explicitly disables the getter

Attribute	Old behavior (0.4)	New behavior (0.5)
`#[var(no_set)]`	New; was implied by `#[var(get)]`	Explicitly disables the setter
`#[var(get = my_fn)]`	Custom getter fn	Same as before
`#[var(set = my_fn)]`	Custom setter fn	Same as before

Before (0.4):

```
// Disabled setter, only getter:
#[var(get)]
my_field: i32,

// Disabled getter, only setter:
#[var(set)]
other_field: f32,
```

After (0.5):

```
// Disabled setter, only getter:
#[var(no_set)]
my_field: i32,

// Disabled getter, only setter:
#[var(no_get)]
other_field: f32,

// Custom getter, generated setter:
#[var(get)]
custom_get_field: i32,
```

`#[var(pub)]` for Rust-side accessors

#1458, #1466: Auto-generated Rust getters/setters for `#[var]` fields are being phased out. In v0.4, `#[var]` fields implicitly generated public Rust accessor methods (e.g. `get_my_field()` / `set_my_field()`). These are now deprecated and will be removed in v0.6.

If you rely on these generated Rust methods, opt in explicitly with `#[var(pub)]`:

```
#[var(pub)]
my_field: i32,
// Generates:   fn get_my_field(&self) -> i32
//              fn set_my_field(&mut self, value: i32)
```

The `#[var(pub)]` accessors use `Clone`-based semantics (via `Var::PubType`), while the internal Godot-facing accessors continue to use `GodotConvert::Via`.

Compile-time type checks for custom `#[var]` accessors

#1469: Custom getters and setters declared with `#[var(get)]` or `#[var(set)]` are now type-checked at compile time. If your getter's return type or setter's parameter type doesn't match the field's `Var::PubType`, you'll get a compile error instead of a runtime failure.

#1518: Additionally, `#[var(no_get)]` combined with `#[export]` is now a compile error. Reading a `#[var(no_get)]` field from GDScript now panics with a helpful error message instead of silently returning the default value.

Const-qualified getters

#1497: A heuristic now const-qualifies Godot methods prefixed with `get_`, `is_`, or `has_`, changing their receiver from `&mut self` to `&self`. This affects a large number of engine methods, which were unnecessarily declared mutable.

An explicit deny-list handles false positives: methods that actually mutate state remain `&mut self`. Examples: `FileAccess::get_line()` which advances a cursor, `StreamPeer::get_8()`, etc.

This is a technically breaking change, but existing code using `mut` bindings will continue to compile. Clippy may emit warnings about unnecessary `mut`.

Before (0.4):

```
let mut api: Gd<MultiplayerApi> = ...;
let id = api.get_unique_id(); // Required &mut self.
```

After (0.5):

```
let api: Gd<MultiplayerApi> = ...;
let id = api.get_unique_id(); // Now takes &self.
```

`GodotShape` replaces property hint APIs

#1513, #1514, #1534: The internal representation of how types describe themselves to

Godot has been overhauled. A new `GodotShape` enum (returned by `GodotConvert::godot_shape()`) replaces several scattered APIs.

The following trait methods have been **removed** in favor of a unified `GodotConvert::godot_shape()`:

- `Var::var_hint()`
- `Export::export_hint()`
- `GodotType::property_info()`
- `GodotType::property_hint_info()`
- `GodotType::godot_type_name()`
- `GodotType::class_id()`
- `Element::element_type_string()`
- `PackedElement::element_type_string()`

If you had custom `Var` or `Export` implementations that overrode `var_hint()` or `export_hint()`, you now need to implement `GodotConvert::godot_shape()` instead, returning an appropriate `GodotShape` variant.

This refactor also enables **enums in collections**. Both engine- and user-defined `#[derive(GodotConvert)]` enums can now be used as elements in `Array<T>` or `Dictionary<K, V>`.

`#[func]` bounds

#1435: Parameters and return types in `#[func]` have been updated. Newly supported are user-defined enums.

Some types have also been restricted -- the following no longer implement `ToGodot` / `FromGodot` and thus cannot be passed to or returned from `#[func]`:

u64

#1413: Primary reason for disallowing `u64` is that GDScript (and `Variant`) cannot natively store integers other than `i64`, and it's very easy to introduce logic errors that silently change the value. See also rationale in the API docs.

If you have existing uses of `u64`, you can use either:

- `i64` with casting.
- any smaller unsigned integer type, if the range is enough.
- a custom type and override `GodotConvert::godot_shape()` with param metadata.

Raw pointers and `RawPtr<P>`

#1436: Raw pointers in engine APIs (native structures, `c_void`, etc.) are now wrapped in `RawPtr<P>`, replacing direct `*const T` / `*mut T` parameters. Construction requires `unsafe`, which wasn't the case before -- you could previously return a pointer to a local variable from `#[func]` without ever typing `unsafe`.

Before (0.4):

```
#[func]
fn pass_native_struct(&self, caret_info: *const CaretInfo) -> VarDictionary {
    let info = unsafe { &*caret_info };
    // use &CaretInfo directly.
}

#[func]
fn native_struct_array_ret(&self) -> *const Glyph {
    self.some_raw_ptr
}
```

Besides the native-structure types, the following raw pointer impls for `ToGodot` / `FromGodot` have been removed:

- `*const std::ffi::c_void`
- `*mut std::ffi::c_void`
- `*mut *const u8`
- `*mut i32`
- `*mut f64`
- `*mut u8`
- `*const u8`

After (0.5):

```
use godot::meta::conv::RawPtr;

#[func]
fn pass_native_struct(&self, caret_info: RawPtr<*const CaretInfo>) ->
VarDictionary {
    let caret_raw: *const CaretInfo = caret_info.ptr();
    let info = unsafe { &*caret_raw };
    // use &CaretInfo directly.
}

#[func]
fn native_struct_array_ret(&self) -> RawPtr<*const Glyph> {
    // SAFETY: caller must ensure returning pointer to Godot is safe.
    unsafe { RawPtr::new(self.some_raw_ptr) }
}
```

`IObject` callback renames

#1527: Virtual callbacks in the `IObject` trait (and related `I*` traits inheriting from `Object`) are now prefixed with `on_`, to avoid naming collisions with same-named methods on `Object` and to clarify that these are callbacks.

Old name	New name
`get_property()`	`on_get()`
`set_property()`	`on_set()`
`get_property_list()`	`on_get_property_list()`
`validate_property()`	`on_validate_property()`

The old names are deprecated and will continue to function until v0.6.

Typed collections

Typed `Dictionary<K, V>`

#1502, #1507, #1516: Dictionaries are now generic over key and value types: `Dictionary<K, V>`. This mirrors the typed `Array<T>` API.

- `Dictionary<K, V>` is the typed dictionary.

- `AnyDictionary` is a type-erased dictionary (analogous to `AnyArray`).
- `VarDictionary` is now an alias for `Dictionary<Variant, Variant>`.
- Engine APIs now accept `&AnyDictionary` instead of `&VarDictionary`, so you can pass any `&Dictionary<K, V>` directly.
- Typed dictionary iterators are available: `dict.iter_shared()` returns `(K, V)` pairs.

This is *mostly* a backwards-compatible change as `VarDictionary` still exists, however there are some places which need attention.

Methods that accept values have changed bounds from `ToGodot` to `AsArg<Variant>`, as a special case of `AsArg<K>` / `AsArg<V>`. Elements whose `ToGodot::Pass` is not `ByValue` (e.g. objects, callables) now need to be passed by reference:

Before (0.4):

```
dict.set("hello", gd);
```

After (0.5):

```
dict.set("hello", &gd);
```

`vdict!` macro uses `=>` separator

#1523: The `vdict!` macro now uses `=>` instead of `:` to separate keys from values. This avoids ambiguity with Rust's token tree parsing and removes the need to wrap complex key expressions in parentheses. It's also more in line with crates like `maplit`.

Before (0.4):

```
let tiles = vdict! {
    (Vector2i::new(1, 2)): Tile::GRASS,
    (Vector2i::new(1, 3)): Tile::WATER,
};
```

After (0.5):

```
let tiles = vdict! {
    Vector2i::new(1, 2) => Tile::GRASS,
    Vector2i::new(1, 3) => Tile::WATER,
};
```

In good old godot-rust fashion, the `:` syntax continues to be supported during a transition

period (with deprecation warnings).

`iarray!` and `idict!` macros for type inference

#1530: godot-rust v0.5 underwent a significant refactoring to maintain decent `Variant` support in `Dictionary<K, V>`. The short story is that we can no longer rely on `K`/`V` being `Variant`, so we had to replace `ToGodot` with the more general `AsArg<K/V>` bounds. This comes at a cost: due to more candidates for `AsArg` conversions, type inference is ambiguous in situations where it cannot be deduced from the context (like function return types).

```
let d = dict! { "key": 10 }; // Error: "key" could be GString, StringName or
Variant.

// dict! now needs explicit type annotations:
let d: Dictionary<GString, i32> = dict! { "key": 10 };
```

Typing three extra identifiers is of course beyond the acceptable, so we have a solution.

New macros `iarray!` and `idict!` ("**i**nferred") create arrays and dictionaries without explicit type hints. They use opinionated but unambiguous type inference. For example, `iarray!["hello"]` infers `Array<GString>`, never `Array<StringName>`. Not all element types are supported as of now.

The existing `array!` and `dict!` macros remain unchanged and still require type context (annotation, function parameter, etc.).

```
// No type annotation needed:
let ints = iarray![3, 1, 4];        // Array<i32>
let strs = iarray!["a", "b"];       // Array<GString>
let d = idict! { "key" => 10 };     // Dictionary<GString, i32>

// array!/dict! still work with type context:
let ints: Array<i64> = array![3, 1, 4];
```

`AnyArray` -- covariant typed/untyped array

#1422, #1434: A new `AnyArray` type provides a type-erased array that can store either typed `Array<T>` or untyped `VarArray`. It is covariant: any `&Array<T>` can be used where `&AnyArray` is expected, thanks to `Deref` coercion.

```rust
use godot::builtin::{Array, AnyArray, VarArray};

let typed: Array<i64> = array![1, 2, 3];

// Deref coercion: use typed array where AnyArray is expected.
let any_ref: &AnyArray = &typed;
assert_eq!(any_ref.len(), 3);

// Explicit upcast for owned values.
let any: AnyArray = typed.clone().upcast_any_array();

// Downcast back to typed.
let back: Array<i64> = any.try_cast_array::<i64>()
    .expect("convert back to typed");
```

Engine APIs that previously accepted `&VarArray` parameters (like `Callable::callv()`) now take `&AnyArray`, which means you can pass any `&Array<T>` directly without manual conversion. If you were explicitly constructing a `VarArray` to pass to such methods, note that `&VarArray` coerces to `&AnyArray` via `Deref`, so no changes should be necessary in most cases.

For owned values (e.g. returned by virtual functions), you can use the `upcast_any_array()` method to convert an `Array<T>` into an `AnyArray`.

Core API changes

Builder APIs for builtin methods

#1424: Builtin types (`GString`, `Vector2`, `Color`, etc.) now support the `_ex()` builder pattern for methods with default parameters, consistent with how engine class methods already work.

Before (0.4):

```rust
let pi = GString::num(std::f64::consts::PI, 3);
```

After (0.5):

```rust
let pi = GString::num_ex(std::f64::consts::PI).decimals(3).done();
```

The non-`_ex` variants remain available and use Godot's default parameter values.

In v0.4, we often emulated the builder pattern -- inconsistently at that:

- by requiring all parameters
- with multiple methods
- with our handcrafted builder

If you used such methods, your code will need to be updated to the new pattern.

Type-safe `duplicate_node()` and `duplicate_resource()`

#1492: New generic duplication methods return the concrete type instead of requiring manual downcasts.

```
let node: Gd<Node2D> = Node2D::new_alloc();
let copy: Gd<Node2D> = node.duplicate_node(); // Directly typed.

// Builder pattern for fine-grained control:
let copy = node.duplicate_node_ex()
    .flags(DuplicateFlags::SIGNALS | DuplicateFlags::GROUPS)
    .done();

let resource: Gd<Resource> = Resource::new_gd();
let copy: Gd<Resource> = resource.duplicate_resource();

// Deep duplication (Godot 4.5+):
let deep = resource.duplicate_resource_ex()
    .deep(DeepDuplicateMode::ALL)
    .done();
```

The old `Node::duplicate()` and `Resource::duplicate()` methods are deprecated in favor of these new type-safe variants.

Geometric API changes

#1447, #1461, #1463: Several geometric types gained new methods:

- `Aabb::get_endpoint()` -- returns a corner vertex of the bounding box.
- `Aabb` cleanup: additional methods like `get_center()` and `get_support()` (previously `support()`).
- `Rect2` / `Rect2i`: corner constructors and additional utility methods.

The `from_corners()` constructors on `Rect2`, `Rect2i`, and `Aabb` have been deprecated in favor of `from_position_end()`, which better describes the parameters (position + end point,

not two arbitrary corners).

Module reorganization

#1531, #1537: The `meta`, `register`, and signal modules have been reorganized. If you import specific items from these modules, your import paths may need updating:

Symbol(s)	Old module (0.4)	New module (0.5)
`GodotShape`, `GodotElementShape`, `Enumerator`	`godot::meta`	`godot::meta::shape`
`ByValue`, `ByRef`, `ByObject`, `ArgPassing`	`godot::meta`	`godot::meta::conv`
`ElementType`	`godot::meta`	`godot::meta::inspect`
`GodotConvert` (derive macro)	`godot::register`	`godot::meta`
`Export`, `Var` (derive macros)	`godot::register`	`godot::register::property`
`TypedSignal`, `ConnectHandle`, signal types	`godot::register`	`godot::signal`
`PropertyHintInfo`, `PropertyInfo`, `MethodInfo`	`godot::meta`	`godot::register::info`

Most commonly used symbols remain available in `godot::prelude` and are not affected.

Trait renames: `ArrayElement` and `PackedArrayElement`

#1506: The traits `ArrayElement` and `PackedArrayElement` have been renamed to `Element` and `PackedElement`, respectively.

If you use these traits as bounds in generic code, update the names accordingly.

Other changes

`GFile::read_as_gstring_entire()` parameter removed

#1494: The `skip_cr` parameter has been removed from `GFile::read_as_gstring_entire()` to provide a uniform API across Godot versions.

Before (0.4):

```
let text = gfile.read_as_gstring_entire(true)?;
```

After (0.5):

```
let text = gfile.read_as_gstring_entire()?;
```

`GString`, `StringName` and `NodePath`: removed `arg()` method

The `.arg()` methods on `GString` and `StringName` have been removed. They were used to convert between Godot string types when passing arguments to functions expecting a different string type. The idea is to come up with a more general `AsArg` conversion method, possibly reserving the `.arg()` name for that in the future.

To migrate, use `T::from()`:

Before (0.4):

```
// Node::is_in_group() takes impl AsArg<StringName>
let group: GString = some_group_name();
node.is_in_group(group.arg());
```

After (0.5):

```
let group: GString = some_group_name();
node.is_in_group(&StringName::from(&group));
```

Environment variable renames

#1521: All environment variables now use a uniform `GDRUST_` prefix. The old names are still recognized but emit a deprecation warning.

Old name	New name
`GODOT4_BIN`	`GDRUST_GODOT_BIN`
`GODOT4_GDEXTENSION_JSON`	`GDRUST_GODOT_API_JSON`

Old name	New name
`GODOT_RUST_NOWARN`	`GDRUST_SUPPRESSED_WARNINGS`
(new in v0.5)	`GDRUST_MAIN_EXTENSION`

Removed deprecated symbols

#1459: All symbols that were deprecated during the v0.4 cycle have been removed. If you have already addressed all deprecation warnings in v0.4.5, you should be covered for the most part. The following table lists the removed symbols and their replacements:

Removed symbol	Replacement
`VariantArray` (type alias)	`VarArray`
`ClassName` (type alias)	`ClassId`
`GodotClass::class_name()`	`GodotClass::class_id()`
`WithBaseField::apply_deferred()`	`run_deferred()` + `run_deferred_gd()`
`Gd::apply_deferred()`	`Gd::run_deferred()` + `Gd::run_deferred_gd()`
`Callable::from_local_fn()`	`Callable::from_fn()`
`Callable::from_local_static()`	`Callable::from_class_static()`
`*::hash()` (Godot hash value)	`*::hash_u32()`
`Array::bsearch_custom()`	`Array::functional_ops()` `.bsearch_custom()`
`ExtensionLibrary::on_level_init()`	`ExtensionLibrary::on_stage_init()`
`ExtensionLibrary::on_level_deinit()`	`ExtensionLibrary::on_stage_deinit`
`ExtensionLibrary::on_main_loop_startup()`	`on_stage_init(InitStage::MainLoop`
`ExtensionLibrary::on_main_loop_shutdown()`	`on_stage_deinit(InitStage::MainLo`
`#[class(no_init, base=EditorPlugin)]` (warning)	Now a hard compile error; use `#[class(init)]`

Migrating to v0.4

This guide covers deprecations and breaking changes when upgrading from godot-rust 0.3 to 0.4.

Table of contents

- Godot version support
- Overhauls to core APIs
  - Argument passing redesign
  - `Callable` return types
  - `Singleton` trait
  - Removed conversions
  - Renames
- Registration changes
  - `#[export(range)]` limits and type validation
  - Export range for radians
  - Editor class validation
- Engine APIs
  - `EngineEnum` + `EngineBitfield` introspection
  - Array and string indexing with negative values

Godot version support

#1292: godot-rust v0.4 **drops support for Godot 4.1**.

The library now supports Godot versions 4.2 through 4.5, including the latest 4.5 release which is now the default (meaning that you need appropriate `api-4-x` feature to run it with previous Godot versions).
See also *Compatibility and stability*.

Overhauls to core APIs

Argument passing redesign

#1285, #1308, #1310, #1314: So far, `AsArg` and `ToGodot` traits have had very similar roles, namely converting Rust types to Godot-compatible types. However, they had only limited interop. For example, it wasn't easily possible to pass user-defined enums to `emit()` of signals.

`ToGodot::Pass` and automatic `AsArg` impls

The `ToGodot` trait has been simplified, replacing the associated type `ToVia<'v>` with `Pass`, which usually has one of two values. `AsArg` is then auto-implemented for types that support `ToGodot`, in the following way:

- `Pass = ByValue`: what you typically want. This implements `AsArg<T>` for owned types `T`, e.g. you can pass `i64` directly to Godot.
- `Pass = ByRef` for specifically optimized types. This implements `AsArg<T>` for references `&T`, e.g. you can pass `&GString` to Godot without cloning. As a user, you typically need to convert to a custom type anyway, so `ByValue` is often easier.

If you follow those, you should never need to implement `AsArg` yourself. `ToGodot` is sufficient, and can often be derived.

Before (0.3):

```
impl ToGodot for MyString {
    type ToVia<'v> = Self::Via;

    fn to_godot(&self) -> Self::ToVia<'_> {
        GString::from(self)
    }
}
```

After (0.4):

```
use godot::meta;

impl ToGodot for MyString {
    type Pass = meta::ByValue;

    // Returns a new GString here, since we have to create one anyway.
    fn to_godot(&self) -> Self::Via {
        GString::from(self)
    }
}

impl ToGodot for MyCachedString {
    type Pass = meta::ByRef;

    // Returns &GString here, since we store a reference.
    fn to_godot(&self) -> &Self::Via {
        &self.cached_gstring
    }
}
```

Return type changes

For reference-counted types (`GString` , `Array` , `Dictionary` , `Variant` ...), `to_godot()` now returns references. A value can be obtained by calling `to_godot_owned()` .

Before (0.3):

```
let a: GString = my_value.to_godot();
```

After (0.4):

```
let a: &GString = my_value.to_godot();
let b: GString = my_value.to_godot_owned();
```

Object argument traits

The specialized `AsObjectArg<T>` trait has been consolidated into the more general `AsArg<Gd<T>>` trait, unifying the type system. It is also capable of expressing optional arguments through `AsArg<Option<Gd<T>>>` .

Before (0.3):

```
#[signal]
fn my_signal(some_node: Gd<Node>);
let node = Node::new_alloc();
let derived = Node2D::new_alloc();

// Argument had to be upcast.
sig.emit(&derived.upcast());

// The type could be inferred, but arguments had to be implicitly upcast.
let _array = array![&node, &derived.upcast()];
```

After (0.4):

```
fn my_signal(some_node: Gd<Node>);
let node = Node::new_alloc();
let derived = Node2D::new_alloc();

// Will be implicitly upcast.
sig.emit(&derived);

// Type must be specified, but arguments will be implicitly upcast.
let _array: Array<Gd<Node>> = array![&node, &derived];
```

`Callable` return types

#1332, #1344, #1346: `Callable` constructor `from_local_fn()` is phased out in favor of `from_fn()`, which supports any return type implementing `ToGodot`, eliminating the need for manual `Variant` conversion and wrapping inside `Result`. The `Err` variant didn't add much purpose but required a lot of boilerplate, and errors can still be supported through panics or `RustCallable` customization.

Before (0.3):

```
let callable = Callable::from_local_fn("answer", |args| {
    Ok(42.to_variant())
});

let callable = Callable::from_local_fn("unit", |args| {
    do_sth(args); // Some side effect, no return value.
    Ok(Variant::nil())
});
```

After (0.4):

```
let callable = Callable::from_fn("answer", |args| 42);

let callable = Callable::from_fn("unit", |args| {
    do_sth(args);
});
```

`Singleton` trait

#1325: Singleton access has been moved to a dedicated `Singleton` trait, enabling generic programming while maintaining backward compatibility with existing `singleton()` methods.

In the future, we may look into other options to provide singletons, especially once we support them for user classes, too.

Before (0.3):

```
let input = Input::singleton();
let engine = Engine::singleton();
```

After (0.4):

If you already include the prelude, no code changes should be necessary:

```
use godot::prelude::Singleton;

let input = Input::singleton(); // Still works
let engine = Engine::singleton(); // Still works

// Now supports generic access:
fn summon_the_one<T: Singleton>() -> Gd<T> {
    T::singleton()
}
```

Removed conversions

#1286, #1316: By-value `From` conversions between string types have been removed to clarify that no buffer optimization occurs during conversion. You can still use `From` for references.

Before (0.3):

```
let gstring = GString::from("hello");
let sname = StringName::from(gstring); // or .into()
```

After (0.4):

```
let gstring = GString::from("hello");
let sname = StringName::from(&gstring);
```

The `From<&'static CStr>` implementation for `StringName` has been removed, since Godot no longer offers a performance benefit, and static storage duration may very easily cause memory leaks.

Furthermore, `impl From<VariantArray> for Packed*Array` has been removed. This was unsound; `VariantArray` can contain arbitrary types.

Renames

- Struct `ClassName` -> `ClassId`, which represents the idea of lightweight IDs better.
- Method `apply_deferred()` -> `run_deferred()` and `run_deferred_gd()`, accepting both `&mut T` and `Gd<T>` parameters.
- Method `Base::to_gd()` -> `Base::to_init_gd()`, for proper access during `init`.
- Method `Callable::from_local_fn()` -> `Callable::from_fn()` (see above).
- Assoc fn `Callable::from_local_static()` -> `Callable::from_class_static()`.
- Trait `ToSignalObject` -> `ObjectToOwned`.
- Trait `WithDeferredCall` -> merged into `WithBaseField`.

Registration changes

`#[export(range)]` limits and type validation

#1320: `#[export(range = ...)]` attribute traditionally required float literals, even for integer fields. The values were also not validated against the field type, allowing out-of-bounds values.

Furthermore, reasonable range bounds are now inferred for all integer types.

Before (0.3):

```
#[export(range = (1.0, 500.0))]  // Float was required, no bound checks.
int_field: i8,

#[export]  // Default: full i64 range, allowing invalid values in editor.
another_field: i8,
```

After (0.4):

```
#[export(range = (1, 127))]  // Must be i8-compatible literals, within bounds.
int_field: i8,

#[export]  // Default: infer range from i8 as (-128, 127).
another_field: i8,
```

This change improves reliability, but may require updates to existing export declarations.

Export range for radians

#1320: The `radians` option was deprecated in Godot itself, so the attribute was updated to use `radians_as_degrees`, which follows Godot's current convention. Using the old `radians` will lead to a descriptive error.

Before (0.3):

```
#[export(range = (radians))]
angle: f32,
```

After (0.4):

```
#[export(range = (radians_as_degrees))]
angle: f32,
```

Editor class validation

#1272: Classes with names starting with "Editor" must now be explicitly marked with the `internal` attribute to prevent accidental exposure in exported builds. Godot has a special and undocumented rule to make those internal, which led to confusion.

Before (0.3):

```
#[derive(GodotClass)]
#[class(base=EditorPlugin)]
struct EditorMyPlugin {
    base: Base<EditorPlugin>,
}
```

After (0.4):

```
#[derive(GodotClass)]
#[class(base=EditorPlugin, internal)]
struct EditorMyPlugin {
    base: Base<EditorPlugin>,
}
```

Engine APIs

`EngineEnum` + `EngineBitfield` introspection

#1232: Since some enums in Godot contain duplicates, there isn't a unique name for an enum variant (which is just an integer internally). Thus, `EngineEnum::godot_name()` sometimes returned incorrect names. The new API provides `all_constants()` for a full introspection, and `values()` for just distinct, useful values (e.g. when you need a drop-down list).

`EngineBitfield` also has `all_constants()` now, but no `values()`.

Before (0.3):

```
let name = MyEnum::Variant1.godot_name();
```

After (0.4):

```
let constants = MyEnum::all_constants();
// Use constants map to find names.
```

Array and string indexing with negative values

#1300: Operations that support negative indexing (arrays, strings, etc.) now use

`SignedRange` instead of accepting raw integer parameters. This provides better type safety for range operations that can include negative indices.

Before (0.3):

```
let slice = array.slice(start, end); // Raw integers
```

After (0.4):

```
use godot::meta::wrapped;

let a = array.slice(wrapped(..-2));  // from 0 to len-2
let b = array.slice(wrapped(1..-2)); // from 1 to len-2
```

Where possible and we consider it likely that the user named symbols in their own code, we provide deprecated aliases for the v0.4 cycle, to ease the transition and provide helpful warnings.

Migrating to v0.3

This chapter will guide you through the changes from godot-rust version 0.2 to 0.3. See also our May dev update for a feature overview, and our changelog for a detailed list of modifications. Breaking changes are marked as such in the changelog, and you can navigate to the respective PRs to get in-depth information.

> **Smooth transition**
>
> To reduce the friction, we recommend first updating to the latest patch release of the current minor version, before switching to the new minor version. Many changes are announced early in the form of deprecation warnings, which contain instructions on how to switch to newer APIs.
>
> You can update your `Cargo.toml` to the latest patch release by running:
>
> ```
> cargo update -p godot
> ```
>
> Once you have addressed all deprecation warnings, you can update to the new minor version:
>
> ```
> cargo upgrade -p godot
> ```

Geometric APIs

Several geometric APIs have seen slight updates in terms of naming and semantics.

- `Aabb`, `Rect2`, `Rect2i` (#1001)
  - `has_point -> contains_point`
  - `has_area -> has_surface`
  - `intersection -> intersect`
  - `intersect_ray`: added in addition to `intersects_ray`, returning intersection point
- `Basis` + `Quaternion` (#1035)
  - `to_euler -> get_euler` + `get_euler_with`
  - `to_quat -> get_quaternion`
  - `from_quat -> from_quaternion`

    - `scale -> get_scale`
    - `new_looking_at -> looking_at`

The main idea behind those changes:

- Use `intersect` for methods returning the intersection, and `intersects` for boolean checks.
- Use `contains` for containment checks, and `has` for checking the presence of a property.
- Use `get_` in situations where only a part is extracted (e.g. `get_euler` instead of `to_euler`, because Basis has non-rotation components).
- Be closer to Godot terminology.

Signals

While v0.3 brings large-scale changes to signals, they are almost fully backwards compatible. The main breaking change is that declaring `#[signal]` now requires the class to have a `Base<T>` field. If you don't need the new type-safe signal API, you can opt out with `#[godot_api(no_typed_signals)]` on your `impl` block.

Relatedly, the `ConnectFlags` enum has been reclassified as a bitfield rather than regular enum. This enables `|` operations, among others.

Final classes

Godot prevents certain classes from being inherited outside the engine, for example `FileAccess` or `IP`. This used to cause runtime errors whose messages weren't always clear.

godot-rust v0.3 properly declares such classes as "final", which has the following implications:

- Rust classes can no longer inherit from them (doing so results in a descriptive compile error).
- The associated interface trait (e.g. `IFileAccess`, `IIp`) no longer exists, since you can neither construct a base nor implement virtual methods. In total, 118 `I*` traits have been removed.
- API Docs clearly state which classes are final, and elaborates the relation between an

interface trait and its base interfaces.

Rename of unsafe virtual methods

Some virtual methods declare raw pointers in their parameters or return types, for example:

```
trait IAudioStreamPlayback {
    unsafe fn mix(&mut self, buffer: *mut AudioFrame,
                  rate_scale: f32, frames: i32) -> i32;
}
```

In the future, we would like to transition these to safe methods. To enable a gradual migration period with coexisting safe and unsafe variants, we renamed all such methods now, by appending the `_rawptr` suffix:

```
trait IAudioStreamPlayback {
    unsafe fn mix_rawptr(&mut self, buffer: *mut AudioFrame,
                         rate_scale: f32, frames: i32) -> i32;
}
```

Details can be seen in pull request #1174.

Entry point

In case you use `#[gdextension(entry_point = ...)]`, the key has now been renamed to `#[gdextension(entry_symbol = ...)]`. This is consistent with the key name in the `.gdextension` file.

Migrating to v0.2

This chapter will guide you through the changes from godot-rust version 0.1 to 0.2. See also our November dev update for a feature overview, and our changelog for a detailed list of modifications. Breaking changes are marked as such in the changelog, and you can navigate to the respective PRs to get in-depth information.

Godot version support

With godot-rust 0.2, Godot **4.3** is supported out of the box.

Godot 4.0 is no longer supported. We're the last binding to abandon it, after 1.5 years. 4.0 offers no compatibility with today's GDExtension API, not even among patch versions, so using it at this point is not recommended.

Argument passing

The biggest breaking change in 0.2 is the way arguments are passed to Godot APIs. What used to be pass-by-value everywhere, has now more nuance, while making calling code more concise.

The following table goes into different kinds of arguments and corresponding call expressions.

Argument type	Parameter type (v0.1 → v0.2)	v0.1 call	v0.2 call
`i32` (`Copy`)	`i32`	`func(i)`	`func(i)`
`GString`	`GString` → `impl AsArg<GString>`	`func(s)` `func(s.clone())`	`func(&s)`
`&str`	"	`func("str".into())`	`func("str")`
`String`	"	`func(s.into())`	`func(&s)`
`StringName` `NodePath`	"	`func(s.into())`	`func(s.arg(`
`Gd<Node>`	`Gd<Node>` → `impl AsObjectArg<Node>`	`func(g.clone())`	`func(&g)`
`Gd<Node2D>`	"	`func(g.clone().upcast())`	`func(&g)`

Most of them are straightforward, noteworthy is maybe `arg()` as a way to convert between the 3 Godot string types. This conversion is done explicitly, because it's much less obvious than conversion from `String` / `&str` but can have significant performance implications due to allocations, re-encoding and synchronization overhead. It also makes you more aware of the string type in use.

Removed APIs

See also #808. Noteworthy changes:

- Renamed crate feature `custom-godot` → `api-custom`.
- Godot enums now use `SHOUT_CASE` enumerators. `PascalCase` aliases have been around for some time, but not anymore.
- `GString::chars_checked()` and `GString::chars_unchecked()` have been removed. There's no more need for unsafety; use `GString::chars()` instead.
- Several collection methods have been migrated, e.g. `Dictionary::try_get()` → `get()`, `Packed*Array::set()` → `[]`.
- Removed ancient pre-0.1 modules `godot::engine`, `godot::log`.
- The `#[base]` attribute is no longer allowed.

Miscellaneous

- Some use cases now require a `Base<T>` field that wasn't previously needed, e.g. `OnReady<T>`.
- Virtual functions that are semantically required by Godot are now also required in the `I*` interface trait in Rust. That is, you must override them in your `impl` block.
- There are new validations around `Export` and `#[class(tool)]`, which no longer accept previously compiling (but broken) code.